Architecture & Infrastructure

Polycarpic — a virtual Single Family Office

Owner: `architect` agent

Status: Draft v1.4 — SECURITY v1.1 re-folded: co-owner approval and the founding owner, the ledger hash chain, the graph / stake / govern roles, purge of a cap-table holder, home-box TLS modes (Principal and SecEng, 2026-11-07)

Traces to: PRD v1.1 (rulings of 2026-11-06), kickoff record **Last updated:** 2026-11-07

CONTENTS

1. System Context
2. Components & Layer Model
3. Data Model & Database Invariants
4. Data Flow
5. Tech Stack & Rationale
6. Plan-phase Research: GL Build-vs-Adopt, Migration Tool
7. Deployment Topology
8. CI/CD Pipeline
9. Integration Points
10. Trade-offs & Alternatives
11. Milestone Map
12. Open Questions

Changes in v1.4 (2026-11-07)

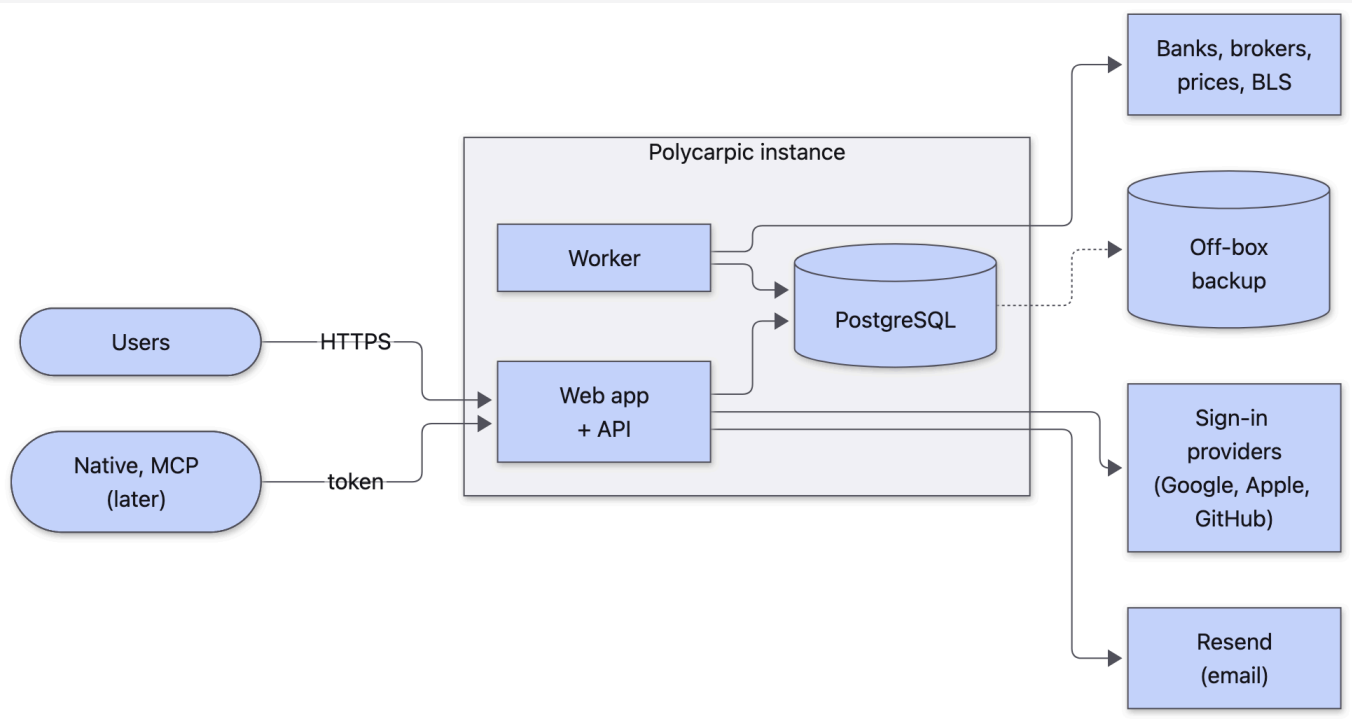
Re-fold of SECURITY v1.1 §12.2 (A-1 … A-22) and the Principal's rulings of 2026-11-07 (DECISIONS.md).

- **Already folded before this pass (checked, unchanged):** A-1 to A-15. A-1, A-3, A-10, A-12 in §3.3; A-2 in §3.4; A-4, A-5 in §3.2; A-6, A-8, A-11 in §3.1; A-7 in §2.2, §3.1 and §7; A-9 in §2.3; A-13 in §6.2; A-14 in §3.5; A-15 in §7.
- **New roles** (§3.3): `poly_graph` (A-16, A-21), `poly_stake` (A-18), `poly_govern` (A-20, A-22), and `poly_journal`, which the chain needs and SecEng has not yet seen (OQ-30). `app.parents_of` moves to `poly_rollup` (A-17).
- **Co-owner approval and the founding owner** (P5b, R-19; A-20, A-22): new §3.1b; `change_request`, `change_approval`, `entity.founding_owner_user_id`; successor step in §3.4a.
- **Ledger hash chain** (P4, C-24): `journal_chain`, §3.2.9, the weekly verifier in §7.4. POLY-M2 scope.
- **Ancestor-invite notice** (P5, A-21) in §3.3b. **Any date range** for roll-up figures, with the consent wording as a security requirement (P2, C-25) in §3.3a, §3.3c, §4.5.
- **Purge of an entity that holds units elsewhere** (A-19): new §3.4b. The holder's name on the other cap table: "Former holder" plus an optional owners' label (Principal, OQ-33).
- **Home-box TLS** (P7, C-76): §7.8 settled on two modes. "Invite as viewer" timing settled in §3.1a (ux-designer, PR #78).
- **Open questions:** OQ-19, OQ-24, OQ-25 closed; OQ-30 to OQ-32 added; OQ-33 added and closed (Principal, option c).

1. System Context

The system and the actors around it. C4 context level.

Polycarpic is a single self-hosted web application backed by one PostgreSQL database. Users (the Principal, household members holding a membership grant, later native iOS/macOS clients and a user-scoped MCP server) reach it over HTTPS. Inside, a genuine double-entry general ledger lives in Postgres under row-level security keyed on the *membership* join. Outbound, the system pulls transactions from bank/brokerage aggregators through a provider-agnostic adapter interface (a deterministic fake first, then CSV/OFX files, direct broker APIs, Plaid and SimpleFIN), end-of-day prices and the CPI-U series as reference data, and sends transactional email through Resend. Identity is local (BetterAuth in our own Postgres) with sign-in through outside identity providers (IdPs): Google, Apple and GitHub confirm who a user is, so the user has no separate password with us, and BetterAuth then starts a session of our own. Nothing about a user's books leaves the box except through a provider call the user initiated or an owner-initiated export.

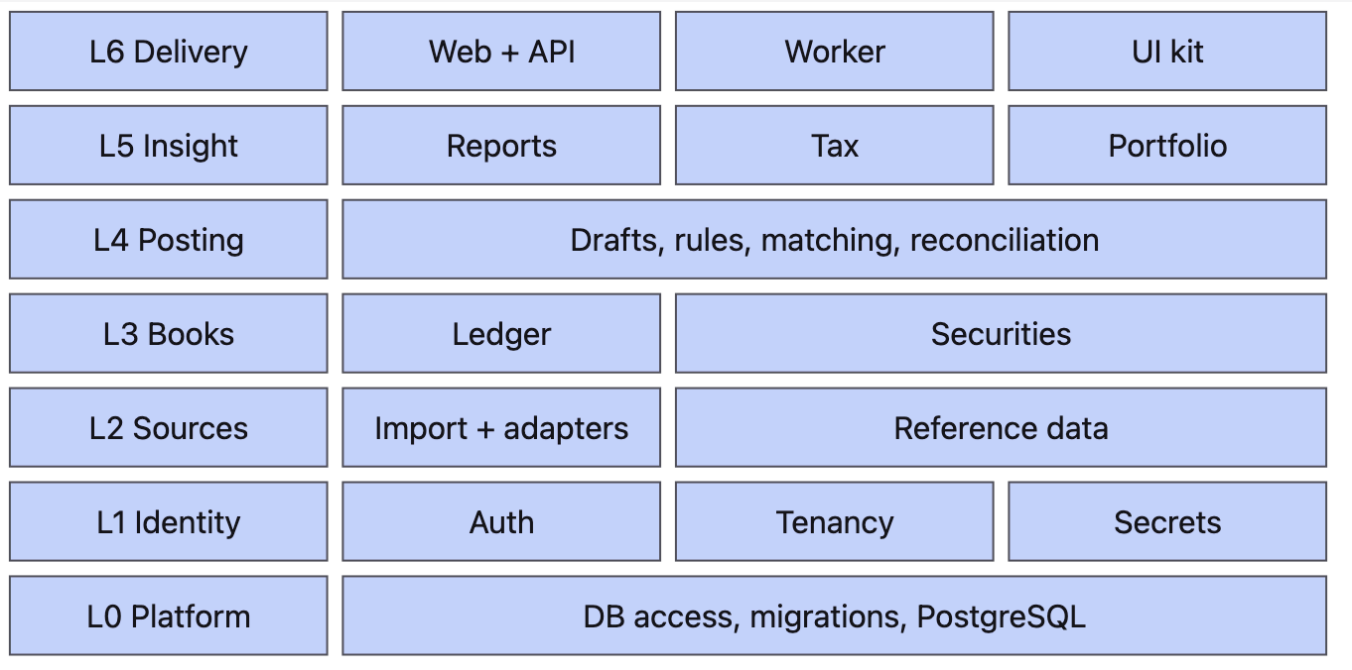


2. Components & Layer Model

Layers depend only downward. Each layer is one or more workspace packages with its own tests; the boundary is enforced by lint, not convention.

2.1 Layer model

Kickoff §3 asks for layers defined before details, depending only downward, with narrow interfaces. The model below is the contract. **Rule:** a package may import only packages in a strictly lower layer or in its own layer's shared `contracts`; `packages/*` never import SvelteKit; only `packages/db` opens connections, and only through `withTenant()` / `withSystem()` (see §3.3). Enforced by `eslint-plugin-boundaries` (or `dependency-cruiser`) in CI.



Why import sits below the books. Kickoff §2.6 calls import "the lowest non-infrastructure layer". That holds for the *raw* side — adapters, the raw-transaction store and de-dup depend on nothing domain-level, so they sit at L2. Confirmation, rules and transfer matching need both raw rows and the ledger, so they are split into `posting` (L4). This keeps "raw data arrives first" true while dependencies stay strictly downward. The ledger never imports the importer: a journal entry carries an opaque `source_ref`; the two-way link and the posted-status flip happen inside one database transaction owned by `posting`.

2.2 Components

Component	Responsibility	Owner agent	Exposes	Consumes
`packages/db`	Owns the schema, migrations, Postgres roles, and the only path to a connection — a tenant- or system-scoped transaction.	backend-lead	`withTenant(userId, fn)`, `withSystem(job, fn)`, Drizzle table objects, migration runner	PostgreSQL
`packages/auth`	Configures BetterAuth (sessions, OAuth, MFA, bearer tokens) against our Postgres; maps a request to a user id.	backend-lead	`getSession(request)`, auth route handler	db, Resend, OAuth IdPs
`packages/tenancy`	Entities, entity types (base vs parent kind), the many-to-many roll-up junction (`entity_rollup`), memberships, the owner/manager/viewer permission matrix as data, the cap table (`equity_total`, `equity_holding`; §3.1a), co-owner approval and the founding owner (`change_request`, `change_approval`; §3.1b), and the Person-entity-at-sign-up hook (§3.1).	backend-lead	entity/membership service, `can(role, action)`, `capTable(entity, asOf)`, `stake(holder, entity, date)`, `requestChange(entity, kind, payload)`, `decideChange(id, decision)`	db
`packages/secrets`	Seals and opens aggregator credentials with HPKE (X25519 / HKDF-SHA256 / AES-256-GCM; SECURITY §6.3). `web` only seals, under the public key; only `worker` holds the private keyring and can open.	backend-lead (SecEng sign-off)	`seal(plain, aad) → {enc, ciphertext, key_id, alg_version}` (web); `open(sealed, aad)` (worker only, enforced by lint)	public keys (web); keyring file (worker)

Component	Responsibility	Owner agent	Exposes	Consumes
`packages/import` + adapters	Provider-agnostic adapter interface; fetches, normalizes, hashes and stores raw transactions and reported balances.	backend-lead	`ImportAdapter` interface, `ingest(connectionId, range)`	db, secrets, provider APIs
`packages/refdata`	Global reference series — EOD prices, CPI-U, per-jurisdiction per-year tax tables — behind provider interfaces.	backend-lead	`PriceProvider`, `SeriesProvider`, lookups	db, BLS, price provider
`packages/ledger`	Chart of accounts by nature, custodial accounts and journal entries; posting and reversal; balances as of a date. DB enforces balance and immutability.	backend-lead	`postEntry()`, `correctEntry()`, `balances(asOf)`	db, tenancy
`packages/securities`	Lots held at custodial accounts, lot matching, quantities, book value adjustments (return of capital, amortization), holdings as of a date. Writes each lot in the same transaction as its entry (§3.6).	backend-lead	lot service, `holdings(asOf)`	db, ledger, refdata
`packages/posting`	Turns raw rows into drafts, applies rules, matches transfers, confirms drafts into entries, reconciles custodial balances.	backend-lead	draft queue, `confirm(draftId)`, rule engine, matcher, reconciliation	import, ledger, securities

Component	Responsibility	Owner agent	Exposes	Consumes
`packages/reports`	Evaluates declarative report definitions (rows by account range / tag / type, formula lines, period or entity columns).	backend-lead	`run(definition, params) → ReportTable`	ledger, securities, portfolio, tax
`packages/tax`	Running annualized accrual, safe-harbor derivation, period-cutoff freezes, reporting-period closes, pass-through allocation by the cap table (§4.5), Form 2210 / Schedule AI export.	backend-lead	`estimate(entity, jurisdiction, asOf)`, `freeze(period)`, `close(entity, periodEnd)`, `sinceClose(entity, asOf)`	ledger, refdata, tenancy
`packages/portfolio`	NAV, allocation vs. target, inflation adjustment of any time series.	backend-lead	`nav()`, `allocation()`, `holdingsByClass(entity, node, asOf)`, `deflate(series, base)`	securities, refdata
`packages/ui`	Owned shadcn-svelte components, LayerChart wrappers, design tokens imported from `docs/DESIGN/`.	frontend-lead	Svelte components, token CSS (and a Swift token export later)	DESIGN tokens
`apps/web`	SvelteKit pages (server `load` calls packages directly) and `/api/v1` thin route wrappers with Zod schemas → OpenAPI.	frontend-lead (pages) / backend-lead	HTML, `/api/v1`, `openapi.json`	every L1–L5 package, ui

Component	Responsibility	Owner agent	Exposes	Consumes
		(routes)		
`apps/worker`	Schedules the idempotent job units (import poll, EOD prices, CPI fetch, tax period freeze, tax close prompt; Cash-in-Transit aging is computed on read, not a job).	backend-lead / devops-engineer	none (outbound only)	job units in L2–L5

2.3 Interface contracts worth pinning now

- **`ImportAdapter`** — `id`; `capabilities` (transactions, balances, holdings, trades); `connect(input) → sealedCredential`; `listAccounts(cred)`; `fetchTransactions(cred, account, range) → RawTxn[]`; `fetchBalances(cred, account) → ReportedBalance[]`; `revoke(cred)` (provider-side revocation, PRD §5). The fake source implements exactly this and is registered as `fake`; switching an instance to SimpleFIN is configuration. A CSV/OFX file adapter implements the same interface with no stored credential (SecEng's preferred credential-free path).
- **`RawTxn`** — provider id, provider account id, posted date, amount (decimal string), currency, payee/memo, provider transaction id (nullable), provider payload (JSON). Before storing the payload, the adapter redacts full account numbers, PANs and any credential fields (SECURITY A-9). The de-dup hash is computed by `import`, never by the adapter: `sha256(provider, provider_account_id, provider_txn_id ?? (date, amount, normalized payee, per-day ordinal))`, enforced by a unique index.
- **`PriceProvider` / `SeriesProvider`** — `fetch(symbolOrSeries, range) → Observation[]` with a `source` and a priority; CPI-U reuses the price-series pattern (PRD §5). Fake EOD provider: seeded random walk per ticker.
- **Job unit** — `run(ctx, params): Promise<Result>`; must be idempotent, short (< 60 s per invocation, chunked otherwise), and triggerable by either scheduler (§7).
- **Screen DTOs** — each page's server `load` returns a DTO type exported by the owning package. The same types back `/api/v1`, so the web UI and native clients share one contract.

The list of screens comes from DESIGN (ux-designer); DTOs get pinned per screen in each package's design file (kickoff §2.13).

2.4 Screen → DTO ownership

The sidebar has seven items (Principal, DESIGN pass 5): **Dashboard**, **Transactions**, **General Ledger**, **Custodial Accounts**, **Allocation**, **Reports**, **Entities**.

- **No persistent header** (DESIGN pass 6). The top of the scroll area is one row: sidebar toggle, page title and breadcrumbs, the entity selector, and a per-page "more" menu holding that page's global actions. Settings, Search and Notifications remain surfaces with *proposed* entry points (rows below). The AI/agent placeholder has no v1 affordance.
- **Sidebar foot:** the avatar opens the user popover (profile, MFA, Log out; proposed: Settings and Notifications).
- **Gone:** the earlier Securities and Import screens. Lots now live inside Custodial Accounts, and import setup lives on the provider item.

Screen layouts are in DESIGN.

A screen that spans packages gets a composite DTO assembled in `apps/web` from package DTOs. A panel whose package hasn't shipped yet renders the empty-state pattern.

Screen / surface	DTO(s) → owning package	First milestone
Dashboard	`NetWorthSeries`, `CashFlowByPeriod`, `RecentActivity` → ledger; `TaxPosition` → tax; `AllocationVsTarget` → portfolio	M2 (panels fill in at M3, M4 and 1.0)
Parent views: gross / economic toggle (Dashboard, Reports, Entities on a parent)	`RollupTotals` (gross, §3.3a) and `EconomicRollup` (the same rows and grain, each weighted by the viewer's stake, plus the viewer's economic net worth; §3.3c) → reports	POLY-1.1

Screen / surface	DTO(s) → owning package	First milestone
Transactions: three accordion cards (Exceptions, Pending, Posted), each with a "show N" limit	`ExceptionList` (aged CIT, link-health banners) → posting + import; `PendingInbox` (draft with locked custodial leg + suggested offset) and `ImportStatus` → posting + import; `PostedJournal` (paged whole entries, correction badges) → ledger	M2 (CIT exceptions in M3)
Journal-entry modal (header + legs), used for split, group, manual "+ New entry" and "Correct entry"	`EntryDraft` in, `GateResult` out → posting (§3.2a)	M2
General Ledger: chart-of-accounts tree	`ChartTree` (category → nature class → custodian sub-account, balances, custodial marker) → ledger	M2
General Ledger: account register page	`AccountRegister` (lines on one account, date order, running balance, link to entry) → ledger	M2
Custodial Accounts	`CustodialAccountList` (book, cash, market and unrealized per §3.6, `ReconciliationStatus`, link-health badge) → posting + import; `LotsCard` (per security: lots, short / long / total unrealized gain, per lot and summed) → securities	M3 (lots card at M4)
Provider-item modal ("+ New" on	`ProviderItem`: source type (aggregator / broker API / file / manual), discovered accounts with a mapping to custodial	M2 (fake + file); direct,

Screen / surface	DTO(s) → owning package	First milestone
Custodial Accounts)	accounts, token refresh, unlink with the revocation-lag note → import	Plaid and SimpleFIN follow per §9
Sync commands (Transactions, Custodial Accounts) and "Import file"	`SyncRequest` → a job in import; on Custodial Accounts it also pulls balances and positions	M2
Allocation: treemap, Actual vs. target card, target editor, in-place holdings popover	`AllocationVsTarget`, `AllocationTree`, `TargetEdit` (two levels; sub-targets are whole-portfolio percent and must sum to their class) and `HoldingsByClass` (one class or sub-category across every custodial account, grouped by account, with lots and a total) → portfolio. The top level reads the nature accounts across custodials (book) or lots by class (market); sub-categories come from `security_allocation` (§3.6). One `withTenant` query serves a class spanning accounts.	M4
Reports	`ReportTable` (Balance Sheet, Income Statement, Cash Flow; NAV and NAV delta), trade blotter and Form 8949 view (§3.6) → reports; `TaxCloseStatus` ("last closed through *date*") → tax; empty state until then	M5
Entities	`EntityList`, `MembershipList`, invite command → tenancy	M1
Entities: cap table, beside the membership list	`CapTable` (total units and holder rows as of a date, each with its access role or none, the outside-owners remainder, and the dated history), `HolderCandidates` (owner-access members first, then the editing user's other Persons, Trusts and Businesses) and `CapTableChange` in (one effective date, a new total and/or holder units; a holder with no access carries an optional viewer invite) → tenancy (§3.1a)	M1 (pass-through allocation reads it at POLY-1.0)

Screen / surface	DTO(s) → owning package	First milestone
Settings (proposed entry: avatar popover)	`ImportSchedule` (per-entity batch schedule and options) → import; app preferences and notification preferences → tenancy	M2
Entity selector (top row)	`EntityList` (the user's memberships only) → tenancy. The selected entity scopes every screen's DTO request, and RLS still decides what returns.	M1
Search (cross-cutting; proposed entry: keyboard shortcut and the page "more" menu)	`SearchResults` grouped by kind: accounts (name and number), payees and memos on posted entries and pending drafts, and report views → a `search` query service in `apps/web` over ledger, posting and reports. It runs inside `withTenant` over Postgres full-text indexes (`tsvector` on payee, memo and account name), so results are RLS-filtered with no separate index to secure. Viewers see results without masked numbers or raw payloads.	M3 (accounts and entries); report views at M5
Notifications (cross-cutting; proposed entry: avatar popover, with an unread badge on the avatar)	`NotificationList` plus an unread count → a `notification` table (entity-scoped, per recipient, RLS-covered). Rows are written by the producers of the alerting events in SECURITY: link health (`token_expired`, `principal_invalid`, `stale`), aged Cash-in-Transit items, an auto-post rule switched off, membership changes, a purge pending, failed import batches, tax due dates, and a tax close due. Delivery is in-app; email via Resend follows the user's preferences.	M2 (import and link health); others as their producers land
AI / agent (v2)	None in v1, and no v1 affordance. It is a placeholder for the future user-scoped MCP / agent surface (PRD §3 non-goal), which would be a client of `/api/v1`.	v2 placeholder

Model consequence: import status on Transactions comes from an `import_batch` row per ingest run, holding status and counts. Rows skipped by `ON CONFLICT DO NOTHING` are counted into the batch as duplicates instead of vanishing silently (§3.1).

Permissions (mirrors the SECURITY §4.2.1 matrix @ 6fb5385):

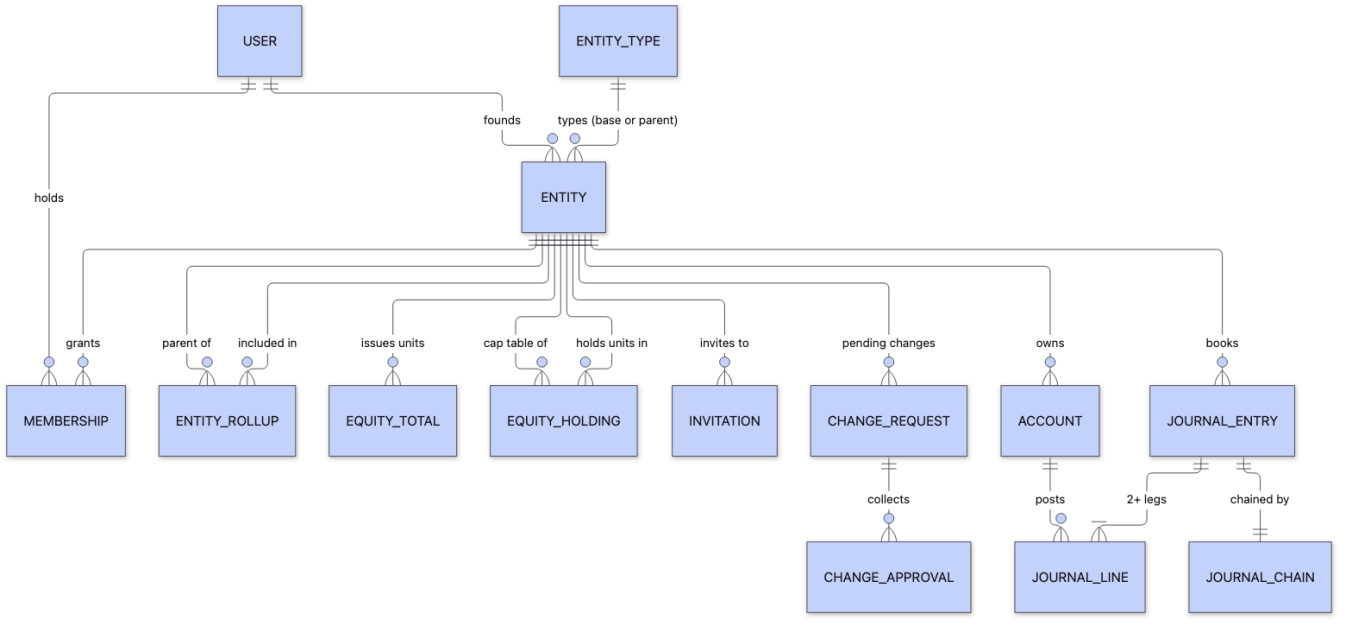
Action	Viewer	Manager	Owner
New provider item, map accounts, refresh token, unlink. Managers act on their own links only, with step-up re-auth; a mapping change writes an `audit_event`.	—	✓	✓
Sync, Import file. Sync enqueues a worker job, and the credential is never opened in web. Rate-limited to the INFRA minimum interval.	—	✓	✓
Post, Split, Group, + New entry, Correct entry	—	✓	✓
Rule auto-post opt-in. `rule.auto_post_by_user_id` is the principal.	—	✓	✓
Settings: per-entity import schedule, floored at the INFRA minimum interval	—	✓	✓
Edit the cap table (total units, holders, effective dates). It is entity metadata, like the tax classification; every change notifies the holders it touches (§3.1a).	—	—	✓
Read every screen above, the cap table included (account numbers masked to last 4; no raw payloads)	✓	✓	✓

Viewers see none of the action buttons. The server enforces every row regardless of what the UI shows.

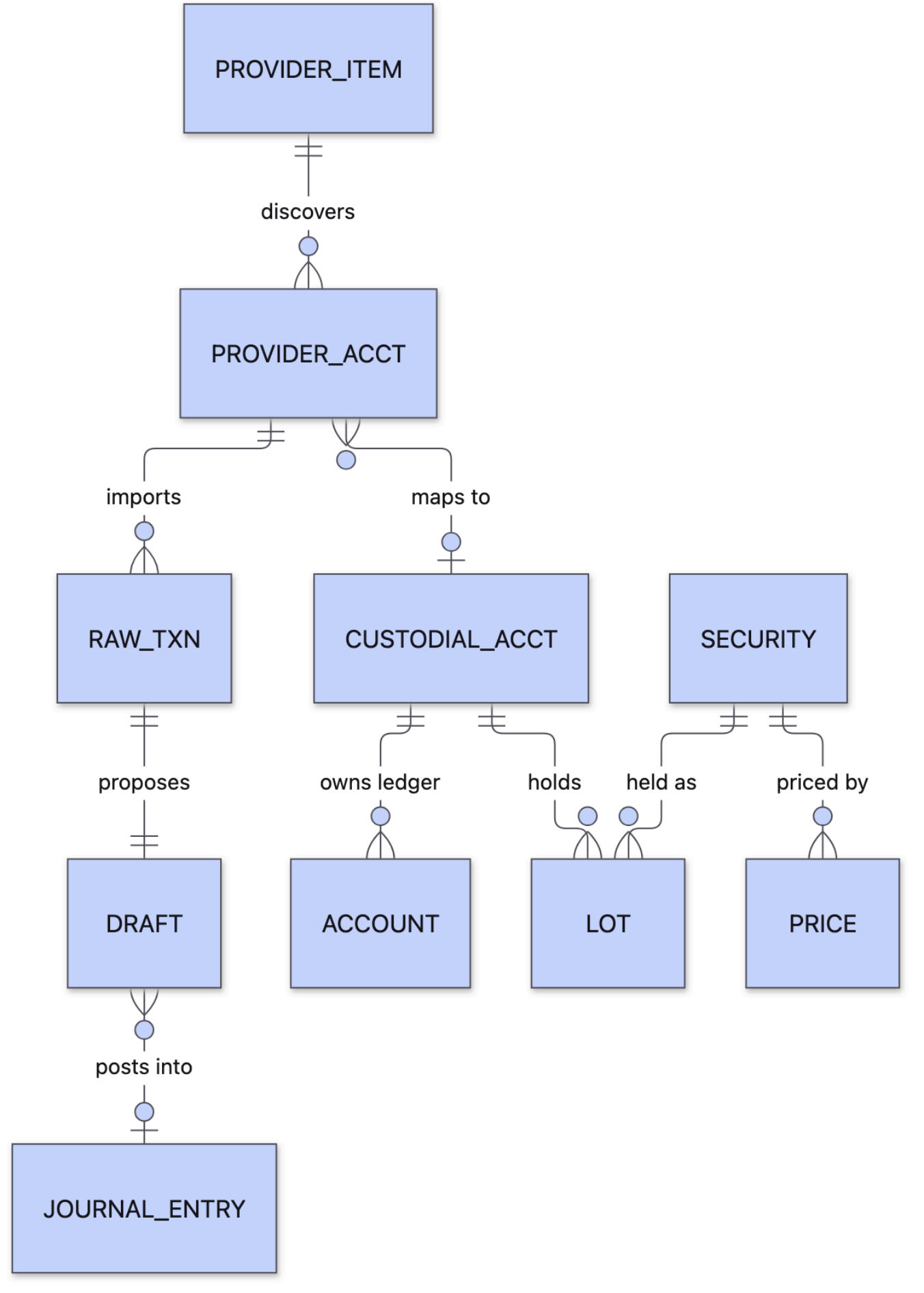
3. Data Model & Database Invariants

Logical model at ARCH grain; column-level design lives in each package's design file.

3.1 Core entities



Tenancy, the cap table and the ledger, above. Access (`MEMBERSHIP`, user to entity) and equity (`EQUITY_HOLDING`, entity to entity) are separate edges. On a multi-owner entity, changes to either pass through `CHANGE_REQUEST` (§3.1b). Import and securities, below. The table that follows lists every table group.


PROVIDER_ITEM
discovers
PROVIDER_ACCT
imports
maps to
RAW_TXN
CUSTODIAL_ACCT
SECURITY
proposes
owns ledger
holds
held as
priced by
DRAFT
ACCOUNT
LOT
PRICE
posts into
JOURNAL_ENTRY

Keys: every tenant-visible primary key is a UUID, never a sequential integer, so ids leak neither counts nor ordering across tenants (SECURITY A-6). Tenancy columns (`entity_id`) and actor columns are not updatable.

Table group	Tenant key	Mutability	Notes
`user` (BetterAuth)	self	mutable; deletion tombstones	Account deletion turns the user row into a tombstone ("deleted user") and removes credentials, sessions and memberships. The row itself is never deleted, and entries are never UPDATEd. `actor_id` FKs are `ON DELETE NO ACTION` , and BetterAuth's `deleteUser` is overridden (SECURITY A-8). The tombstone transaction refuses while the user is still the founding owner of any entity that has another owner (§3.1b). After the bootstrap user, an invitation is the only way to create a user, and no configuration flag reopens sign-up (Principal, 2026-11-07, P6).
`entity` , `membership` , `entity_type`	entity id; types: `owner_user_id` or null (system)	mutable; tenancy columns not updatable	Five system types in two kinds (Principal, 2026-11-02). Any entity may hold accounts and books (Principal, 2026-11-06): a Household's joint checking account lives in the Household. *Base* (Person, Trust, Business) never have children. *Parent* (Household, Holding) also roll up whole base entities and other parents, never individual accounts of theirs. Holding is a generic umbrella, not a household. `entity_type.kind` is `base` | `parent` , and a user-defined type declares its kind. **Every user gets a Person entity and an owner membership at account creation**: BetterAuth's `user.create.after` hook calls `tenancy.provisionPerson(userId)` , which inserts the entity (with `self_user_id` set, marking it as that user's own Person; §3.1a), the owner membership and the seeded accounts. Whether that hook shares the user insert's transaction is unmeasured (OQ-16); until the M1 spike confirms it, the provisioning is idempotent and the session loader re-runs it for any user holding no owner membership on a Person, so no signed-in user is ever without one (this is why the entity selector is always present). Tax-classification attribute on `entity` , settable on every entity (individual; trust grantor / non-grantor; business pass-through / C-corp; for a parent, `joint filers` or `pass-through to members` , the default for a new Household and, by extension, a new Holding; §4.5). User-defined types readable by creator plus holders of any membership on an

Table group	Tenant key	Mutability	Notes
			entity of that type — the narrow cross-tenant metadata exception SecEng flagged. **`founding_owner_user_id`** NOT NULL, set to the creator (for a Person, its own user), transferable only through `app.transfer_founder`; a deferred constraint trigger refuses any commit where the founder does not hold owner (§3.1b, SECURITY A-22). **`purged_at`**: a purged entity's row stays as an id-only tombstone, so cap-table rows on other entities still resolve (§3.4b).
`invitation`	entity id (null for an operator invite)	insert; `accepted_at` / `revoked_at` only	A live invitation: email, role, inviter, single-use token hash, expiry. On a multi-owner entity the row is written only when its approved `change_request` applies, and the email goes out after that commit; until then the invite exists only as the request's payload, visible to owners alone (§3.1b). Inserting a row on a parent-kind entity fires the ancestor-invite notice (§3.3b). The operator command `polycarpic user invite <email>` writes a row with no entity: it shares nothing, is audited, and the invitee gets only their own Person, so neither the approval path nor the notice applies (P6).
`change_request`, `change_approval`	entity id	append-only except `change_request.status` (trigger)	Co-owner approval (§3.1b, SECURITY §4.2.2, A-20). Request: entity, `kind` (`invite` | `role_raise` | `owner_remove` | `owner_demote` | `cap_table`), target user (nullable), payload, the entity version it was made against, requester, created, expiry, status (`pending` | `applied` | `declined` | `expired` | `withdrawn`), `forced`. Approval: request, approver, decision (`approve` | `decline`), time. Readable by the entity's owners only. Status flips only through `poly_govern` definers. Purged with the entity; their `audit_event`s survive.

Table group	Tenant key	Mutability	Notes
`entity_rollup` (the parent link)	parent entity id	insert / delete only; no UPDATE	Many-to-many: `(parent_entity_id, child_entity_id)`, both FK to `entity` `ON DELETE CASCADE`, primary key on the pair, `CHECK (parent_entity_id <> child_entity_id)`. A trigger requires the parent's type to be of kind `parent`; the child may be of either kind, so parents nest to any depth (Principal, 2026-11-06). A base entity never has children. Any entity may sit in any number of parents. The graph stays acyclic and within the sanity limit by the trigger in §3.3b. **Insert** requires owner on both the child and the parent (SECURITY A-11 / C-19, kept unchanged for the junction; it is what stops T-16 injection). **Delete** requires owner on the parent or on the child (Principal, 2026-11-06). **Read** iff the user holds any membership on the *parent*, or owner on the *child*. A child's owner gets the parents' names and nothing else about them, through `app.parents_of(entity_id)`, which lists every parent above with the direct ones marked. It is owned by `poly_rollup` and reads under that role's second predicate, the ancestors of entities the caller owns, and sits on the definer allowlist (SECURITY A-17). The row never grants a row of the child (§3.3a).
`account`	entity id	mutable until first posting; then name/parent only	Number, name, parent, class (asset / liability / equity / income / expense), normal side, currency, `nature_class` (nullable; §3.6), `custodial_account_id` (nullable), default tax-treatment tag. Seeded per entity: the four nature classes under Assets, Cash in Transit, Suspense, Opening Balance Equity, realized-gain income accounts by holding period (§3.6), and the per-jurisdiction tax accounts listed in §4.5. Prepaid-tax accounts belong to a `custodial_account` for the tax authority (IRS, the state).
`entity_journal_`	entity id	one row per entity;	Holds `last_number` for the Journal ID (§3.2.7) and the chain head, `head_number` and `head_hash` (§3.2.9). Created with the entity and purged with it. `poly_app` and `poly_worker` hold no write grant on it; only the `poly_journal` triggers write it.

Table group	Tenant key	Mutability	Notes
`counter`		written only by the trigger below	
`journal_chain`	entity id	**append-only** (trigger)	One row per posted entry: `(entity_id, journal_number, entry_id, prev_hash, entry_hash, alg_version)`, unique on the first two. Written only by the deferred chain trigger (§3.2.9); the entry itself is never UPDATEd to hold its hash. Readable by the entity's members. Purged with the entity; the final head goes into `entity.purged` (SECURITY C-24).
`custodial_account`	entity id	mutable; tenancy columns not updatable	A real account at an institution: institution, account type, number mask, currency, provider link. It replaces the old `is_custodial` + custodian flag. It owns ledger accounts one-to-many, and lots point at it (§3.6).
`journal_entry`, `journal_line`	entity id (denormalized onto	**append-only** (trigger)	Entry: `journal_number` (per entity, gapless; §3.2.7), posting date (exactly one), description, `actor_kind` (user / rule / system) + `actor_id`, `reverses_entry_id`, `replaces_entry_id`, `source_ref`. Line: account, `currency`, `native_amount`, `base_amount`, `fx_rate`, category, tax-treatment tag, memo. Amounts are `numeric`, signed (debit > 0); never JS floats.

Table group	Tenant key	Mutability	Notes
	the line)		
`import_batch`, `raw_transaction`, `reported_balance`, `reported_position`	entity id (via linked account, denormalized)	raw rows insert-only; `status` draft → posted (then frozen by trigger); batch mutable while running	Unique de-dup hash. Full provider payload kept for audit; `payload` is readable by manager+ only, never by viewers (SECURITY §4.2.1). Batch holds run status and new / duplicate / needs-category counts (§2.4).

Table group	Tenant key	Mutability	Notes
`draft`, `rule`, `transfer_match`	entity id	mutable until confirmed	One draft per raw row. Several drafts post into one entry when the user groups them (Flow B), so draft → entry is many-to-one. A split (Flow A) is one draft posting into one multi-leg entry. A posted draft carries exactly one `entry_id`. The draft carries the proposed lines (JSON), with the custodial cash leg locked (§3.2a), and an actor. Rule carries `auto_post` opt-in (default false). Opting in counts as the user's confirmation (Principal, 2026-09-30). `auto_post_by_user_id NOT NULL` records who opted in. If that user drops below manager, `auto_post` flips off and an `audit_event` is written (SECURITY §4.2.1). An auto-posted row is immutable like any other, and "Correct entry" applies to it.
`provider_account`	entity id	mutable	One account discovered on a provider item (a Fidelity brokerage, a Schwab checking account). It carries a `tracked` flag and a nullable mapping to a `custodial_account`, never to a ledger account, set in the modal's mapping step. Raw transactions, reported balances and reported positions hang off it.
`provider_connection` (the UI's "provider item")	entity id + linking user	mutable	The first-class record for one provider login. It owns the discovered `provider_account` rows. Source type: `aggregator` | `broker_api` | `file` | `manual`. File and manual items hold no credential. Link health is `ok` | `token_expired` | `principal_invalid` | `stale`. Sealed credential only: `enc`, `ciphertext`, `key_id`, `alg_version`, with AAD = (connection, entity, provider, key_id, alg_version) (SECURITY §6.3). `poly_app` may INSERT and UPDATE those columns but never SELECT them; only `poly_worker` reads them. `linked_by_user_id NOT NULL` is the principal that job writes run under. A `principal_invalid` state is set when that user's membership is revoked or their account is deleted; it surfaces on the dashboard and writes an `audit_event`, so it never fails silently (SECURITY §4.4). On unlink, `web` enqueues a revoke job; `worker` opens the credential, revokes at the provider, then deletes the row.
`security`, `pri`	global ref ere	writable by syste	Price: unique (security, date, source) + source priority (borrowed from mosko-fintech). Tax table: jurisdiction × filing year → due dates, cutoffs, multipliers, cumulative percentages — a new year is a data row.

Table group	Tenant key	Mutability	Notes
`ce_observation`, `cpi_observation`, `tax_table`	nce data	m jobs / operator import only	
`allocation_node`, `security_allocation`, `allocation_target`	entity id	mutable	The allocation hierarchy (tier, class, sub-category), seeded per entity from the DESIGN classes of 2026-11-05. A tier is a label; targets sit on class and sub-category nodes as whole-portfolio percent. The four classes are the nature classes of the chart (§3.6), so they are fixed; sub-categories are editable. `security_allocation` maps a security to a sub-category for that entity, since `security` itself is global, and its parent class is the security's nature class. A reported holding with no mapping shows in Allocation as a visible gap, never silently excluded.
`lot`, `lot_match`,	entity id	append-only	A lot references `custodial_account_id` and the security, never a ledger account. Lot-match is an append-only many-to-many junction with a `match_seq`; the latest batch is current (borrowed from mosko-fintech 032). It stores proceeds and per-lot basis as values, and a `kind` of `sale` or `transfer` (§3.6).

Table group	Tenant key	Mutability	Notes
`lot_adjustment`			
`tax_period_freeze`	entity id	append-only	Annualized income, tax, cumulative required, paid — per jurisdiction per period; source of the 2210 / Schedule AI export.
`tax_close`	entity id	append-only	One row per reporting-period close: jurisdiction, filing year, `period_end`, revision, `accrued_to_date`, `entry_id`, unique on the first four. It is the record of what has been accrued for a filing year (§4.5). The cadence is `entity.tax_close_cadence`: `monthly`, the default, or `quarterly`.
`equity_total`, `equity_holding` (the cap table)	entity id	**append-only** (trigger)	Replaces `tax_allocation_share` (Principal, 2026-11-07). `equity_total`: `(entity_id, total_units, effective_from)`. `equity_holding`: `(entity_id, holder_entity_id, units, effective_from)`. Holders are entities of kind Person, Trust or Business, never users and never parents. Independent of `membership`: any role or none may hold units. `holder_entity_id` FKs `entity` `ON DELETE NO ACTION`; a purged holder stays as a tombstone, and `equity_holder_label` carries the owners' optional label for it (§3.4b). Rules, reads and triggers in §3.1a; on a multi-owner entity every edit is a `change_request` (§3.1b).
`report_definition`	owner user id (clones)	system templates read-only; clone	Upgrades replace system templates in place; clones are never touched.

Table group	Tenant key	Mutability	Notes
	/ null (system templates)	s mutable	
`attachment`, `attachment_blob`, `entry_attachment`	entity id	attachment insert-only; links deletable by manager+ (audited)	Supporting documents such as statements and receipts, stored as `bytea` in Postgres (see §3.5). They are RLS-covered, backed up, exported and purged along with everything else.
`notification`	entity id + recipient user id	insert-only, except `read_at`	One row per recipient per alert event (§2.4): among them change requested, reminder, approved, declined, expired, forced, founder transferred, placement, ancestor invite, cap-table holder, former holder, chain break. Producers write it in the same transaction as the event. Purged with the entity, and deleted with the recipient's account.

Table group	Tenant key	Mutability	Notes
`audit_event`	entity id (nullable after purge)	append-only, outside purge cascade	Records purges, exports, revocations, membership changes, and `principal_invalid` transitions; survives the purge it describes. Readable by owners only (SECURITY §4.2.1).

3.1a The cap table: equity, separate from access

Principal, 2026-11-07: every entity carries a cap table; equity is independent of the access roles; it replaces `tax_allocation_share`.

Membership answers "who may do what"; the cap table answers "who owns how much". A 0-unit owner is a bookkeeper; a 25% holder may have no access at all. The two never derive from each other.

- **Shape.** Two append-only tables, both effective-dated. `equity_total(entity_id, total_units, effective_from)` holds the unit count; `equity_holding(entity_id, holder_entity_id, units, effective_from)` holds each holder's count. Units are `numeric`, `units ≥ 0`, `total_units > 0`. The total is dated, not a column on `entity`, because a past allocation must use the total in force on its date: issuing units in June must not rewrite January's split.
- **The split on date *d*.** The latest `equity_total` row with `effective_from ≤ d`, and for each holder its latest `equity_holding` row with `effective_from ≤ d`; a holder at 0 units has left. Stake = units / total. The remainder, total − Σ units, is **outside owners**: real holders the system does not track, whose share allocates to no one.
- **Corrections are new rows.** Nothing is updated or deleted. A change is a set of rows sharing one `effective_from`; each row carries `recorded_at`, and for one holder and one date the latest recorded row wins. A change entered from the wrong date is undone by a row at that date restoring the earlier units, plus the row at the right date.

- **A Person's table is implicit:** itself at all units. No rows are stored for a Person entity, and a trigger refuses any. A parent (Household, Holding) carries a cap table like any entity, and a parent is never a holder.
- **Defaults.** A new non-Person entity is created with 100 units and the creating user's own Person holding all 100, from the creation date; the creation form shows the table so the user can change it before saving. Every user's own Person is marked by `entity.self_user_id` (unique, nullable, set by `provisionPerson`; §3.1).
- **Invariant.** Σ units ≤ total units at every date. A `DEFERRABLE INITIALLY DEFERRED` constraint trigger checks it at commit, at every change point from the earliest `effective_from` written in the transaction onward, so a new total and new holder rows commit together.
- **Who writes:** owner of `entity_id` only (it is entity metadata). While the entity has one owner the write is direct; with more than one, every edit, the entity-creation default excepted, is a `cap_table` `change_request` that applies once every other owner approves, re-validated against the Σ invariant at apply (§3.1b). **Who may be a holder:** a Person, Trust or Business that is either visible to the editing user (any membership) or the own Person of a user holding a membership on this entity. The editor lists owner-access members first, then the other members, then "add holder" over the editing user's other entities. Anyone else is reached by inviting them as a member first, or left in the outside-owners remainder. **"Invite as viewer" appears in both places** (settled for ux-designer, PR #78): in the save that adds a Person holder whose user has no access, and afterwards on any saved row whose Person holder's user still has none, so a declined or skipped offer can be taken up later. The invite is its own action, never bundled into the cap-table write: on a sole-owner entity it is an ordinary invitation insert in the same transaction; on a multi-owner entity it is a separate `invite` request raised beside the `cap_table` request, and either may apply or fail without the other. An entity holder gets no offer, since membership belongs to users.
- **Who reads:** any member of the entity, by ordinary RLS on `entity_id`. The `equity_*` SELECT policies are not widened for anyone else. Everything beyond that goes through definers owned by the NOLOGIN, NOBYPASSRLS role `poly_stake`, all on the allowlist (SECURITY A-18, C-27):
  - `app.my_stakes()` is a holder's owners' read path. It returns entity id, display name, units, total units and effective-from for every entity in which the caller's own Person, or an entity the caller owns, holds units. Never another holder's row.
  - `app.cap_table_candidates(entity_id)`, for an owner of `entity_id` only: id and display name of members' own Persons and of Person, Trust and Business entities visible to the caller.
  - `app.allocated_income` (§4.5).

`poly_stake`'s role-targeted SELECT policies are confined to `app.held_entities()`: entities in which the caller's own Person, or an entity the caller owns, has ever held units, plus one link onward; candidates also read members' own Persons.

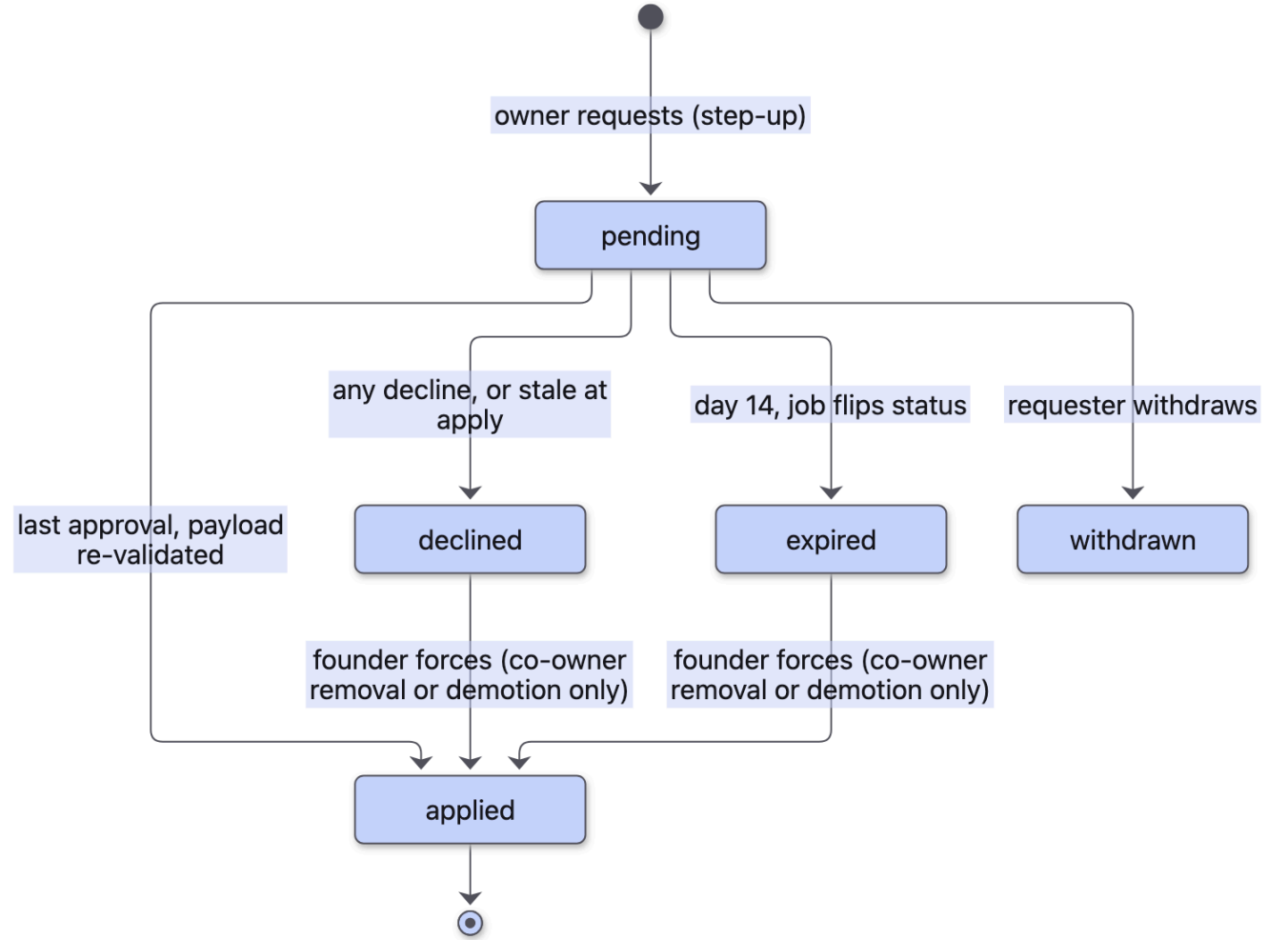
- **Notice.** Adding or changing a holder writes a `notification` in the same transaction: to the user whose own Person it is, or, for a Trust or Business holder, to each of its owners. The editor may not see those owners, so the write is done by the definer trigger `app.check_equity_holding()`, owned by `poly_graph` (§3.3b).
- **Cycles refused.** The holder graph (entity → holder) must stay acyclic: an entity never holds itself through any chain. `app.check_equity_holding()` runs BEFORE INSERT, `SECURITY DEFINER` owned by `poly_graph` (graph-wide read; SECURITY A-16), `search_path = ''`, under `pg_advisory_xact_lock` on its own fixed key, the same mechanism as the parent-link trigger. It also refuses a new row naming a tombstoned holder (§3.4b). An edge is any pair that has ever had a row with units > 0, whatever the date, so the check needs no reasoning about date ranges. It is stricter than needed, and the cost is that a long-exited holding still blocks the reverse one.
- **Depth: one entity link in v1, deeper chains refused.** An entity may be held by an entity whose own holders are Persons (an LLC held by a trust held by its beneficiaries), never by an entity that is itself held by an entity. The same trigger refuses a row that would make an entity-to-entity chain two links long, from either end. Refusal, rather than treating the deeper share as outside, means no income ever silently drops out of tax (§4.5). The cost is that a holding company inside a trust cannot be modelled in v1 (unmeasured; no structure described so far needs it). Raising it is one migration and a composition change in §3.3c and §4.5. Errors name no hidden entity, as in §3.3b.
- **A holder that is purged** becomes a former holder: its rows stay, shown as "Former holder (purged <date>)" with an optional label the entity's owners type themselves (a cap-table edit), and its units count as outside owners from the purge date (§3.4b, OQ-33).
- **Tests (pgTAP):** Σ over total refused at a later change point; a Person row refused; a direct and an indirect cycle refused, including through an entity the editor cannot see; a two-link chain refused; a non-owner write refused; `app.my_stakes` returns only the caller's holder rows, and a holder's owner reads 0 rows from `equity_holding` directly; a non-owner gets 0 candidates; a notification lands for an owner invisible to the editor; with two owners, a direct `equity_holding` INSERT is refused.

3.1b Co-owner approval and the founding owner

Principal, 2026-11-07 (P5b, R-19); SECURITY §4.2.2, C-20, C-28; A-20, A-22. Co-trustees and partners govern as a group, and owner is an access role, not a stake.

What needs approval. On an entity with more than one owner, these take effect only once every other current owner approves: (a) inviting a member or raising a role; (b) removing or demoting a co-owner; (c) any cap-table edit. Removing a viewer or manager, and lowering or giving up one's own role, stay single-owner actions. A sole-owner entity is unaffected.

Action	Sole owner	Multi-owner	Founder extra
Invite a member, any role	direct	`invite` request; email after apply	none
Raise a role	direct	`role_raise` request	none
Add the first co-owner (invite or raise)	direct: nobody to approve	n/a	none
Remove or demote a co-owner	n/a	`owner_remove` / `owner_demote` request; the target votes	force after decline or expiry
Remove a viewer or manager; lower or give up one's own role	direct	direct	must transfer founder before giving up owner
Cap-table edit	direct	`cap_table` request	none
Transfer the founder role	n/a (a Person's founder is its user)	founder only, direct, step-up, every owner notified	the founder's own act



A forced apply sets `forced = true` and writes `change.forced`. The day-7 reminder is a notification, not a state.

- **The direct path is gated in the database.** A BEFORE INSERT / UPDATE / DELETE trigger on `membership`, `invitation`, `equity_total`, `equity_holding` and `equity_holder_label` raises when `current_user = 'poly_app'`, the action is one of (a) to (c), and `app.owner_count(entity_id) > 1`. A trigger, not a policy, because telling "remove a viewer" from "remove a co-owner", or "lower my own role" from "demote someone", needs `OLD`. `app.owner_count` is a `poly_authz` definer, and the trigger takes the entity row `FOR SHARE` so the count cannot change under it. A `can()`-only gate in the app fails the §4.2 design test.
- **The multi-owner path.** `poly_app` INSERTs a `change_request` (owner of the entity, step-up within 5 minutes) and `change_approval` rows (an approver must be a current owner other than the requester). Every status change and every resulting write happens in a `poly_govern` definer:

    - `app.apply_change(change_id)`, called after each decision. One decline flips the request to `declined`. Otherwise it recomputes the approvers as every current owner other than the requester, counts only approvals by current owners, and applies when none is missing. Before applying it re-validates the payload: the role still exists, the cap-table Σ invariant holds, and the entity version is unchanged or no conflicting change has applied since. A failed check declines the request as stale and nothing applies. It runs in the last approver's transaction. An invitation row is written only here, and the route sends the email after commit.
    - `app.withdraw_change(change_id)`, requester only.
    - `app.expire_changes()`, executable by `poly_worker` only, run daily: flips `pending` past 14 days to `expired` and writes the day-7 reminder notification once per request. It changes nothing but status and notifications.
    - `app.force_change(change_id, reauth_at)`: caller is the founder; kind is `owner_remove` or `owner_demote`; status is `declined` or `expired`; the target is not the founder; second-factor step-up within 5 minutes. Re-validates as apply does, writes `change.forced`, notifies every owner, the target included. The founder may have raised the request; the force still waits for the decline or expiry, so the target always has warning.
    - `app.transfer_founder(entity_id, to_user_id, reauth_at)`: caller is the founder; target a current owner; step-up; audit and notice to every owner. Stepping down is this call.
- **`poly_govern`** is NOLOGIN, NOBYPASSRLS, owns no tables, and holds role-targeted INSERT / UPDATE / DELETE on `membership` and `invitation`, INSERT on `equity_total` and `equity_holding`, UPDATE of `status` and `forced` on `change_request`, UPDATE of `founding_owner_user_id` on `entity`, INSERT on `notification` and `audit_event`, and SELECT on the rows those need. Its policies are keyed to the request's entity through a transaction-local GUC the definer sets itself, as `poly_purge` does (§3.4).
- **Limits on requests.** One open request per kind and target. An expired request may be re-raised at most once per 7 days. A new owner becomes a required approver of every pending request; a departed owner drops out.
- **Invitations issued while sole owner.** An invitation the sole owner sent directly may still be outstanding when a second owner joins. When the owner count goes from 1 to 2, every other outstanding direct-path invitation on that entity is revoked and its inviter notified to re-raise it as a request; otherwise a sole owner could pre-issue owner invites that land after the approval rule applies. SecEng confirms (OQ-31).
- **The founding owner.** `entity.founding_owner_user_id` is set to the creator; a deferred constraint trigger refuses any commit where the founder does not hold owner, which is also what forces a transfer before the founder gives up owner. The founder has no extra power over invites, role raises or cap-table edits, and is never a `force_change` target. Others cannot

remove a hostile founder in-app (accepted risk). A founder who vanishes is handled by the operator's audited CLI (`polycarpic entity set-founder`), which notifies every owner.

- **An owner's tombstone** (§3.4a) re-evaluates every pending request on their entities in the same transaction. Their approvals stop counting and they stop being required. A request that is now fully approved is not auto-applied: its requester is notified to confirm, with step-up, by calling `apply_change` . Requests they raised are withdrawn.
- **Audit and notice.** Every step writes `change.requested` , `.approved` , `.declined` , `.withdrawn` , `.expired` , `.applied` or `.forced` . Every owner is notified on request and on outcome. The invitee sees nothing until apply.
- **Tests (pgTAP, SECURITY C-28):** with two owners, a direct membership INSERT, a role raise or an `equity_holding` INSERT by one owner is refused; removing a viewer stays direct; `apply_change` with one approval missing does nothing; an ex-owner's approval does not count; a decline and an expiry block; a stale cap-table payload is declined; a sole owner's invite applies at once; the first co-owner is added directly; outstanding direct invites are revoked at the 1 → 2 transition; every `force_change` misuse (non-founder, pending, invite or cap-table kind, founder as target) is refused; a forced removal notifies the target; the founder cannot drop owner without a transfer; the tombstone is refused while the user founds a multi-owner entity. Integration: no invite email before apply; each step without a recent step-up returns 403.
- **Scope:** the gate and the founder column ship with tenancy in POLY-M1, since any entity can gain a second owner; the request flow, screens and expiry job ship with sharing in POLY-1.0 (POLY-70). Between the two, a multi-owner entity's gated actions are simply refused, which is safe on staging, where no real books exist.

3.2 Invariants the database enforces (not the app)

1. **Balanced entries.** A `DEFERRABLE INITIALLY DEFERRED` constraint trigger on `journal_line` checks `Σ base_amount = 0` per entry at commit, and at least two lines. The app computes balance too, for good error messages; the trigger is the backstop.
2. **Append-only posted rows.** `BEFORE UPDATE OR DELETE` triggers on `journal_entry` , `journal_line` , `journal_chain` , `change_approval` , `lot` , `lot_match` , `tax_period_freeze` , `tax_close` and on `raw_transaction` where `status = 'posted'` raise, and so do `BEFORE TRUNCATE` statement triggers on the same tables. No login role holds `TRUNCATE` . The one exception is DELETE by `current_user = 'poly_purge'` for the entity named in `app.purge_entity` (§3.4). Corrections go through `correctEntry(original, newLines)` . In one transaction it writes a reversal (a mirror entry with `reverses_entry_id` set) and a replacement entry with `replaces_entry_id` set. The UI calls this action "Correct

entry". The UI never offers "edit" or "reverse" on a posted row; it badges the replacement and shows the struck-through reversal on click-through (DESIGN). `reverseEntry()` on its own stays internal, for a correction whose replacement is empty.

3. **Two-way link.** Confirming one or more drafts inserts the entry and, in the same transaction, sets each draft's `entry_id` and flips each raw row to `posted`. The link is held on the draft side (many drafts → one entry). The entry's `source_ref` only marks it as import-sourced; it doesn't point back to a single row. A check ensures a posted raw row always has an entry (gate G4, §3.2a).
4. **Single posting date.** One date column per entry; a `posting_date_allowed(entity, date)` function hook (always true in v1) is the seam where period locking plugs in later without a schema change.
5. **Same-entity legs.** Every line's account belongs to the entry's entity (composite FK `(entity_id, account_id)`). Cross-entity transfers are two entries, one per entity's books (§4.3).
6. **Actor recorded.** `actor_kind` + `actor_id` are NOT NULL on entries and drafts, and are *forced* by a BEFORE INSERT trigger from the transaction's context rather than merely defaulted. `actor_kind = 'user'` requires `actor_id = app.current_user_id()`, so a caller cannot attribute an entry to someone else (SECURITY A-5).
7. **Gapless Journal ID.** Every entry carries `journal_number`, unique on `(entity_id, journal_number)` and shown as `JE-000214`; the UUID stays the key. A BEFORE INSERT trigger assigns it, so a caller cannot supply one: `UPDATE entity_journal_counter SET last_number = last_number + 1 … RETURNING`. Since `poly_app` has no write grant on the counter, the trigger function is `SECURITY DEFINER`, owned by the NOLOGIN, NOBYPASSRLS role `poly_journal`, which also owns the chain trigger below (OQ-30). A plain sequence would burn a number on rollback. The counter row is locked until commit, so a rolled-back posting, including one failing the deferred balance check, returns its number, and postings in one entity serialize. Different entities never contend. The append-only trigger is unaffected, because the number is set before the row exists and the counter is a separate table. A correction takes two numbers, reversal then replacement, in its one transaction. A transaction touching several entities takes the counters in `entity_id` order, which rules out deadlock. Auto-post batches commit per entry or in short groups, so a batch never holds the lock against a user posting by hand. The cost is one row lock per posting (unmeasured; household write rates make contention unlikely).
8. **Lots tie to the ledger.** For each custodial account and securities nature class, the ledger balance equals the summed adjusted basis of open lots there (§3.6). Lot write and entry commit in one transaction; a periodic assert job is the tripwire, and a mismatch alerts rather than repairs.

9. **Hash-chained ledger** (Principal, 2026-11-07, P4; SECURITY C-24; POLY-M2). Each entity's posted entries form a SHA-256 chain in `journal_number` order: `entry_hash = sha256(prev_hash ‖ canonical(entry, its lines))`, the first entry chaining from a fixed per-entity seed. The canonical encoding is pinned and versioned (`alg_version`): field order, numeric text form, UTF-8 NFC.
   - **Written at commit.** The hash covers the lines, which do not exist when the entry is inserted, so it is computed by a `DEFERRABLE INITIALLY DEFERRED` constraint trigger on `journal_entry`, `app.chain_entry()`. It reads the entry's lines, reads the head from `entity_journal_counter`, inserts the `journal_chain` row and moves the head. The ruling says "after the balance check"; deferred triggers on two tables fire in queue order, so the order is not guaranteed, and it does not need to be: a failed balance check aborts the transaction, chain row included, so no chain row ever commits for an unbalanced entry.
   - **No new lock.** The counter row is already locked by the numbering trigger until commit, so the head moves under the same serialisation as the gapless number, and a rolled-back posting leaves neither a number nor a chain row. Two entries in one transaction (a correction) chain in number order, which is their queue order.
   - **The trigger is the only writer.** `poly_app` and `poly_worker` hold no grant on `journal_chain` or the head columns. `poly_journal` holds INSERT on `journal_chain`, UPDATE on the counter, and SELECT on `journal_entry` and `journal_line`; the wide read is acceptable for the reason SecEng gives for `poly_graph`: a function returning `trigger` cannot be called to return data. Posted entries are never UPDATEd to hold their hash.
   - **Verifier.** A weekly job unit per entity, running under the entity's earliest owner through `app.job_targets` (§3.3), recomputes the chain from the entity's lines and compares every link and the head. A break is SEV1 ledger tampering: an `audit_event`, a notification and an email to every owner, and an email to the operator (§7.4).
   - **Head leaves the box.** Every export carries the head `(journal_number, entry_hash)`, recorded in its `export.completed` event. A purge deletes the chain and records the final head in `entity.purged` (§3.4).
   - **Limit.** The operator's superuser can disable the triggers and recompute the whole chain. Detection then rests on a head held off the box, an export the owner kept (R-1, re-confirmed 2026-11-07).
   - **Cost.** One SHA-256 over an entry of a few lines and one row insert per posting, inside a lock already held (unmeasured; expected negligible against the 2 s dashboard NFR, which is a read path).
   - **Tests:** a superuser edit (triggers disabled) to an entry, a line, or the order is caught by the verifier; a rolled-back posting leaves no chain row and no gap; `poly_app` cannot insert

into or update `journal_chain`; a correction's two entries chain in number order; the export's head equals the verifier's.

3.2a Posting gates

A draft or group of drafts leaves Pending only by passing every gate below. The Principal's rule (2026-10-02): an item whose entry is not correct stays in the Pending queue. `posting.confirm()` evaluates the gates and returns a `GateResult` that the modal shows live; the Post button stays disabled until all gates pass. The database re-checks gates G1, G2, G4 and G5 at commit, so a bug in the app cannot bypass them.

#	Gate	Enforced by
G1	**Balanced:** Σ base_amount = 0 over the entry's lines, with at least two lines.	App (live) + deferred constraint trigger (§3.2.1)
G2	**Accounting equation per entity:** every line's account belongs to the entry's entity. With G1, every entity's books then keep Assets = Liabilities + Equity (+ Income − Expense) after every posting. No separate entity-wide sum runs per posting; a property-based test asserts the trial balance is zero after random postings.	Composite FK (§3.2.5) + G1; test
G3	**Dates:** an entry has exactly one posting date, equal to the bank date of every raw row it consumes. Legs with different dates never share an entry: a transfer whose legs land on different dates posts as two entries through Cash in Transit (PRD §5, the 2026-09-27 decision). A group (Flow B) may only combine drafts with the same bank date, so a daily summary is allowed and a weekly one is not (Principal, 2026-11-06).	App + a check in `confirm()`
G4	**Linked:** every draft being posted gets exactly one `entry_id`, and its raw row flips to `posted` in the same transaction. A posted draft without an entry, or an entry claiming a draft that is still pending, is rejected.	Check constraint + trigger (§3.2.3)
G5	**Custodial leg locked:** for each custodial account touched by the consumed raw rows, the entry carries lines on that custodial's cash ledger account (its liability account for a credit card) summing exactly to those rows' amounts. The user edits only the offsetting legs; the bank's reality is never editable. A split keeps one locked leg and divides the	App (the locked leg is not editable in the modal) + trigger comparing

#	Gate	Enforced by
	offset; a group aggregates the locked legs into one line per custodial account.	against the raw rows
G6	**Allowed:** the actor holds manager+ on the entity, or the post comes from a rule whose `auto_post_by_user_id` still holds manager+ on the entity. `posting_date_allowed()` passes; it always does in v1, and is the hook for period locking later.	RLS + trigger (§3.2.6)

G7, securities lines carry lots: a line on a securities nature account needs its lot write in the same transaction, and the security needs a class mapping. An unmapped trade stays in Pending until it is mapped, in keeping with the rule above. This applies to manual entries and corrections too, and `posting.confirm()` enforces it; the §3.2.8 assert is the backstop.

A manual entry ("+ New entry") consumes no raw rows, so G3–G5 do not apply to it; G1, G2, G6 and G7 do. A "Correct entry" replacement runs G1, G2 and G6, and must keep the original's custodial lines unless the user explicitly re-links the raw rows.

3.3 RLS mechanism and connection-pooling safety

Confirmed by SecEng in SECURITY §4.3 / §12 (OQ-2), with the four additions listed last. The permission matrix is SECURITY §4.2.1: roles are uniform across entity types.

- **Roles.** `poly_owner` (NOLOGIN; owns every object); `poly_migrator` (LOGIN; used only by the migrate step; runs migrations as `set role poly_owner … reset role`, which mosko-fintech measured as the working shape); `poly_app` (LOGIN; DML grants only; `NOBYPASSRLS`; not the table owner, so RLS applies); `poly_worker` (LOGIN; `NOBYPASSRLS`; reference-data writes plus tenant DML limited to the import path, SECURITY §4.4); `poly_authz` (NOLOGIN; owns the authorization functions and nothing else); `poly_purge` (NOLOGIN; `NOBYPASSRLS`; owns the purge function; DELETE on purgeable tables only, plus the tombstone UPDATE on `entity`, §3.4b). The remaining NOLOGIN, NOBYPASSRLS roles own definer functions and no tables:
  - `poly_rollup`: `app.rollup_totals`, `app.rollup_economic`, `app.rollup_tree`, `app.rollup_descendants`, `app.parents_of`; SELECT only, confined to descendants of the caller's parents and to ancestors of entities the caller owns (§3.3a).
  - `poly_graph`: the trigger functions `app.check_rollup_edge()`, `app.check_equity_holding()`, `app.notify_ancestor_invite()` and `app.notify_former_holder()`; graph-wide SELECT on `entity_rollup` and

`equity_holding`, SELECT on owner `membership` rows and `entity.kind`, INSERT on `notification` (§3.3b, SECURITY A-16, A-21).

- `poly_stake`: `app.cap_table_candidates`, `app.my_stakes`, `app.allocated_income`, `app.held_entities`; SELECT only, confined to `app.held_entities()` (§3.1a, SECURITY A-18).
- `poly_govern`: `app.apply_change`, `app.withdraw_change`, `app.expire_changes`, `app.force_change`, `app.transfer_founder`; the writes listed in §3.1b (SECURITY A-20, A-22).
- `poly_journal`: the numbering and chain triggers of §3.2.7 and §3.2.9 (OQ-30).

Every one of these functions is on the definer allowlist (C-15) and every role is in the role-hygiene set (C-10). Calling a trigger function directly fails. Role hygiene follows SECURITY §4.3(k). The operator's superuser bypasses RLS by definition — documented as an accepted risk (PRD §6).

- **Context per transaction, never per session.** `withTenant(userId, fn)` opens a transaction and runs `select set_config('app.user_id', $1, true)` (`is_local = true`) before any query. The setting dies at COMMIT/ROLLBACK, so a pooled connection cannot carry a previous request's identity. This works with the in-process `node-postgres` pool on the VPS and with Neon's transaction-mode pooler. Session-level `SET` is banned by lint.
- **Default deny.** Policies read context only through `app.current_user_id()` = `nullif(current_setting('app.user_id', true), '')::uuid`. The `nullif` matters: once a backend has set the GUC and the transaction has ended, `current_setting` returns `''`, not NULL. Without `nullif`, a reused backend would compare against an empty string instead of denying cleanly (SECURITY A-3). Every tenant table has RLS `ENABLE`d and `FORCE`d, including drafts, raw rows, lots and staging tables.
- **Policy shape.** `using (entity_id = any (app.visible_entities()))` for reads; writes add `app.has_role(entity_id, 'manager')` or `'owner'` per the permission matrix. Both functions are `STABLE SECURITY DEFINER`, owned by `poly_authz` (not `poly_owner`, because under FORCE RLS the owner is itself subject to policies). They read `membership` through a role-targeted `SELECT … TO poly_authz` policy, with `search_path = ''`. Every definer function is on a CI-checked allowlist (SECURITY A-1, §4.3(h)). Tenant policies never consult parent/child structure — structure grants no rows. The single place `entity_rollup` is read for authority is the consolidation function in §3.3a, and it returns totals, never rows.
- **System jobs.** `withSystem(job, target, fn)` sets `app.user_id` to the target's principal and `app.actor = system:<job>`. It never runs context-free; if no principal resolves, the job errors. Targets come from one allowlisted definer function, `app.job_targets(job)`, which returns `(entity_id, principal_user_id, connection_id)` and is executable only by `poly_worker`. Entity-bound jobs with no connection (the tax-period freeze) run as the

entity's earliest owner membership. Reference-data policies are role-targeted (`TO poly_worker`), and `poly_app` gets SELECT only. Cash-in-Transit aging is computed on read by the exceptions report, not stored by a job (SECURITY A-12). Tenant writes by a job (import polling) run under the linking user's membership (`provider_connection.linked_by_user_id`) with actor = `system:<job>` , so authority and attribution stay separate. There is no system bypass clause in tenant policies (SECURITY §4.4, OQ-3).

- **SecEng additions (OQ-2).** (a) The RLS coverage probe is derived from `pg_catalog` , never from a hand-maintained list. (b) `poly_app` also loses `CREATE` on `public` . (c) Policies are per-command with an explicit `WITH CHECK` on every write; `FOR ALL` is banned. (d) The inversion canary stays in the battery. The review added (e) to (l) (SECURITY §4.3):
  - (e) Authorization functions are owned by `poly_authz` .
  - (f) Context is read in the `nullif` form.
  - (g) TRUNCATE triggers.
  - (h) `search_path = ''` , plus a definer allowlist.
  - (i) Tenancy columns are not updatable.
  - (j) Views are `security_invoker = true` , and there are no materialized views over tenant data (a matview has no RLS). A report needing a cache materializes into a tenant table under RLS.
  - (k) Role hygiene.
  - (l) UUID keys.
- **Proof.** Every migration run in CI executes an RLS battery: an *inversion self-test* (an unprotected canary table must leak, proving the probe has teeth — borrowed from mosko-fintech), a per-table cross-tenant probe generated from the catalog (a table with RLS disabled fails the build), and a pool-reuse test. That test must run on a backend that has *already* served a request: tenant A's transaction, then tenant B on the same physical connection sees zero A rows, and a context-free query on that dirty backend sees zero rows. The full battery list is SECURITY §12.3 (qa-engineer).

3.3a Parent roll-up under RLS: totals, not rows

Access stays membership-only and is never inherited (PRD §5): membership on a parent grants nothing on its children, and the reverse. Yet a parent member must see the parent's consolidated figures, including children they hold no membership on, and drilling into such a child stops at the boundary panel (DESIGN §5.7). Plain RLS cannot serve that: under `poly_app` the child's lines are invisible, and §3.3(j) bans materialized views over tenant data.

Mechanism: one allowlisted definer function computed on read. `app.rollup_totals(parent_entity_id, as_of, period_from, period_to)` is `STABLE SECURITY DEFINER`, `search_path = ''`, owned by a new NOLOGIN, NOBYPASSRLS role `poly_rollup`, with `EXECUTE` granted to `poly_app`. It sits on the definer allowlist.

1. It returns nothing unless `app.has_role(parent_entity_id, 'viewer')` holds for `app.current_user_id()`.
2. `poly_rollup` owns no tables. It reads `entity_rollup`, `entity` (name and type only), `account`, `journal_line`, `lot` and the reference prices through role-targeted `FOR SELECT TO poly_rollup` policies whose predicate is `entity_id = any (app.rollup_descendants())`, the recursive walk of §3.3b from the parents the current user belongs to. Even a bug in the function body can read only entities beneath those parents.
3. It returns aggregates only: one row per (entity, report line, currency) for the parent's own accounts and each entity beneath it, each entity once however many paths reach it (§3.3b), carrying the entity's display name and a `viewer_has_membership` flag, which tells the UI whether a click opens the register or the boundary panel. Report lines are account class as of `as_of`: assets, of which each of the four nature classes (§3.6; cash at its balance, the three securities classes at market) and other assets (real estate, prepaid tax and the like, at book); liabilities; net worth; plus income and expense by top-level category for the period. Never an account, entry, line, payee, memo, lot or position. This grain is the Principal's ruling of 2026-11-06 (OQ-18). **The period is any range, by ruling** (Principal, 2026-11-07, P2): the function takes whatever `period_from` / `period_to` it is given, with no snapping to whole months and no period guard. For a small entity, one-day ranges effectively reveal individual amounts and their dates; that exposure is consented, and the consent wording on the inclusion step and on the placement and ancestor-invite notices is a security requirement (SECURITY C-25, T-72, R-17), changed only through a security-gate review.
4. A companion, `app.rollup_tree(parent_entity_id)`, returns the edges beneath the parent with display names and no figures. The UI uses it to draw subtotals for nested parents, including ones the viewer holds no membership on, from the rows above.
5. The function's result shape is fixed by its `RETURNS TABLE`. A pgTAP test asserts that a parent viewer without child membership gets totals equal to the child owner's own figures, and zero rows from every tenant table for that child.

Alternative rejected: worker-materialized per-child aggregates in a tenant table keyed by the parent. A job must run under a principal holding membership (§3.3), and the child's principal is the wrong one to write the parent's rows. The cache goes stale on every posting and needs invalidation triggers. And it buys no extra isolation: the function above already confines its reads

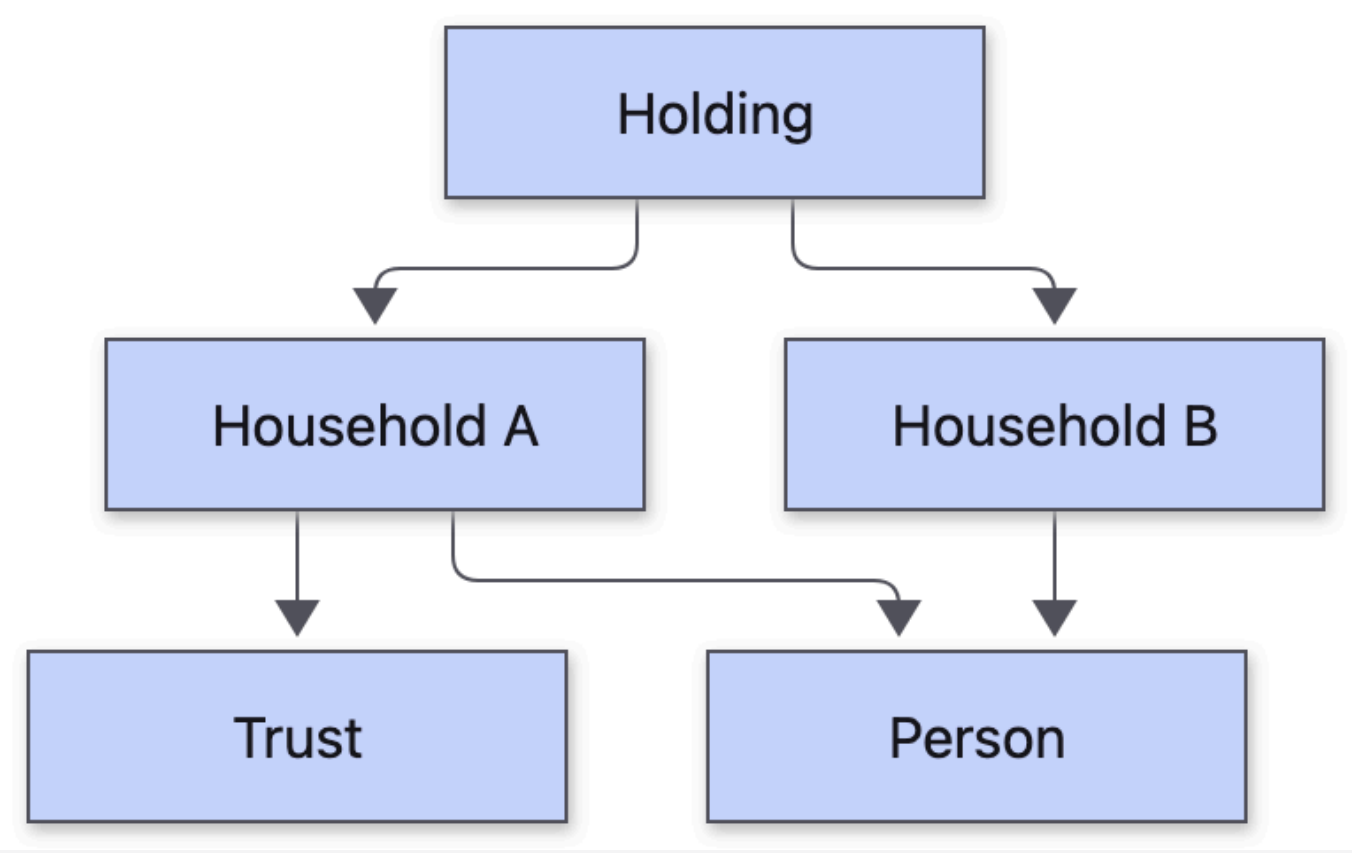
by role and by policy. Revisit only if the dashboard's <2 s NFR fails measured at parent scale (unmeasured; a household has a handful of children).

Holdings popover on a parent (DESIGN, 2026-11-06). The popover lists positions, which this function never returns. So on a parent it lists the parent's own accounts and those of entities the viewer holds a membership on, read through the ordinary RLS path, and any other included entity shows its total and the boundary panel. It belongs to the roll-up milestone; SecEng confirmed the gating (SECURITY C-25): no definer returns a position.

Scope: the junction and its trigger (§3.3b) ship in M1 with tenancy. The consolidated view, `app.rollup_totals`, its economic sibling (§3.3c) and the popover land in POLY-1.1, the quick follow-up right after GA (§11). SecEng signed off the function and `poly_rollup` (SECURITY C-19 amended, C-25; OQ-19 closed), with one change folded in §3.3b: the graph triggers belong to `poly_graph`, not `poly_rollup`.

3.3b Nesting: an acyclic graph, each base entity counted once

A parent may include base entities and other parents to any depth, and a base entity may sit in several parents (PRD §5, 2026-11-06). So the structure is a directed acyclic graph, not a tree, and two paths can reach one entity:



Person is reached twice from Holding and is summed once.

- **Traversal: a recursive CTE on read, no closure table.** `WITH RECURSIVE` over `entity_rollup`, using `UNION` rather than `UNION ALL`, yields each descendant once, which is the count-once rule, and it would terminate even on bad data. A closure table would read faster, but it is a second copy of the structure, and on a graph with shared entities a delete needs path counting to avoid removing a pair another path still justifies. Structures here are small (unmeasured; a household has a handful of entities) and the only reader is the roll-up, so that cost buys nothing. Revisit only if the dashboard's <2 s NFR fails measured at parent scale.
- **Membership and RLS do not depend on depth.** Access stays per entity: tenant policies call `app.visible_entities()`, which reads `membership` alone and never walks the graph. The only traversal is `app.rollup_descendants()` and the two functions above, all on the definer allowlist. Depth therefore adds no cost and no risk to any ordinary read.
- **Cycle prevention: a trigger, under a lock.** `app.check_rollup_edge()` runs BEFORE INSERT and rejects a child that is the parent or any ancestor of it. It must be `SECURITY DEFINER`, because a cycle can pass through entities the inserting user cannot see under RLS. It is owned by `poly_graph`, with `search_path = ''` and a graph-wide read of `entity_rollup`: `poly_rollup`'s reads are confined to the caller's descendants, so a check run as `poly_rollup` would miss a cycle outside them (SECURITY A-16, T-77). Two concurrent inserts could each pass while blind to the other, so the trigger first takes `pg_advisory_xact_lock` on one fixed key. That serializes every parent-link insert on the instance; these are rare, so the cost is nil. A delete cannot create a cycle and takes no lock. UPDATE is already refused.
- **Sanity limit: 8 levels.** The same trigger rejects an insert that would make the longest path from any top parent to any entity beneath it exceed 8 links. The product sets no cap. This one is a judgement, not a measurement (unmeasured): it bounds the work of a definer function, and it sits well above the deepest structure anyone has described (Holding over Household over base entity, 2 links). It is a constant in the function; raising it is one migration.
- **Validation errors name no hidden entity.** The user sees "this would make an entity its own ancestor" or "this structure is too deep (limit 8 levels)". The refusal does disclose that some link exists, one bit about structure the user may not see; SecEng accepted it (R-15).
- **Consent travels upward, with notice (Principal, 2026-11-06).** Agreeing to a parent accepts that parent's later placement under another parent. So linking Household into a Holding shows every entity beneath Household to the Holding's members as totals. In the same transaction, `app.check_rollup_edge()` writes a `notification` to every owner beneath, which a definer must do because the inserter may not see them. Each can see the parents above (`app.parents_of`) and remove their own entity's link. That link is the direct one: a Person's owner leaves a Holding by leaving the Household inside it. The notice names the new top

parent and states what becomes visible, in the consent wording (C-25), with an `audit_event`.

- **Notice when a parent gains a member (Principal, 2026-11-07, P5; SECURITY A-21).** A second `poly_graph` trigger, `app.notify_ancestor_invite()`, runs AFTER INSERT on `invitation` on a parent-kind entity, and AFTER INSERT on `membership` on a parent-kind entity when no invitation produced the row (the operator CLI). An invitation row exists only once it is live, after any §3.1b approval, so the trigger fires at exactly the moment the audience grows. In the same transaction it walks every descendant at any depth and writes one `notification` per owner per person, deduplicated across paths and entities, naming the parent and the new member, with the consent wording and the pull-out link. Any role triggers it, since viewers see totals too; a later role raise (an UPDATE) does not; an accepted invitation's membership does not fire again. Nothing waits on the recipients.
- **Tests (pgTAP):** a diamond sums once; a direct and an indirect cycle are refused, including one through an entity invisible to the inserter, and one through an entity invisible to `poly_rollup`'s predicate; two concurrent opposing inserts leave at most one row; a ninth link is refused with the validation error; a parent viewer gets totals and zero rows from every tenant table at every depth; an invitation going live on a Holding notifies the owners of a Person two levels down whom the inviter cannot see, once each, and a role raise does not; calling a `poly_graph` function directly fails.

3.3c The economic view: the gross roll-up, weighted by the viewer's stake

Principal, 2026-11-07: ships with POLY-1.1 beside the gross roll-up, and reveals nothing the gross view does not.

The gross roll-up answers "what does this Household hold"; the economic view answers "what of it is mine". In a marriage with separate property, a trust one partner inherited counts fully in the gross view and not at all in the other partner's economic view.

- **Mechanism: a sibling definer, same grain.** `app.rollup_economic(parent_entity_id, as_of, period_from, period_to)`, owned by `poly_rollup`, on the allowlist, with the same guard and the same `RETURNS TABLE` as `app.rollup_totals` plus a `stake` column. It takes no Person argument: the viewer's Person is the entity whose `self_user_id = app.current_user_id()`, so nobody can request another member's economic view. The gross function, its rows and the OQ-18 grain are unchanged.
- **Same set, never wider.** It weights exactly the rows `app.rollup_totals` returns for that parent and adds none, so an entity the viewer holds units in but which sits outside this parent contributes nothing here (Principal, 2026-11-07, OQ-29). A cross-parent "my net worth" is a later option that needs SecEng.
- **Weight of a row of entity *H***, for the viewer's Person *P*:

    - *H* = *P*: 1. Another Person: 0, since a Person's table is implicit (§3.1a).
    - Otherwise *P*'s stake in *H*, plus, for each entity holder *B* of *H*, *B*'s stake in *H* × *P*'s stake in *B*. One link only, which §3.1a guarantees is the deepest chain there is. *B*'s cap table is read whether or not *B* is beneath the parent.
    - Balances use the split in force on `as_of`. Period income and expense use the split in force on each entry's posting date, the same rule as tax allocation (§4.5), so a mid-year change splits the year.
- **Economic net worth** = Σ over the selected parent's roll-up set (the returned rows) of net worth × weight, each entity once (§3.3b). It is not the sum over every entity the Person holds units in. It is a figure of the economic view only; the gross view's net worth is unchanged.
- **Disclosure.** Dividing an economic figure by its gross figure gives the viewer's own stake in that entity, which the cap-table notice already tells them. It gives nothing about any other holder. SecEng confirmed (SECURITY C-25). Like the gross view, it takes any date range, with no period guard (P2).
- **Former holders.** A purged holder's units weight as outside owners from the purge date, and nothing looks through it on any date, since its own cap table is gone (§3.4b).
- **Tests (pgTAP):** on every row, Σ over all holders' economic figures + the outside-owners share = the gross figure; a spouse's separate trust weights 0 for the other spouse; a stake changed mid-period splits the period's income at the change date; a viewer gets no economic row for an entity absent from the gross rows.

3.4 The privileged purge path

A sole owner's whole-entity purge (PRD §5) runs through one `SECURITY DEFINER` function, `app.purge_entity(entity_id, export_id, reauth_at)`. The function is owned by the dedicated NOLOGIN, NOBYPASSRLS role `poly_purge`, with `search_path = ''`, and `EXECUTE` is granted to `poly_app` (SECURITY §4.5, OQ-4).

Why not `poly_owner`: under FORCE RLS the owner is itself subject to policies. Append-only tables have no DELETE policy, so an owner-run purge would delete zero rows and still report success (T-51).

How the deletes are allowed: each purgeable table carries `CREATE POLICY purge … FOR DELETE TO poly_purge USING (entity_id = nullif(current_setting('app.purge_entity', true), '')::uuid)`. The append-only triggers allow DELETE only when `current_user = 'poly_purge'` and the row's entity matches. `poly_app` can set the GUC but can never become `poly_purge`.

The roll-up junction has two entity columns, so its purge policy matches either: `parent_entity_id = … or child_entity_id = …`. Purging a base entity removes it from every parent it sat in, and each parent's owners get a notification. Purging a parent removes its own books and its junction rows, upward and downward; everything beneath it stays intact, and a nested parent simply loses that link. **The cap table is the opposite case:** `equity_*` 's purge policy matches `entity_id` only, never `holder_entity_id`, because the rows naming the purged entity as a holder are the other entity's history. Those rows stay, and the purged entity's row stays as a tombstone so they resolve (§3.4b). Pending `change_request`s on the entity are deleted with it (a purge needs a sole owner, so none can be awaiting another owner); their audit events survive.

The guards run in this order, in one transaction:

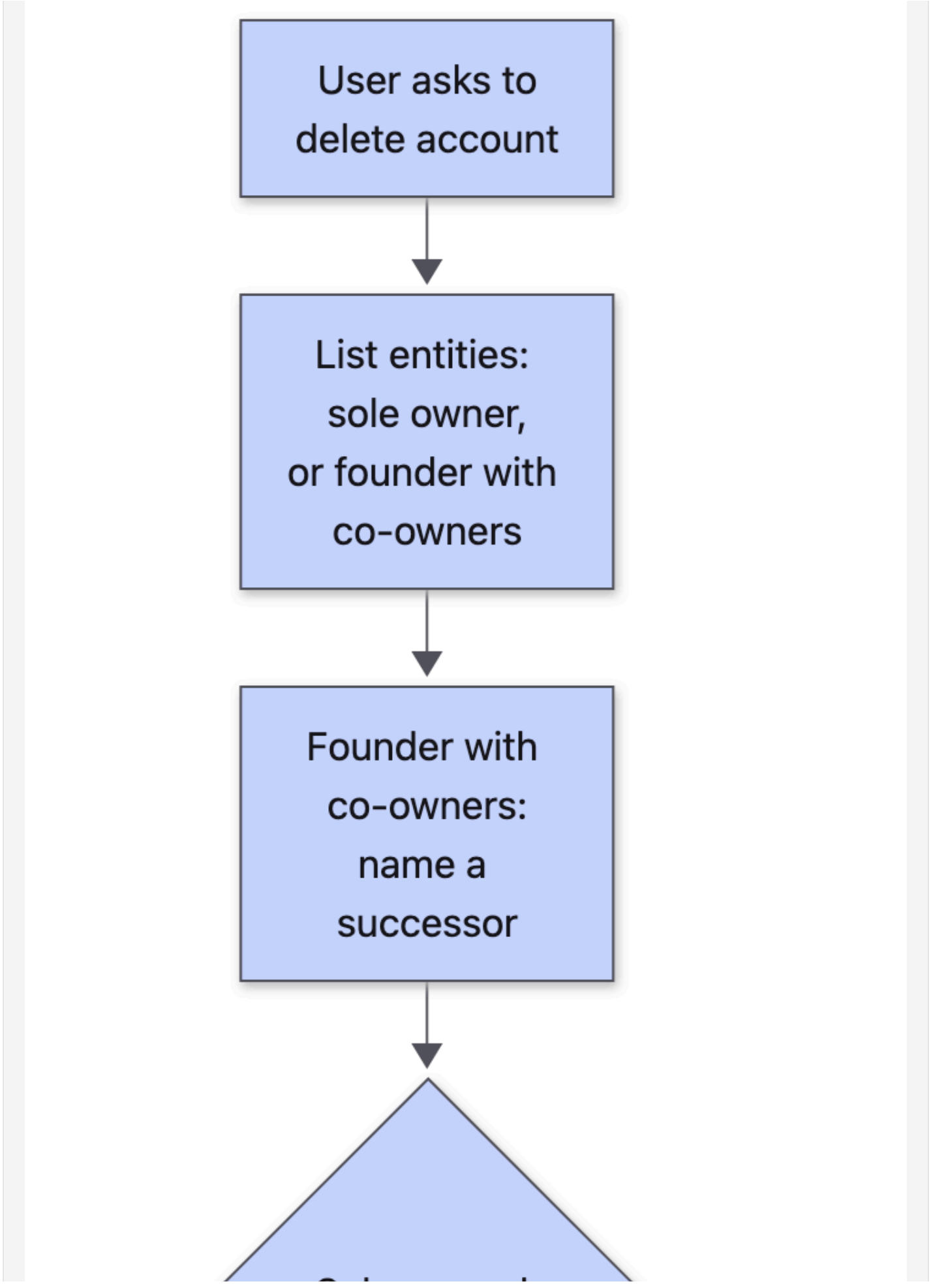
1. Re-authentication within 5 minutes. The route checks it, and the function requires the timestamp as an argument.
2. Sole-owner check against `app.current_user_id()`.
3. An `export.completed` audit event for this entity, by this user, within 24 hours, carrying a SHA-256.
4. No tenant write to the entity since the export's snapshot marker. The entity row is locked `FOR UPDATE`.
5. An `entity.purged` `audit_event` is written before the first delete. It records the actor, entity id and name, the export event, per-table row counts, the final chain head (§3.2.9) and the ids of the entities on whose cap tables it was a holder (§3.4b). `audit_event` has no FK to `entity` and no purge policy.
6. Every connection has been revoked at the provider by the worker revoke job. A failed revocation blocks the purge.
7. Set `app.purge_entity` transaction-locally and delete in dependency order, attachments, `journal_chain` and the counter included. Then reduce the `entity` row to its tombstone (§3.4b).
8. Assert zero rows remain for the entity in every purgeable table before commit, and that the `entity` row is in tombstone shape (fail loud). `entity` is not a purgeable table: it has no purge DELETE policy, so the catalog-derived list of purgeable tables never contains it, and the tombstone assert covers it instead.

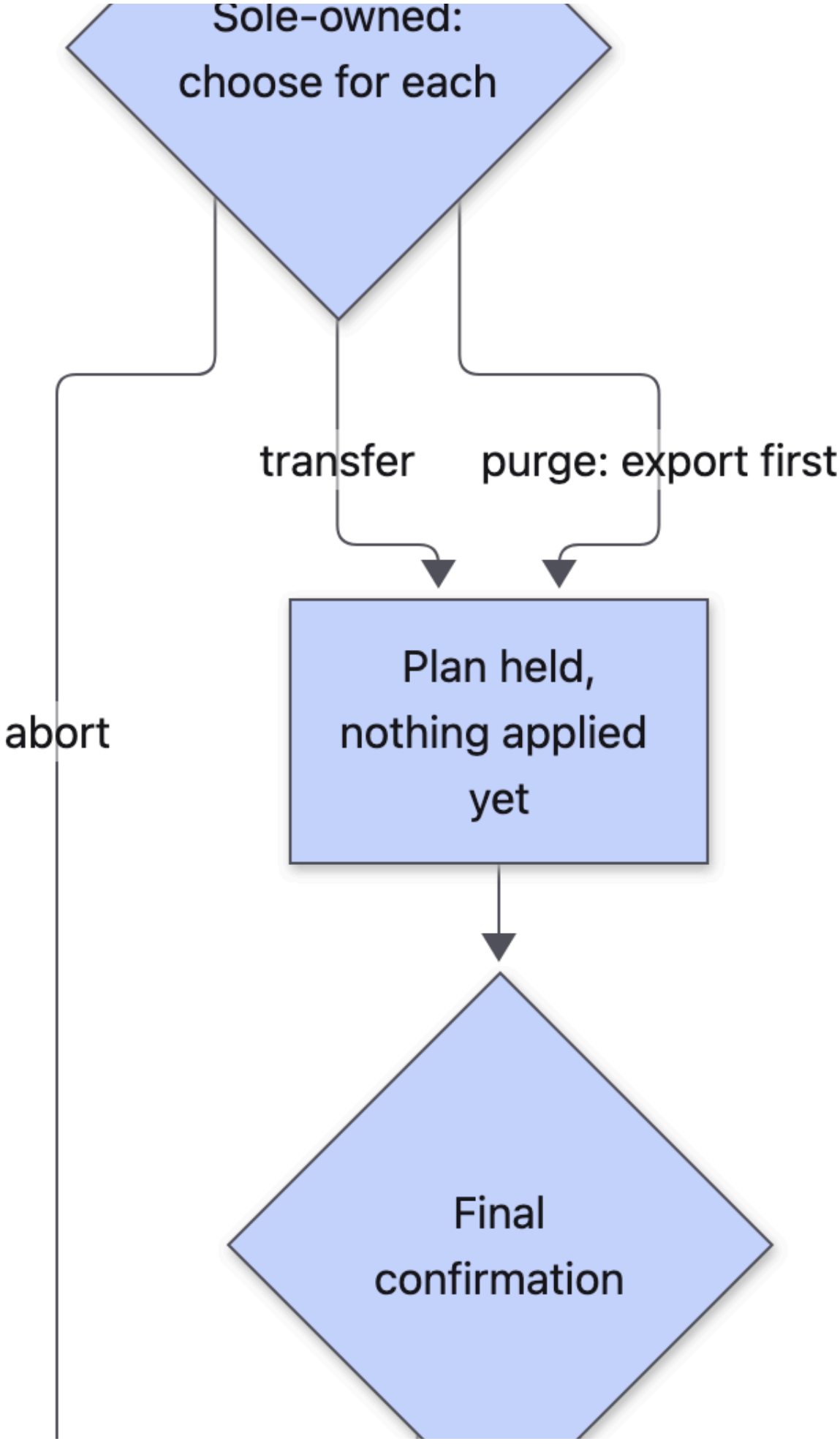
Not permitted: purge by a job; purge through the API without typed confirmation of the entity name; purge of a partial range; purge while a second owner exists. **Rejected:** `DISABLE TRIGGER` (DDL, affects every tenant), tombstoning the entity's data in place of deleting it (breaks PRD §5; the id-only tombstone of §3.4b holds no data), `BYPASSRLS` (broader than needed). **No execution**

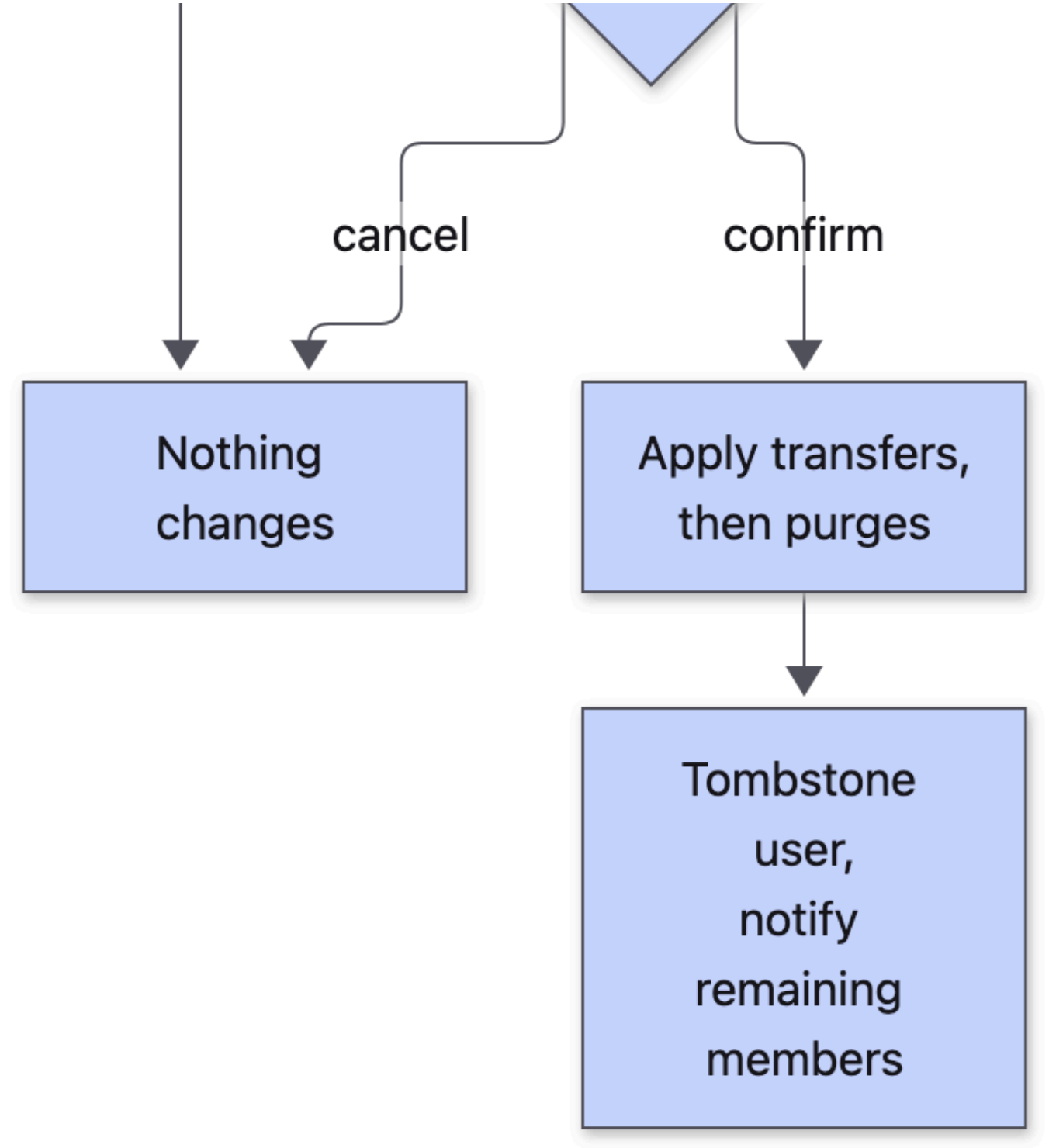
delay (Principal, 2026-11-06, OQ-23): the guards above stand, operator backups remain, and every member is emailed immediately, before the entity's own rows go. SecEng had recommended a cancellable 24-hour delay (SECURITY §12.3 Q2).

3.4a Account deletion in-flow

PRD §5, ruling of 2026-11-06, replacing the rule that refused deletion while it would orphan an entity. The invoker of every step is the account being deleted, signed in with a fresh re-authentication.


User asks to delete account
List entities: sole owner, or founder with co-owners
Founder with co-owners: name a successor


Sole-owned:
choose for each
transfer
purge: export first
abort
Plan held,
nothing applied
yet
Final
confirmation



1. **Choices come first, effects last.** A purge cannot be undone, and "abort the whole deletion" must mean what it says. So the flow collects one choice per sole-owned entity without applying any; exports for the purge choices are produced here. Abort at any point before the final confirmation changes nothing. Transfer is never preselected.
2. **Founder succession.** For each entity the user founded that has other owners, the flow requires a successor named among those owners; it is applied with the transfers, through `app.transfer_founder`, and every owner is notified (§3.1b, SECURITY §4.6). An entity they co-own but did not found needs no step: they simply leave it.

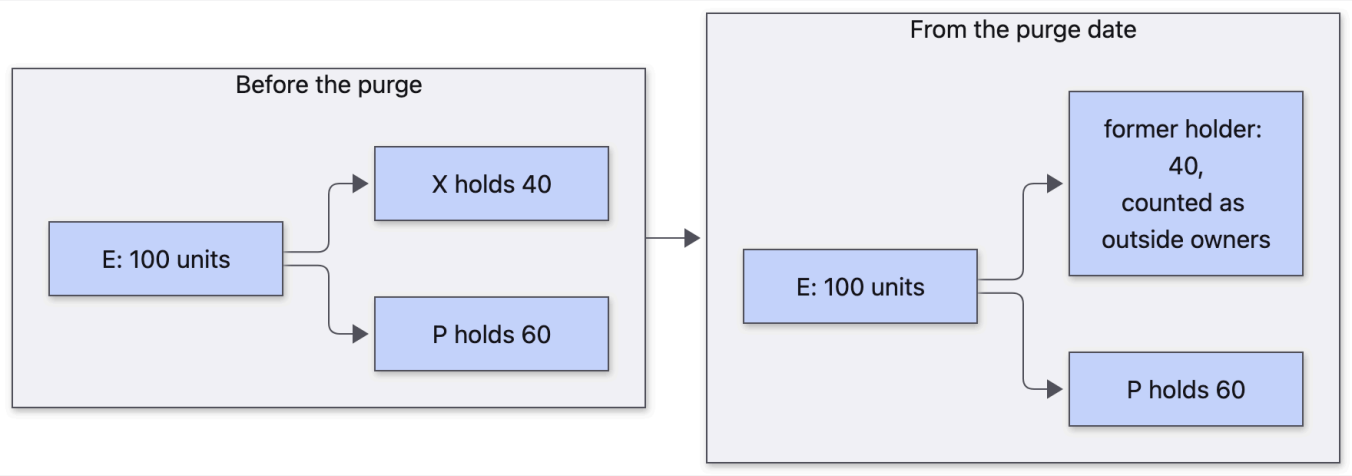
3. **Transfer** makes a named existing member an owner, in one transaction, with an `audit_event`. It names a current member who is not mid-deletion themselves, so no invitation is pending at deletion time. The entity is sole-owned, so the transfer is a sole-owner action needing no approval, and it sets the recipient as founding owner in the same transaction.
4. **Purge** runs `app.purge_entity` unchanged, per entity, one transaction each, with every §3.4 guard. The invoker still holds owner at that moment, so the sole-owner check passes on its own terms. The five-minute re-authentication window restarts the prompt if the run outlasts it. No job purges on the user's behalf.
5. **A failed provider revocation stops the run** (guard 6). Entities not yet reached stay as they were, entities already purged stay purged, and the account is not deleted. The user sees which and retries. Full atomicity is not available, since revocation is an outside call made by the worker.
6. **Tombstone last.** One transaction locks the user's `membership` rows and re-checks that no sole-owned entity remains, so a change made during the flow cannot orphan an entity. It also refuses while any entity with another owner still names the user as founder. It then applies §3.1: tombstone, credentials, sessions and memberships removed, connections `principal_invalid`, and every pending request on their entities re-evaluated (§3.1b). While the flow runs they remain an owner and may still approve or decline. The deletion's `audit_event` and the `entity.purged` events keep the tombstoned user id, which still resolves because the row is never deleted.
7. **Departing owner on a shared entity.** The same transaction writes a `notification` for every remaining member of each entity that keeps another owner, plus an `audit_event`. Nothing waits on them. Any other removal of an owner writes the same notification (PRD §5).

Who invokes: the account being deleted, and only it, for every step: signed in, with a fresh step-up including the second factor for an owner; never a job, another member or the operator CLI. The plan is held server-side and bound to the session that made it; another session starts again from the list (SECURITY §4.6, C-52; OQ-25 closed).

3.4b Purging an entity that holds units on other cap tables

SECURITY A-19, SecEng's proposal, designed here; before POLY-1.0. What the other entity's owners see in place of the name was ruled by the Principal on 2026-11-07 (OQ-33, option c).

Entity *X* holds units in *E*: *E*'s `equity_holding` rows name `holder_entity_id = X`. Those rows are keyed to *E*, append-only, and part of *E*'s history, so purging *X* must not touch them. It must not leave them dangling either, and *X*'s share must go somewhere.



- **A tombstone, always.** Every purge reduces *X*'s `entity` row to `id` and `purged_at`, every other column null (name, type, self_user_id, founder, tax attributes), instead of deleting it; a `CHECK` admits that shape only when `purged_at` is set. Always, not only when *X* is referenced: one code path, one deterministic assert, and a referencing row written concurrently cannot slip between a "referenced?" check and the delete. The cost is one id-only row per purged entity, which says only that some entity existed and when it was purged. The PRD §5 promise holds: every row of *X*'s data is gone. It is the same move as the user tombstone (§3.1).
- **How the tombstone is written.** `poly_purge` gains one role-targeted policy, `FOR UPDATE TO poly_purge USING (id = purge GUC)`, with UPDATE granted only on the columns it nulls and `purged_at`. No DELETE policy exists on `entity`. `holder_entity_id` FKs `entity` `ON DELETE NO ACTION`, so a bug that tried to delete the row instead fails loudly.
- **The append-only trigger is untouched.** *E*'s rows are neither updated nor deleted, and *E* gains no rows. So no exception is added to the `equity_*` trigger, the Σ invariant still holds as written, and no `cap_table` request on *E* is needed: *X*'s owner purging their own entity is not an edit of *E*'s cap table.
- **The share moves by a read rule, not a write.** The split on date *d* (§3.1a) adds one clause: a holder whose entity has `purged_at::date ≤ d` contributes 0, so its units fall into outside owners = total − Σ live units. One SQL function computes the split, and the economic view (§3.3c), allocated income and tax allocation (§4.5) all call it. Before the purge date the rows stand as recorded and the share remains the former holder's, so *E*'s past closes and reports still agree with themselves. Closes are never reopened, so a close already posted is unchanged.
- **Look-through ends on every date.** If *X* was an entity holder with its own holders, *X*'s cap table is purged with it, so nobody's economic weight in *E* through *X* can be computed for any date, even before the purge. The economic view therefore shows *X*'s units as outside owners

for a past `as_of` too. That is a deliberate inconsistency with *E*'s direct split, bounded to the look-through path, and the view says so when it applies. Keeping *X*'s cap table past the purge would break the PRD §5 promise.

- **No new rows naming a former holder.** `app.check_equity_holding()` refuses any new row whose holder is tombstoned. *E*'s owners reassign the units by an ordinary edit, a new row for another holder (through a `cap_table` request if *E* has several owners); a correction of *X*'s units before the purge date is not possible after it. A pending `cap_table` request on *E* whose payload names *X* fails re-validation at apply and is declined as stale.
- **The cycle and depth checks** keep treating the former edges as edges, which is harmless: a tombstoned entity can never hold or be held again.
- **Notice.** The tombstone UPDATE fires a `poly_graph` trigger, `app.notify_former_holder()`, AFTER UPDATE OF `purged_at`, which writes one `notification` to each owner of each entity whose cap table named *X*, in the purge's transaction. `poly_graph` already holds the graph-wide `equity_holding` read, owner `membership` SELECT and `notification` INSERT this needs, so `poly_purge` gains no read of other entities' rows. *X*'s own members are emailed as every purge begins (OQ-23); *E*'s owners are the other side.
- **What *E*'s owners see in place of the name (Principal, 2026-11-07, OQ-33).** The system shows "Former holder (purged <date>)" and copies nothing of *X*: the tombstone holds no name, and no snapshot is taken at purge. *E*'s owners may add an optional label they type themselves, recording what they already know. The label is *E*'s own data:
  - **Shape.** `equity_holder_label(entity_id, holder_entity_id, label, recorded_at)`, keyed to *E*, append-only like the rest of the cap table; the latest row recorded for a holder wins, and clearing writes an empty label. A trigger admits a row only for a tombstoned holder, so a live holder's name always comes from its entity.
  - **Who writes.** It is a cap-table edit: owner of *E* only, direct while *E* has one owner, a `cap_table` `change_request` otherwise (§3.1b). The multi-owner gate covers `equity_holder_label` along with `equity_*`, and `poly_govern` gets INSERT on it.
  - **Who reads.** Members of *E*, by ordinary RLS, like *E*'s other cap-table rows.
  - **Purge.** Purged only with *E*: its purge policy matches `entity_id`, never `holder_entity_id`, so purging *X* never touches it.
- **Tests (pgTAP):** purging *X* leaves *E*'s `equity_holding` rows unchanged and resolvable; the zero-rows assert passes with the tombstone present and fails if any other column is non-null; *E*'s split on the day before the purge includes *X* and on the purge date counts its units as outside; Σ economic + outside = gross still holds for *E* after the purge; a new row naming *X* is refused; a pending `cap_table` request naming *X* is declined as stale; each owner of *E*, including one the purging user cannot see, gets one notification; deleting the tombstone row

fails; the row shows "Former holder (purged <date>)" with no trace of *X*'s name; a label for a live holder is refused; with two owners, a direct label INSERT is refused; purging *X* leaves *E*'s labels intact.

3.5 Attachments

PRD §5 includes attachments (statements, receipts, closing documents) in export and purge, but v1 had no storage design (SECURITY A-14). The design:

- **Location:** Postgres `bytea` in an entity-scoped `attachment` table. It is RLS-covered like every other tenant table, and it automatically falls inside backup, export and purge. It works unchanged on every hosting target, including Vercel + Neon.
  - *Rejected for v1:* a filesystem or S3-compatible object store. That would be a second access-control system to keep in lockstep with RLS, and a second thing to back up, export and purge.
  - *Cost:* database size. At household scale, a few hundred MB per year is expected (unmeasured).
  - *Revisit* if the attachment table exceeds a size set in INFRA.
- **Limits:** 10 MB per file. Type allowlist: PDF, PNG, JPEG, and CSV/OFX (kept as import evidence). The type is sniffed server-side, and the client-declared type is never trusted. A SHA-256 is stored for de-dup and for the export manifest.
- **Serving:** only through an app route inside `withTenant`, with `Content-Disposition: attachment`, `X-Content-Type-Options: nosniff`, the sniffed `Content-Type`, and a restrictive CSP. Nothing is rendered inline.
- **Linking:** `entry_attachment` links an attachment to a journal entry, draft or account. A link doesn't change the entry, so attaching to a posted entry is allowed. Unlinking is manager+ and writes an `audit_event`. Attachment bytes themselves are insert-only.
- **Boundary:** uploads cross TB-11 (untrusted file ↔ parser). Attachments are stored, never parsed. CSV/OFX import files are parsed by the import adapter under TB-11's controls.

SecEng conditions (approved on security, all binding):

1. *Filename:* `Content-Disposition` uses the RFC 6266 `filename*=UTF-8''` form, with CR/LF and quotes stripped. The CSP is `default-src 'none'; sandbox`.
2. *Entity binding:* `attachment` and `entry_attachment` both carry `entity_id`, with composite FKs, so a link can never cross entities. Viewers read; manager+ uploads.
3. *Purge:* both tables get the `poly_purge` DELETE policy and the append-only trigger exception, and are included in the zero-rows-remain assertion (C-50).

4. *Quota:* a per-entity storage quota and an upload rate limit protect the disk. The numbers are set in INFRA.
5. *Isolation:* the bytes live in their own table (`attachment_blob`, 1:1 with `attachment`), so list and DTO queries never select `bytea`. Content is never parsed, and no AV scanning is an accepted consequence of that.

3.6 Custodial accounts and the chart by nature

Principal, 2026-11-06 (OQ-22, Option C): the ledger is organised by nature, with a custodian sub-account under each nature account.

- **Chart.** Under Assets sit the four nature classes that are also Allocation's top level: Cash and cash equivalents, Fixed income securities, Marketable securities, Alternative investment assets. Each has a sub-account per custodian ("Marketable securities, Schwab brokerage"). The former `Assets › Investments` grouping is gone.
- **Sub-account naming (2026-11-07).** A custodian sub-account is named institution + account type ("Schwab brokerage"). When one institution holds two custodials of the same type, the number mask is appended ("Schwab brokerage ··4821"); `posting` adds the mask to the existing sibling in the same transaction that creates the second, so neither name is ambiguous. The `custodial_account` keeps its own name, which the user may edit freely; the ledger name is derived once and is mutable after posting like any account name (§3.1).
- **Real estate and other no-custodian assets (Principal, 2026-11-07)** sit under Assets beside the four nature classes, not inside them: an `account` with `nature_class` null and no `custodial_account_id`, one per property, like Prepaid tax but with no authority behind it. It has no lots, no provider link and no reconciliation, and is carried at the value the user books (an opening balance, then manual entries). It counts in the balance sheet, net worth and the roll-up's "other assets" line (§3.3a), and never in Allocation's classes, targets or percentages.
- **Mapping is one-to-many.** A ledger `account` carries `custodial_account_id`; a custodial owns one ledger account per nature class it holds, enforced by a partial unique index on `(custodial_account_id, nature_class)`. A checking account owns exactly one, and a credit card one liability account with a null nature class. There is no junction: a ledger sub-account never spans two custodials. `posting` creates a missing sub-account in the transaction that first needs it.
- **Tax authorities** are custodial accounts with no provider link. Their prepaid-by-year and refund accounts (§4.5) have a null nature class, so the index does not limit them.
- **Lots** reference the custodial and the security. The security's class picks which of the custodial's ledger accounts its basis posts to.

- **Reclassifying a held security** moves its basis between nature accounts, so the mapping change posts that entry in the same transaction. The rulings leave this case open; this keeps §3.2.8 true.
- **Money-market funds** in the Cash class are carried as cash with no lots, at their stable value. §3.2.8 therefore covers the three securities natures only, and the cash figure below stays one balance.
- **The money-market sub-category** under Cash in Allocation therefore has no lot source. It reads the latest `reported_position` on that custodial of a security mapped to the sub-category, at the provider's reported value; the rest of the cash balance is plain cash. The class total is still the ledger cash balance, so the two always sum to it. A reported figure above the cash balance shows as a reconciliation gap, never as negative cash, and a custodial with no reported positions (a file or manual source) shows its whole balance as cash.

Roll-up per custodial:

- Book = the sum of its ledger account balances.
- Cash = its cash account balance.
- Market = cash + the sum of lot quantity × latest price over lots held there.
- Unrealized = market − book.

Reconciliation compares each ledger account against the reported balances and positions of the mapped provider account. Allocation reads the nature accounts across custodials (book) or lots by the security's class (market), and each ties to its own total by construction. Sub-categories come from `security_allocation` on lots (and, for money-market, on reported positions), never from the ledger. Accounts with no nature class are outside Allocation altogether.

Entries:

Event	Journal entry	Sub-ledger
Purchase	Dr securities nature / custodian; Cr cash / custodian	One lot
Sale	Dr cash / custodian; Cr securities nature / custodian for matched basis; Cr or Dr realized gain or loss	One `lot_match` batch, kind `sale`
Dividend	Dr cash; Cr dividend income	None
Return of capital	Dr cash; Cr securities nature	Lot adjustment

Event	Journal entry	Sub-ledger
Bond premium amortization	Dr interest income; Cr fixed income	Lot adjustment
Inter-custodial transfer	Moves basis between the two sub-accounts, no gain	Re-homes the lots

Re-homing keeps lots append-only. A transfer never UPDATEs a lot. It closes the old lot with a `lot_match` of kind `transfer`, carrying no proceeds, and writes a successor at the new custodial with the original acquisition date and basis and a `transferred_from_lot_id`. Holding period survives. A "Correct entry" on an entry that wrote a lot compensates through `lot_adjustment` or a reversing match in the same transaction.

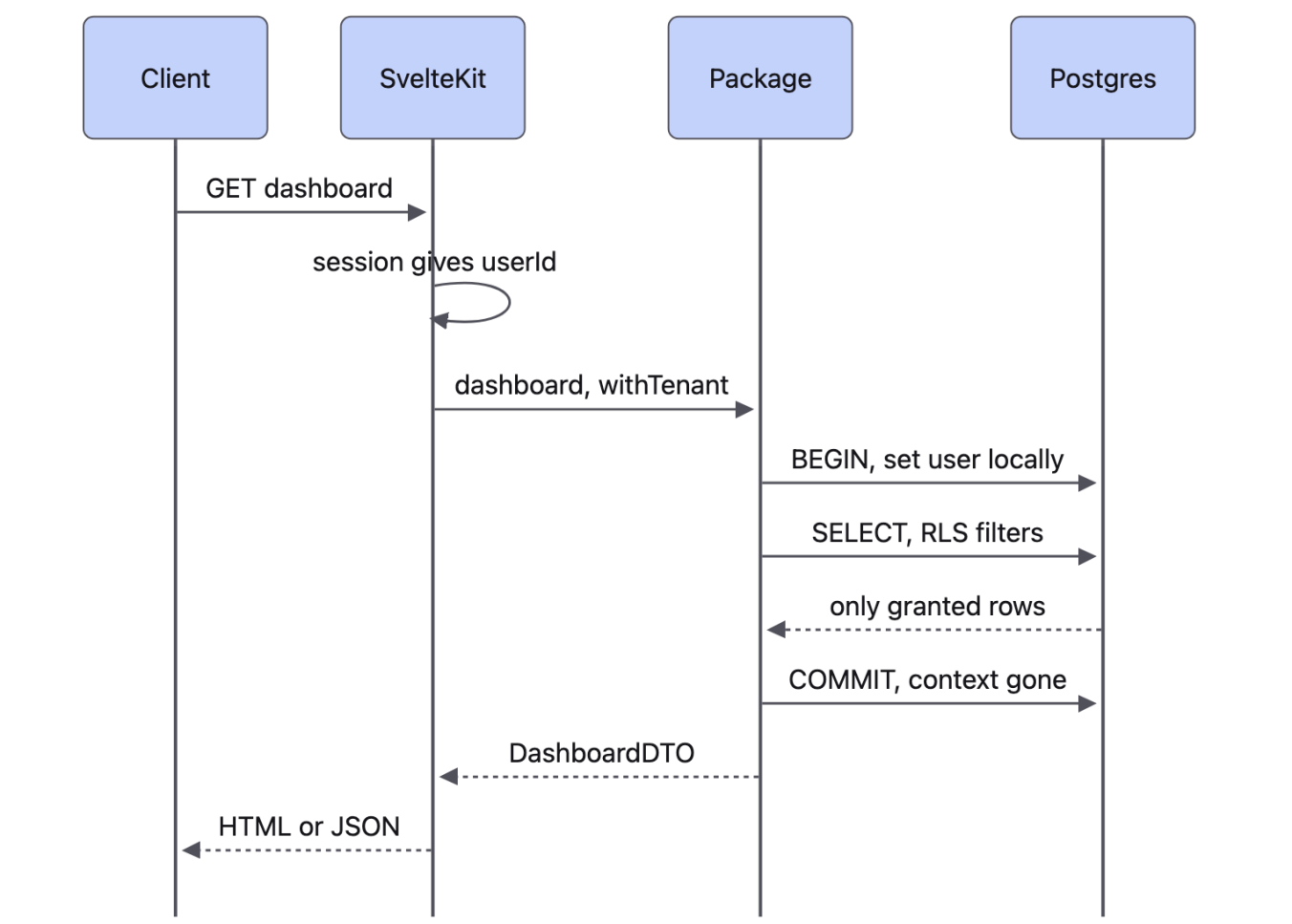
Imports from a brokerage split by nature before posting, using the security mapping. The custodial cash leg stays locked on the draft (G5).

Reporting facts are stored, not derived. `lot_match` holds proceeds and per-lot basis as values, so a later price or FX correction never changes a filed year. Form 8949 = lot-match rows of kind `sale` grouped by holding period, with adjustments from `lot_adjustment`. The trade blotter = lots (buys) + matches (sells), joined to their entries. Realized-gain income accounts split by holding period only, short-term and long-term, so ledger totals tie to the 8949 sections without reclassification. Jurisdiction lives on the tax side: one gain posts once, and the close charges tax on it per jurisdiction (§4.5).

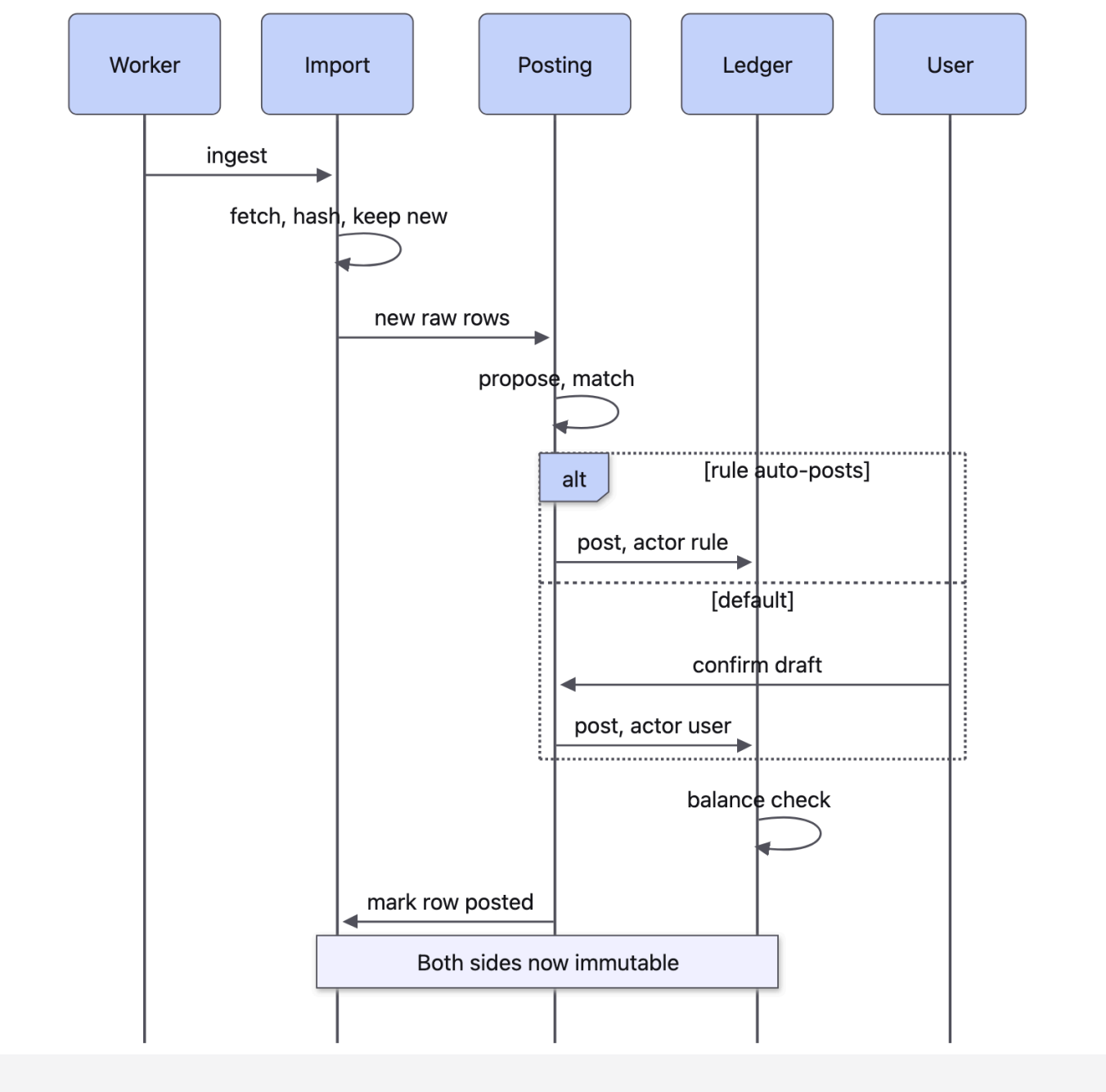
4. Data Flow

Only flows that surface a non-trivial design choice.

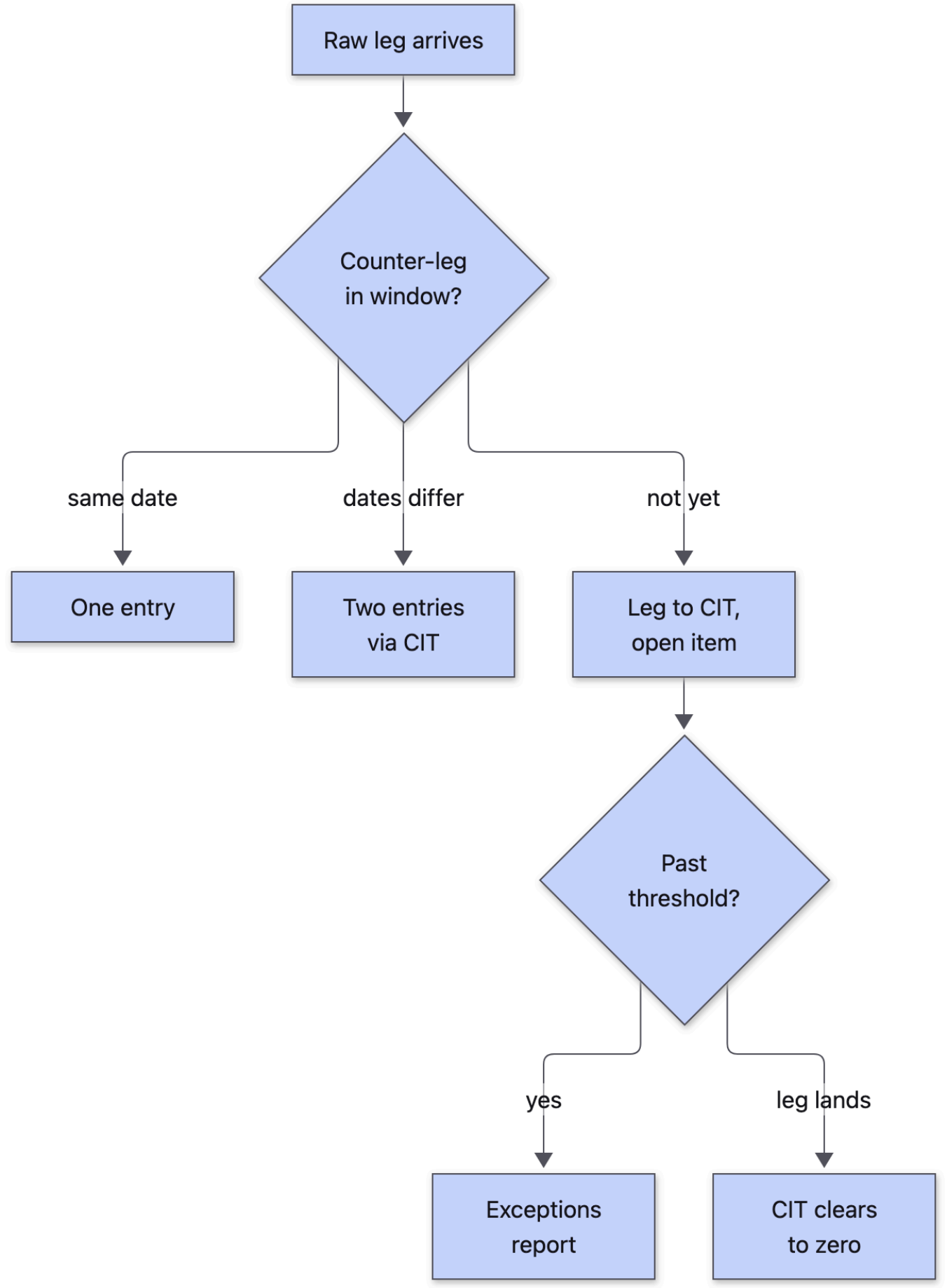
4.1 Any authenticated read (PRD Story 1, 8)



4.2 Import → draft → confirm (PRD Story 2, 3)



4.3 Transfer matching and Cash in Transit (PRD §5)

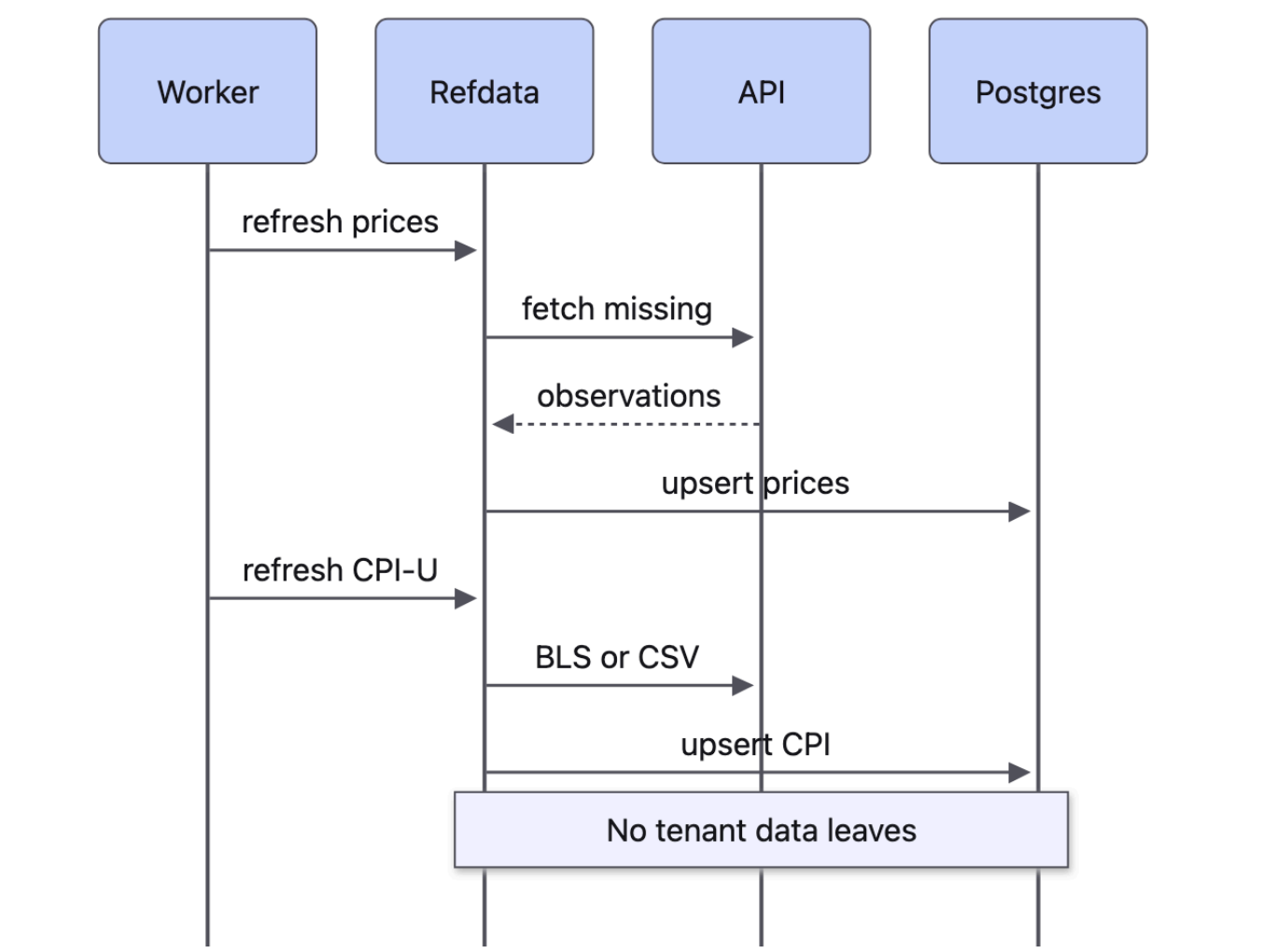

Raw leg arrives
Counter-leg in window?
same date
dates differ
not yet
One entry
Two entries via CIT
Leg to CIT, open item
Past threshold?
yes
leg lands
Exceptions report
CIT clears to zero

Only legs between two of the entity's own linked accounts are transfer candidates. A payment to an outside party has no counter-leg and never enters matching or Cash in Transit. That includes an estimated-tax payment made at the bank: it is categorized and posted like any other draft (§4.5). Matching lives in `posting` (the import side of posting), never inside `ledger.postEntry()`.

Between two entities (Principal, 2026-11-06), the same transfer posts one entry per entity, and the user picks the treatment per transfer on the draft: *contribution / distribution*, preselected, or *due-to / due-from*, which expects repayment. `transfer_match` stores the choice. Pairing and posting both sides needs manager on both entities. The choice is explained in plain language at that point (DESIGN).

Cash-in-Transit aging is computed on read by the exceptions report, not by a job (SECURITY §4.4).

4.4 Reference data: EOD prices and CPI-U (PRD Stories 5, 9)



4.5 Estimated-tax accrual (PRD Story 4)

No sequence diagram. `tax.estimate(entity, jurisdiction, asOf)` is a pure function over (a) year-to-date income by tax-treatment tag from the ledger, (b) the tax table row for that jurisdiction and filing year, (c) prior-year filed tax, typed per entity, per jurisdiction, per year, with the ledger's settled-filing figure shown beside it as a check once it exists (Principal, 2026-11-06), and (d) the balance of the custodial prepaid-tax sub-account for that jurisdiction and filing year, written *P* below. It returns:

- **Pending liability** = annualized tax to date − *P*.
- **Both safe-harbor values** and the **required payment to date** (the lesser of the two cumulative safe-harbor amounts).
- **Gap** = required payment to date − *P*, which is what is still owed by the next due date.

(Principal ruling, 2026-09-30.) A job unit calls `freeze()` the day after each period cutoff and appends a `tax_period_freeze` row. The estimate is recomputed on read and never stored, except the frozen period figures. These freezes follow the authority's installment cutoffs and post nothing; the reporting-period closes below are separate.

The app never pays tax. v1 initiates no payment. The user pays the tax authority at their bank, and the payment arrives as an ordinary imported transaction with no counterparty leg. Nothing is accrued between closes: the pending liability is a computed figure, and the tax liability account is touched only at a period close and at filing.

Accounts, seeded per entity for each jurisdiction *J*:

- *Prepaid tax — J*: an asset, held at the authority's `custodial_account`. It has one sub-account per filing year *Y*.
- *Income tax payable — J*: a liability.
- *Tax refund receivable — J*: an asset, held at the same custodial account.
- *Income tax expense — J*: an expense.

Grantor trusts and pass-throughs use the entity's existing *Distributions / owner draws* equity account in place of the expense account (PRD §5 tax-classification routing).

Who accrues (Principal, 2026-11-06): a taxpayer entity accrues its own tax at each close. That means an individual, a non-grantor trust, a C-corp, or a parent classified `joint filers`. A grantor trust, a pass-through business or a parent classified `pass-through to members` accrues nothing: its tagged income allocates to its holders by the cap table (§3.1a), and they accrue. A

parent classified `joint filers` ignores its cap table for tax: it accrues its whole income itself. A filing group combining a Household and its Persons on one joint return is not in v1 (OQ-28).

Allocation by the cap table (Principal, 2026-11-07). Replaces `tax_allocation_share`.

- **The split in force on each entry's date.** Each tagged income line of a pass-through entity *E* allocates to each holder by that holder's stake in *E* on the entry's posting date (latest rows with `effective_from ≤ posting_date`). A holder who joins in July takes nothing from June's income.
- **One level onward.** A holder that is an entity *B* is treated by its own classification. A taxpayer *B* (C-corp, non-grantor trust) adds the allocation to its own estimate and accrues it. A pass-through or grantor *B* allocates it onward to its holders by their stakes in *B* on the same posting date. Those holders are Persons, because §3.1a refuses a deeper chain, so nothing is left unallocated except the outside owners' share.
- **Outside owners** take their share off-system: it allocates to no tracked entity. With the old share table that was an error (shares had to total 100%); now it is the normal case of an untracked partner.
- **The read.** `tax.estimate` for any taxpayer entity, the Person-estimate definer of OQ-28 included, adds allocated income through one allowlisted definer, `app.allocated_income(taxpayer_entity_id, from, to)`: income-by-tag totals × stake, never rows, because a holder's owner may hold no membership on the entity. It is owned by `poly_stake`, guarded by a membership on the taxpayer entity, and its reads are confined by role-targeted policies to `app.held_entities()` (SECURITY C-27, A-18). It reads `equity_holding` and `equity_total` for the two entities in the chain and nothing more. Any date range is accepted, with no period guard (P2). A purged holder's share is outside owners from the purge date (§3.4b).
- **A cap-table change after a close** is late data: closes are never reopened, and the next close's delta carries it, as for a back-dated import.

Why a sub-account per filing year: a January Q4 payment belongs to the prior year. When a payment draft is created, the rule proposes the filing year from the due-date table (the year whose next unpaid due date the payment falls on or before), and the user can change it before confirming.

Reporting-period close (Principal, 2026-11-06). The accrual is trued up at each close, monthly by default, per the entity's cadence. Tax is a real liability, so net assets should not show it as one lump a year; and tax is nonlinear, so accruing per entry would mean correcting earlier entries.

- **The close.** At period end a job unit raises a notification; it never posts. The user confirms one entry per jurisdiction, dated the period end. `tax.close()` writes the entry and its `tax_close` row in one transaction.
- **The amount** is *A* = the annualized tax to date from `tax.estimate` at the period end, before prepaid is subtracted, because Prepaid tax stays a separate asset on the balance sheet. Using the pending liability, which already nets out *P*, would count prepaid twice.
- **The delta** is *A* − the last `accrued_to_date` for that filing year, so the first close of a year posts the full figure and later ones the change. It is taken from `tax_close`, not the payable balance, because in the months before filing that one account also holds the prior year's accrual.
- **Late data.** A back-dated import after a close is picked up by the next close's delta. Closes are never reopened. A skipped period needs no catch-up, and a cadence change applies from the next period end.
- **A wrong close** is fixed by closing that period again: a new revision posts the difference. "Correct entry" is not offered, so the record and the ledger cannot part.
- **Between closes,** the balance sheet and NAV reports add one computed line, "estimated tax since last close": `tax.sinceClose()`, the estimate at the report date minus the last `accrued_to_date` on or before it, across unfiled years. Net assets are right on any day with no posting on read. The line is labelled as an estimate, is not a ledger balance, and stays out of the trial balance.
- **Year end** is simply the last close of the year. The line and the status "last closed through *date*" arrive with tax in POLY-1.0; the POLY-M5 statements render without them until then.

Event	Source	Journal entry
Between closes	none	No accrual entry. The pending liability and the since-close line are computed on read.
Estimated payment	Imported bank debit, confirmed by the user (or a rule the user opted in)	Dr Prepaid tax — J — Y / Cr Bank
Period close in Y (to-date tax *A*, last accrued *E*), one entry per jurisdiction	User-confirmed; taxpayer entities only	Dr Income tax expense — J / Cr Income tax payable — J (*A* − *E*), reversed if the estimate fell. A pass-through or grantor entity posts nothing.

Event	Source	Journal entry
Filing (actual tax *T* for year Y, accrued through the last close *E*, prepaid balance *P*)	Entered by the user from the filed return; one multi-leg entry dated the filing date	True-up: Dr Income tax expense — J (*T* − *E*) / Cr Income tax payable — J, reversed if *E* > *T*. Settle: Dr Income tax payable — J / Cr Prepaid tax — J — Y min(*T*, *P*), leaving *T* − *P* payable if *T* > *P*. A pass-through or grantor entity that prepaid has no payable: Dr Distributions / Cr Prepaid tax — J — Y. If *P* > *T* and the user elects a refund: Dr Tax refund receivable — J (*P* − *T*) / Cr Prepaid tax — J — Y (*P* − *T*). If *P* > *T* and the user elects to apply it to next year: Dr Prepaid tax — J — Y+1 / Cr Prepaid tax — J — Y (*P* − *T*). Penalty or interest, if any: Dr Tax penalties expense / Cr Income tax payable.
Balance due paid	Imported bank debit, confirmed	Dr Income tax payable — J / Cr Bank
Refund received	Imported bank credit, confirmed	Dr Bank / Cr Tax refund receivable — J

After filing and settlement, Prepaid tax — J — Y and Income tax payable — J both read zero. The liability is recognized at each close, trued up to the return at filing, and settled from the prepaid account, so any remainder is the true balance due. Custodial reconciliation (POLY-72) can compare the prepaid sub-account against the authority's account transcript. A filing entry made in error is fixed with "Correct entry", like any other entry.

5. Tech Stack & Rationale

Justify every row. "We know it" is a valid reason — write it down.

Layer	Choice	Why	Alternatives considered
Language	TypeScript (strict) end to end, Node LTS	One language across UI, API, jobs and schema; the Principal's prior project (mosko-fintech) is TS/SvelteKit.	Go / Python backend (a second language for no gain at this scale)
Frontend framework	SvelteKit + shadcn-svelte + Tailwind; LayerChart	Kickoff §2.9: owned component source, headless primitives; shadcn-svelte charts are LayerChart. Monarch/Origin feel from dashboard-01.	Flowbite Svelte (rejected: fixed look, ApexCharts)
API	SvelteKit `/api/v1` routes, Zod schemas, OpenAPI generated and committed (CI fails if stale)	Thin wrappers over packages; one contract for native clients and a future MCP server.	Separate API server (tRPC / Fastify): a second deployable for no gain
Datastore	PostgreSQL, latest GA major (18 at Plan time; bump deliberately)	RLS, deferred constraint triggers, `numeric`, SECURITY DEFINER — the whole integrity story lives here.	SQLite (no RLS), external ledger services (excluded by kickoff §2.5)
ORM / query layer	Drizzle ORM	TS schema as the single source; RLS policies and roles declared in schema; BetterAuth ships a Drizzle adapter, so auth tables share one migration chain. See §6.2.	Kysely + generated types; Prisma (weak RLS / raw-SQL story, heavy engine)

Layer	Choice	Why	Alternatives considered
Migrations	drizzle-kit `generate` (SQL files, committed) + hand-written custom SQL migrations for triggers / functions / roles, applied by a one-off migrate container	Plain SQL in the repo, a tracking table, never hand-run (kickoff §2.13). Conditional on the M1 spike — see §6.2.	dbmate (runner-up), Atlas, graphile-migrate, Supabase CLI (mosko-fintech incumbent)
General ledger	Home-grown schema in our Postgres (build)	See §6.1.	pgledger (adopt), mosko-fintech's journal model
Money arithmetic	Postgres `numeric`; decimal strings on the wire; a decimal library (e.g. `big.js`) in TS	No binary floats anywhere near money.	Integer minor units (awkward for share quantities and FX rates)
Auth	BetterAuth (Drizzle adapter), OAuth: Google / Apple / GitHub; MFA; bearer plugin; email via Resend	Kickoff §2.8: identity in our DB, no leased tier.	Clerk (rejected: external identity store), Lucia (maintenance mode)
Jobs	Plain idempotent job units; VPS: `apps/worker` with a Postgres-backed scheduler (pg-boss); Vercel: Vercel Cron → an authenticated internal route that runs the same unit	No Redis, no extra service; Postgres is already the system of record. Dual-target rule (kickoff §2.8).	BullMQ + Redis (extra service), system cron in the container (no retries or visibility)
Monorepo	pnpm workspaces + Turborepo (affected-only)	Kickoff §2.10; per-package tests and versions.	Nx (heavier), plain pnpm filters (no caching)

Layer	Choice	Why	Alternatives considered
Tests	Vitest per package; Postgres via a disposable container (Testcontainers or CI service); pgTAP for the RLS / trigger battery; Playwright smoke suite (login, import a batch, confirm a draft, view a statement)	Kickoff §2.13. DB invariants are tested in SQL where they live.	Jest (slower ESM story)
Hosting / runtime	Primary: Hetzner VPS, Docker Compose (Caddy, web, worker, Postgres). Secondary: a user-owned Linux box on a local network, same Compose, no inbound from the internet (§7.8). Tertiary: Vercel + Neon, CI-verified	Principal, 2026-11-06: the two Compose paths share one deployment shape; the Vercel path is the zero-ops last resort, an adapter-only difference (kickoff §2.8).	Fly.io, Railway (leased), Kubernetes (overkill)
Reverse proxy / TLS	Caddy	Automatic HTTPS in one short config file, which suits the novice-operator runbook.	Traefik, nginx + certbot
Observability	Structured JSON logs (pino) with a redaction list (credentials, tokens, account numbers); Postgres slow-query log; uptime ping. OTel export is optional	Cheap stack. Logs must be safe by construction because PII lives in memos.	Hosted APM (subscription)

6. Plan-phase Research

The two research items the PRD (§10) assigns to the architect. Each ends in a decision candidate for `process/DECISIONS.md`.

6.1 General ledger: build vs. adopt

Scope (kickoff §2.5): the ledger lives in our Postgres under our RLS. External services (Formance, TigerBeetle, Blnk), plain-text engines and ERPs are already out. The question is only: a home-grown schema, or a library that manages tables inside our own Postgres.

Candidate	What it is	Fit against our requirements
pgledger (pgr0ss, MIT)	A single SQL file of tables, functions and views: accounts with denormalized balance + version, two-legged transfers, entries carrying previous/current balance.	✕ No tenant column, no RLS. ✕ The primitive is a two-legged transfer; our entries are multi-leg with one posting date. Batched transfers approximate this, but reports would have to re-group them. ✕ Single currency per account and no `native / base / fx_rate` per line. ✕ No hierarchical CoA, normal side, custodial flag, or reversal link. ✕ No versioned migrations yet (its own TODO). Upgrades would mean diffing a vendored file.
TS/Node double-entry libraries (e.g. Medici)	App-level ledger libraries.	✕ Medici is MongoDB. We found no mature TS library that manages Postgres tables under host-app RLS (web survey, 2026-09-27).
mosko-fintech journal model	Single-entry `account_trans` legs grouped post-hoc by `journal`; GL derived by an imputation function (`fn_gl_entries`); `users_id` as sole tenant anchor.	✕ Not a stored double-entry GL, and direct-owner tenancy contradicts our membership model. Its *patterns* port well, though (below).

Candidate	What it is	Fit against our requirements
Home-grown	~6 tables (§3.1) + 2 triggers + a posting service.	✓ Fits every requirement directly. Cost: we own correctness. We mitigate that with DB-enforced invariants (§3.2), property-based tests (random balanced / unbalanced entries), and the RLS battery.

Recommendation: build. A GL core is small. The hard parts here are membership tenancy under RLS, multi-leg / multi-currency-ready lines, the securities sub-ledger, and declarative reports, and no candidate library provides any of them. Adopting pgledger would mean forking it to add all of them, which is building with extra steps.

Borrow from pgledger: append-only entries as the source of truth; optional per-line running balance (deferred until a measured need — at household scale, `SUM` over an indexed `(entity_id, account_id, posting_date)` is expected to be fast (unmeasured; the dashboard <2 s NFR is the guard)).
Borrow from mosko-fintech: the immutability-trigger fence (004), the append-only lot-match junction with `match_seq` (032), price observations with source priority (019), structural de-dup by unique index + `ON CONFLICT DO NOTHING` (021), the RLS inversion self-test (tests/00), account-number masking (002), and the NOLOGIN-owner / `set role … reset role` migration ownership pattern. **Leave behind:** direct-owner `users_id` tenancy, post-hoc journal grouping, and Supabase-specific `auth.uid()` / JWT policy clauses.

6.2 Migration tool

Constraints (kickoff §2.13): migrations are files committed with the code and never hand-run; CI applies them to a fresh Postgres on every run; the VPS runs a one-off migrate step before the app starts; Vercel runs it as a build step against Neon. On top of that, our schema is heavy on triggers, SECURITY DEFINER functions, roles and RLS policies.

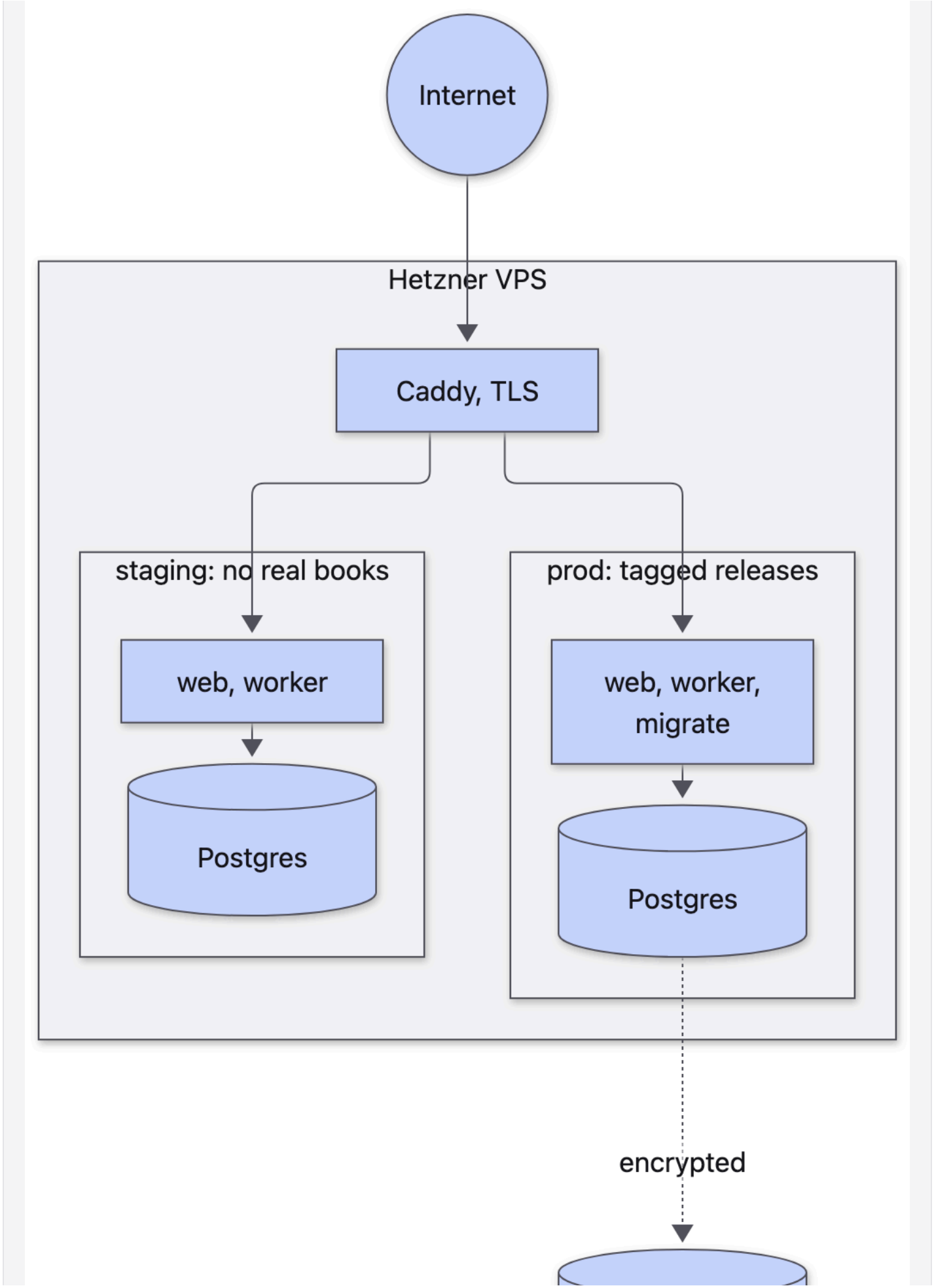
Incumbent check (mosko-fintech): 120 hand-written plain-SQL files applied with `supabase db push`, tracked in `supabase_migrations.schema_migrations`. Lessons worth keeping: a tracking table is non-negotiable (a bare `psql -f` loop was ruled out there); migrations must be tested *through the real apply path*, not psql (their `set local role` was a measured no-op under `db push`); objects need a NOLOGIN owner distinct from the applying login. The tool itself doesn't carry over: Supabase CLI assumes a Supabase stack we don't run.

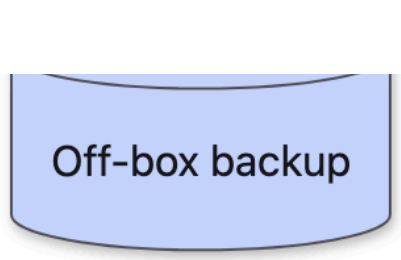
Tool	For	Against
Drizzle Kit (recommended)	TS schema → generated SQL; `pgPolicy` / `pgRole` / `enableRLS` in schema; custom SQL migrations in the same chain for triggers and functions; BetterAuth's Drizzle adapter keeps auth tables in one chain; the query layer and the migration tool share one schema, so types never drift.	Triggers and functions aren't modelled, so they live in hand-written files. Policy diffing maturity is unmeasured — hence the spike.
dbmate	Plain SQL, tiny, language-agnostic, boring; tracking table; trivially run in a container.	No schema-as-code. Types would come from a codegen step (Kysely), and BetterAuth tables go through its own CLI generator.
Atlas	Declarative diffing, strong linting.	Some Postgres object types sit in the paid tier (unmeasured); a second schema language (HCL) alongside TS.
graphile-migrate	Excellent SQL-first dev loop (`current.sql`).	Postgres-only SQL workflow with no TS schema; smaller community.

Recommendation: Drizzle Kit, gated by an M1 spike. Spike acceptance: (a) a `pgPolicy` round-trips through `generate` twice with no spurious diff; (b) BetterAuth's generated Drizzle schema migrates cleanly; (c) a custom SQL migration containing a trigger and a SECURITY DEFINER function applies through the real runner as `poly_migrator` with objects owned by `poly_owner`, and the tracking row is written; (d) the same runner works against Neon from a Vercel build step; (e) FORCE RLS, a role-targeted `pgPolicy` (`to:`), and a definer function owned by a non-owner NOLOGIN role (`poly_authz` / `poly_purge`) all round-trip through `generate` and apply as `poly_migrator` (SECURITY A-13). If (a), (c) or (e) fails, fall back to **dbmate + Kysely codegen**. That fallback changes only `packages/db`, since every other package consumes `withTenant()` and typed table objects.

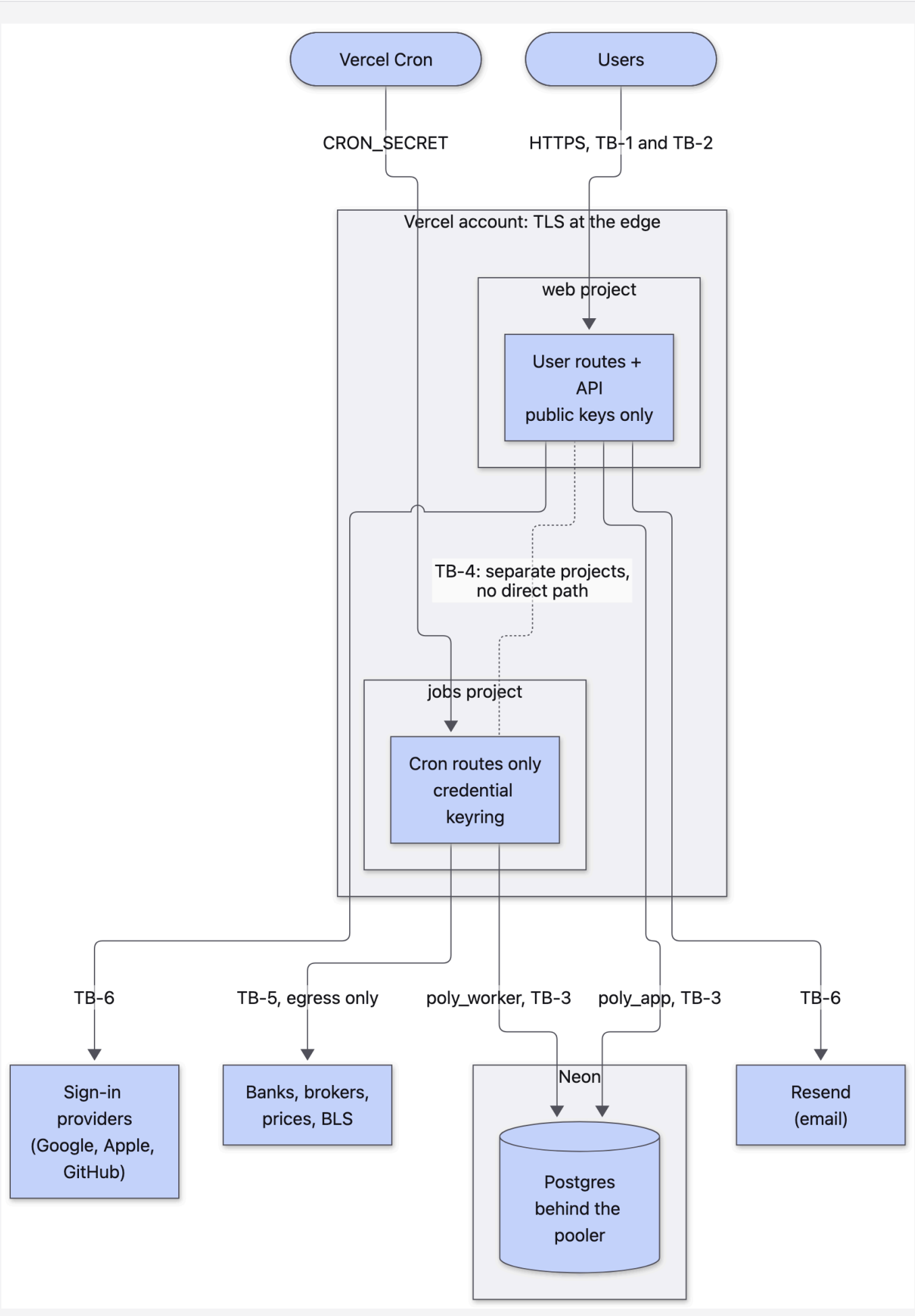
7. Deployment Topology

Environments, network paths, trust boundaries. DevOps owns the INFRA-level detail; cross-reference SECURITY.


Internet
Hetzner VPS
Caddy, TLS
staging: no real books
web, worker
Postgres
prod: tagged releases
web, worker, migrate
Postgres
encrypted



Co-located: the Docker Compose path on the Hetzner VPS. One box runs everything behind Caddy.


Vercel Cron
Users
CRON_SECRET
HTTPS, TB-1 and TB-2
Vercel account: TLS at the edge
web project
User routes +
API
public keys only
TB-4: separate projects,
no direct path
jobs project
Cron routes only
credential
keyring
TB-6
TB-5, egress only
poly_worker, TB-3
poly_app, TB-3
TB-6
Sign-in
providers
(Google, Apple,
GitHub)
Banks, brokers,
prices, BLS
Neon
Postgres
behind the
pooler
Resend
(email)

Split: the Vercel + Neon path (§7.6). What differs from Compose: there is no Caddy, because Vercel terminates TLS. There is no worker container: `jobs` runs the same job units when Vercel Cron calls them, capped at once daily on the Hobby plan. Postgres is Neon's, reached through its pooler. Secrets are Vercel env vars instead of mounted files. Migrations run as a build step. The `web` ↔ `jobs` line is the same TB-4 as `web` ↔ `worker` on the box, held by a project boundary instead of a container boundary.

- **Trust boundaries** (numbering and controls from SECURITY §3):
  - TB-1: internet ↔ Caddy.
  - TB-2: clients ↔ app.
  - TB-3: app / worker ↔ Postgres (limited roles, RLS).
  - TB-4: `web` (seals with public keys only) ↔ `worker` (holds the private keyring, mounted into worker only; plaintext exists only inside one `open()` call frame). Redefined by SECURITY §6.3.
  - TB-5: worker ↔ providers (egress only).
  - TB-6: app ↔ sign-in providers (Google, Apple, GitHub) and Resend.
  - TB-7: box ↔ backup store (encrypted before it leaves the box).
  - TB-8: operator SSH ↔ box (superuser; accepted risk R-1).
  - TB-9: ordinary write path ↔ purge path.
  - TB-10: prod ↔ staging compose projects on one host (shared kernel; accepted risk R-4).
  - TB-11: user upload (CSV / OFX, attachments) ↔ web parser.
  - TB-12: supply chain (GitHub CI / Actions / Renovate / registry) ↔ prod images. Whoever can publish an image that prod runs owns the box.
  - TB-13: operator SSH tunnel ↔ loopback-bound Admin listener, never routed by Caddy. Placeholder, defined under POLY-81 (OQ-15).
- **Postgres is never exposed publicly.** It lives on the compose network only; the operator reaches it through `docker compose exec` or an SSH tunnel.
- **Only Caddy publishes ports** (SECURITY C-70): every other service (`web`, `worker`, Postgres, migrate) has no `ports:` mapping at all, reachable only over the compose network. Prod and staging run as separate compose projects on separate compose networks — no shared network, so a staging container can't reach a prod one by DNS name even if TB-10's shared kernel is compromised. No Docker socket is ever mounted into a container. Every container runs non-root, `read_only`, with `cap_drop: [ALL]` and `no-new-privileges`; a compose-policy lint enforces this in CI (C-70).
- **The firewall is the Hetzner Cloud Firewall**, not host `ufw` (SECURITY C-71): it sits outside the host, so a Docker-published port can't bypass it the way it bypasses `ufw`. Only 22 (SSH,

key-only, no root login), 80 and 443 are open. Unattended security upgrades are enabled on the box.

- **Environments:** *dev* — local compose (Postgres + web + worker) with the `fake` provider; *CI* — fresh Postgres service per run; *staging* — same VPS, separate compose project and database, no real books, auto-deploys every merge to `main`; *prod* — tagged releases from `v1.0.0` only (kickoff §2.13). No real books exist before v1.0 (Principal, 2026-11-06). Staging on the same box is a cost trade-off (§10).
- **Hosting order (Principal, 2026-11-06):** primary the Hetzner VPS; secondary a local-network box, the maximum-security option (§7.8); tertiary Vercel + Neon. These are deployment targets, not separate architectures.
- **Tertiary target:** the same app built with `adapter-vercel` against Neon, as **two Vercel projects from the same repo** (resolves OQ-13; SECURITY C-61 — see §7.6). CI builds it and runs the smoke suite against a Neon branch (or a local Postgres standing in for one); no Vercel-specific APIs anywhere.
- **Sizing:** CX32 (4 vCPU / 8 GB RAM) as the starting VPS — see §7.3.

7.1 Environments

Environment	Data	Deploy trigger	Access
dev	`fake` provider only	local compose, manual	developer machine only
CI	fresh Postgres service per run, torn down after	every PR	GitHub-hosted runner only
staging	The deterministic `fake` source as the repeatable fixture, plus sandbox data through real adapters: Plaid Sandbox, and SimpleFIN's demo token. Never real books.	auto, every merge to `main`	same VPS, separate compose project + DB; reachable behind Caddy basic auth / IP allowlist — not indexed, not linked from prod

Environment	Data	Deploy trigger	Access
prod	real data, from v1.0	tagged releases only, via `/merge-pr`. A `0.x` tag cut before GA is a staging release, and the prod poll ignores it.	public, TLS via Caddy

7.2 Backup & Disaster Recovery

Resolves OQ-9.

- **Staging co-location:** staging stays on the prod VPS (confirms §10's trade-off). Staging holds no real books, so its compromise has no ledger-confidentiality blast radius; splitting to a second box roughly doubles the fixed monthly cost for a single-household deployment with no compliance driver forcing separation. Revisit only if the app is offered to other self-hosters at meaningfully larger scale, or staging starts touching anything beyond fake data.
- **Backup store:** Hetzner Storage Box (same account, cheap) as the default, configured for **append-only** access (a restricted SSH/SFTP subaccount with no delete/overwrite permission on existing objects — Hetzner Storage Box's "no substitution" / read-append mode **(unmeasured)** until the C-73 cannot-delete test in the restore drill below actually passes against it); an S3-compatible bucket with Object Lock in compliance mode is the documented drop-in alternative for self-hosters outside Hetzner. Either way the store credential itself cannot delete or overwrite history (SECURITY C-73's own test: "a test that the store credential cannot delete").
- **Mechanism, revised per SECURITY §6.6 (asymmetric recipient, not restic's symmetric repository password):** a nightly `pg_dump --format=custom` logical backup and continuously-shipped WAL segments are each encrypted with `age` (X25519, the same primitive family as the credential keyring's HPKE scheme) to a **recipient public key**, then uploaded. The box holds only the age *public* key — sufficient to encrypt, useless to decrypt. This is a deliberate change from an earlier `restic`-based draft: restic's repository password is symmetric, so a box holding it to *write* backups could also *read* them, which fails SECURITY §6.6's "box holds only the public half" requirement outright. Dedup/incremental snapshotting is given up for this; at single-household data volumes the cost is negligible.
- **Key custody:** in plain terms, backups are locked with a key that never lives on the server, so stealing the server does not open them, and losing both offline copies of that key means the

backups can never be opened. The age keypair is generated once at bootstrap, off the box. The public half is placed on the VPS (non-secret — it can only encrypt). The private half lives in the operator's password manager and a printed offline recovery copy per the runbook; it is **never** on the VPS, and is a wholly separate key from the credential keyring (§6.1/§6.4) — the two are never stored together, so a single compromised secret can't unwrap both live credentials and backup history. Rotation (new keypair) is a manual runbook step on suspected compromise; old backups stay readable under the old private key, which the runbook says to retain until its last covered backup expires.

- **RPO:** ≤ 15 minutes via continuous, individually-encrypted WAL segment shipping; the nightly full dump is the tested fallback restore point (RPO 24h) if WAL replay is unavailable.
- **RTO:** ≤ 4 hours for total loss of the VPS — provision a replacement box from the same IaC, decrypt the latest dump and WAL with the offline private key, restore, replay, redeploy the last tagged release image. ≤ 30 minutes for a bad-migration rollback that doesn't need a box rebuild (§8.1).
- **Restore drills:** monthly (SECURITY C-73, tightened from this section's earlier quarterly draft), scripted, and run **off the prod VPS** — the operator's own machine, or a throwaway scratch VPS spun up and destroyed for the drill — never a scratch compose project on the box itself: that would put the age private key on the VPS for the drill, which this section's own key-custody rule (§7.2 above) forbids outright, drill or not. Checked against a fixture query (trial balance sums to zero); logged as a runbook checklist item, not assumed to work from the backup job succeeding alone. The drill script also asserts the store credential cannot delete an existing object (C-73's second test, and the measurement the append-only bullet above is waiting on).

7.3 Sizing

CX32-class (4 vCPU / 8 GB RAM) rather than CX22 (2 vCPU / 4 GB): Postgres (prod + staging) + web + worker + Caddy fit comfortably inside 8 GB idle, and the extra headroom absorbs backup/WAL-shipping runs, migration bursts, and the staging Playwright smoke without contending with prod. **Unmeasured** — confirmed against real resource metrics after POLY-M1 lands; downgrading to CX22 is a one-line IaC change if it proves oversized.

7.4 Observability & Operations

- **Logs:** web / worker emit structured JSON to stdout (one schema — `ts, level, req_id, entity_id?, event, …`; exact fields agreed with backend-lead before POLY-M1 ships anything that logs). Docker's `json-file` driver with `max-size` / `max-file` rotation captures

them; `docker compose logs` / `journalctl` is the query surface at this scale. No log-aggregation service until a second box or third-party hosting is in play.

- **Metrics:** a `/api/v1/health` route reports DB connectivity, last successful migration, and last successful backup timestamp. Polled by a free external uptime monitor. Container resource metrics via `docker stats` on demand; no Prometheus/Grafana until scale or a second operator justifies the extra service.
- **Alerts** (all by email — no on-call rotation for a household of one): backup job failure or a backup older than 26h; `/health` failing for >5 minutes; CI deploy-to-prod failure; **a ledger hash-chain break**, SEV1, from the weekly per-entity verifier (§3.2.9), which also emails the entity's owners; a verifier run that has not completed in 8 days alerts as well, so a stopped verifier is not silence.
- **Dashboard:** `/health` plus `docker compose ps` is the v1 dashboard — the thing the team-lead can point to at launch. A Grafana-class dashboard is a documented future initiative once observability volume justifies the extra container.

7.5 Secrets Delivery & Postgres Config

DevOps's INFRA-level implementation of SECURITY §6.2 and §12.3's remaining asks.

- **Secrets as files, not env vars (SECURITY §6.2, C-74):** every operator secret lives under `/etc/polycarpic/secrets/` on the host (root:root, files `0400`, directory `0700`), mounted into containers as Docker Compose `secrets:` at `/run/secrets/<name>`. Apps read them through a `*_FILE` convention (e.g. `DATABASE_PASSWORD_FILE=/run/secrets/db_password`), never a bare env var — `docker inspect`, `/proc/<pid>/environ` and an error reporter dumping `process.env` all leak plain env vars, never a file only the right container can read. Per-container scoping: `web` never gets the credential private keyring or the backup private key; only `worker` mounts the keyring secret. Staging gets its own distinct secret values and its own keyring, which seals sandbox tokens only; it never holds a real provider credential. No secret is ever baked into an image or passed as a build arg; the app refuses to start outside development if a required secret file is missing.
- **Postgres logging (SECURITY C-75):** `log_parameter_max_length = 0` and `log_parameter_max_length_on_error = 0` (bound parameter VALUES never reach the log, even on error — a sealed credential's ciphertext is one thing, but a query parameter carrying a raw token before it's sealed is exactly what this guards), `log_statement = 'ddl'` (schema changes only, never row data). Set in the Postgres compose service's config, asserted by a config-check in the DB battery (§8's DB battery row).

- **Connection-pooler mode, asserted on both targets:** the VPS's Postgres and Neon (the tertiary target) must both run in a pooler mode where `set_config(..., true)` (the transaction-local RLS GUC, ARCH §3.3) is safe under connection reuse — transaction-mode pooling, not session-mode, on any pooler in front of either target (PgBouncer on the VPS if one is added later; Neon's own built-in pooler). A CI assertion connects through each target's pooler and confirms the GUC resets between transactions on a reused connection (the same pool-reuse property qa-engineer's pgTAP battery checks at the RLS layer, asserted here at the pooler-config layer instead).

7.6 Vercel Tertiary Target: Two-Project Split (OQ-13)

Resolves OQ-13; SECURITY C-61/R-3 @ 3c66ce1 (phase/plan-security). DevOps builds the CI check; the split itself is a SecEng ruling.

- **Topology:** the Vercel target is two Vercel projects deployed from the same repo, not one. `web` exposes only user-facing routes, holds only public keys and the `poly_app` DB credentials. `jobs` exposes only cron routes (each checking `CRON_SECRET` with `timingSafeEqual`), holds the credential keyring and the `poly_worker` credentials. This is the same TB-4 (`web` seals / `worker` opens) split the VPS target already has via separate compose services — Vercel just can't isolate them inside one deployment, so it takes two deployments instead.
- **CI check, three legs (SECURITY C-61's own "build tests"):**
  - *Route manifest vs. allowlist.* Each project's build output (SvelteKit's route manifest under `.vercel/output/functions/`) is diffed against a checked-in allowlist per project — an exact-set match, not a subset check, so a route silently added to the wrong project's allowlist is caught too, not just one silently missing. `jobs`'s allowlist is exactly the cron routes; `web`'s is everything else.
  - *No keyring reference in the `web` build.* A bundle-scan (grep the built output for the keyring module's resolved import path, same mechanism the existing `packages/secrets` import-boundary lint uses source-side) asserts `open()` and the keyring module never got bundled into `web`'s build artifact — catches a route misclassified into `web`'s allowlist that would otherwise pass the manifest check by having the right PATH but the wrong CODE behind it.
  - *Node build 404s job routes.* The VPS's `adapter-node` build (one image, `web` / `worker` / `migrate` entrypoints, §8.1) is hit on its `web` entrypoint at each cron route path and must 404 — job routes exist only behind the `worker` entrypoint there, and this is the cross-check that the VPS and Vercel targets enforce the identical TB-4 boundary by two different mechanisms (separate compose service vs. separate Vercel project) without drifting apart.

- **Env check in CI (C-61):** a Vercel API call lists each project's configured env var *names* (never values) and asserts the keyring variable name is present in `jobs` and absent from `web` — catches an operator (or a future IaC change) setting it on the wrong project directly in the Vercel dashboard, which the build-time checks above can't see.
- **Cron frequency — measured:** Vercel's Hobby plan caps cron jobs at once per day, with timing guaranteed only within the hour (Pro lifts this to per-minute). Since Hobby is already the ceiling this self-hoster fallback target targets, daily is not a choice DevOps is making for cost reasons — it is the only cadence Hobby allows at all. Import polling, EOD prices and period-end jobs on the Vercel target therefore run once daily, a real (accepted) data-freshness gap against the VPS target's more frequent worker loop; SecEng's framing ("daily polling is acceptable for this fallback target") holds given Vercel is the tertiary target, the last resort of the hosting order.
- **Cost:** two Hobby-tier Vercel projects, same as one — Hobby is free per project, so the split costs nothing beyond the CI check's own build time (one extra `adapter-vercel` build, ~same cost as the existing one). If POLY-77 ever needs Pro-tier cron frequency, that's a per-project upgrade, not a re-architecture.

7.7 Sync Rate Floors

The INFRA minimum interval SECURITY §4.2.1 and §3.1 defer to, for "Sync now" and the per-entity import schedule. Rejected below the floor (SECURITY §4.2.1).

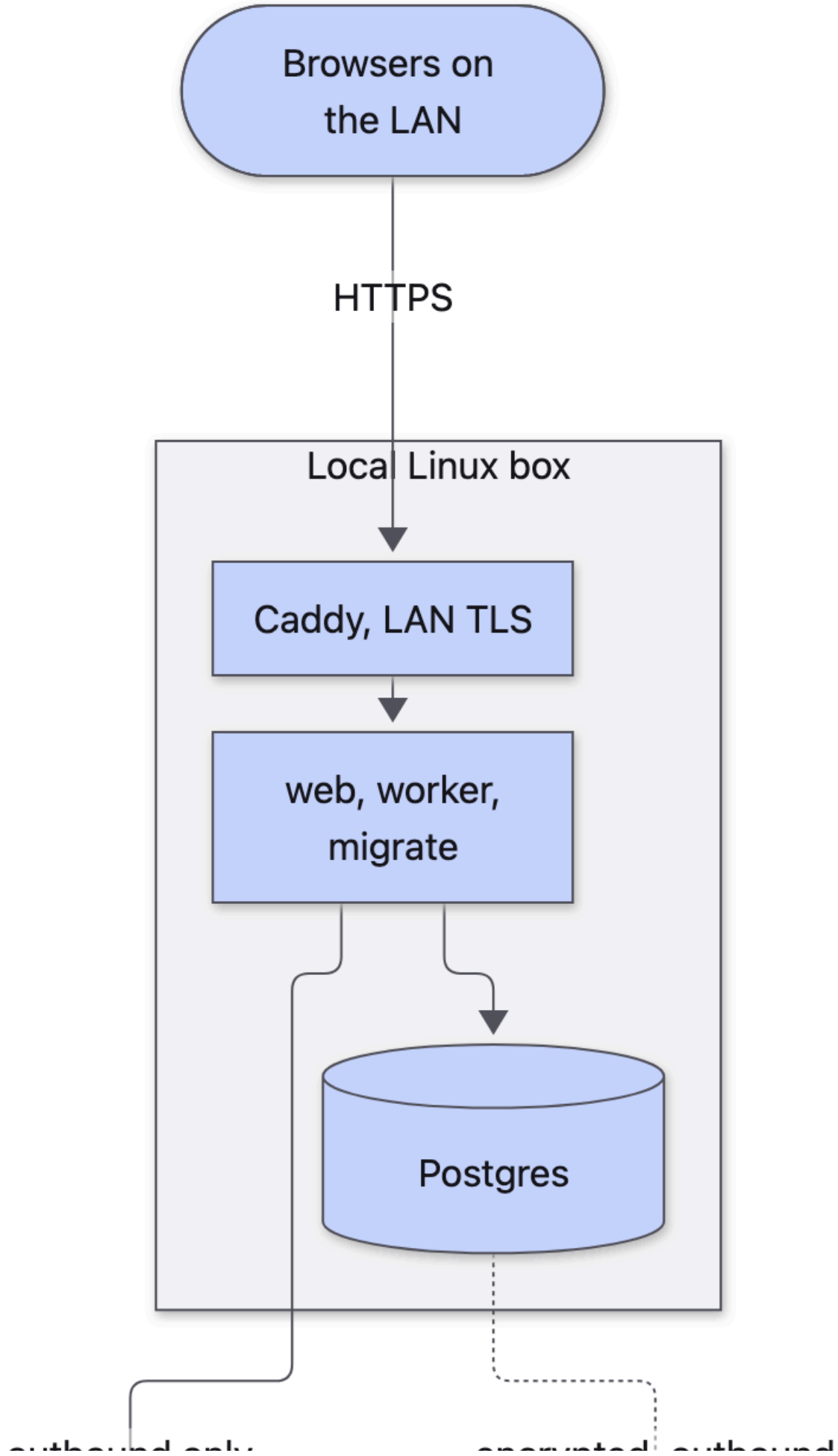
- **Aggregators (SimpleFIN, Plaid) — floor 2 hours, default 6 hours.** Measured (provider docs, 2026-09-28): SimpleFIN Bridge caps a connection at 24 requests/day, with warnings then access-token disablement beyond that — the binding constraint, since it has no per-call cost at all. Plaid's own `/transactions/sync` limit (50 req/min/Item in production) is far looser, and Transactions is billed per active Item monthly, not per call, so Plaid imposes no comparable floor of its own. One shared floor governs both providers rather than branching per-provider in the schedule UI: **2 hours** (≤ 12 scheduled requests/day) leaves ≥ 12/day of SimpleFIN's budget as headroom for manual "Sync now" clicks and backoff retries without risking disablement, even if every connection runs at the floor. The out-of-the-box **default is looser than the floor, 6 hours** (4 syncs/day) — ample freshness for a personal dashboard that isn't real-time trading, and it leaves most of the SimpleFIN budget unused for the common case.
- **Broker APIs (Schwab, IBKR) — floor 1 hour, default 24 hours (EOD), both provisional (unmeasured against a registered app).** Schwab's Trader API publishes generous, per-app-configurable limits (on the order of 120 req/min for market data) with no stated per-call cost; IBKR's are comparably loose for read-only position/lot queries. Neither is the binding constraint for a buy-and-hold personal-investor persona — product reasoning is: lots and positions settle end-of-day, so there is little value polling a brokerage intraday the way a

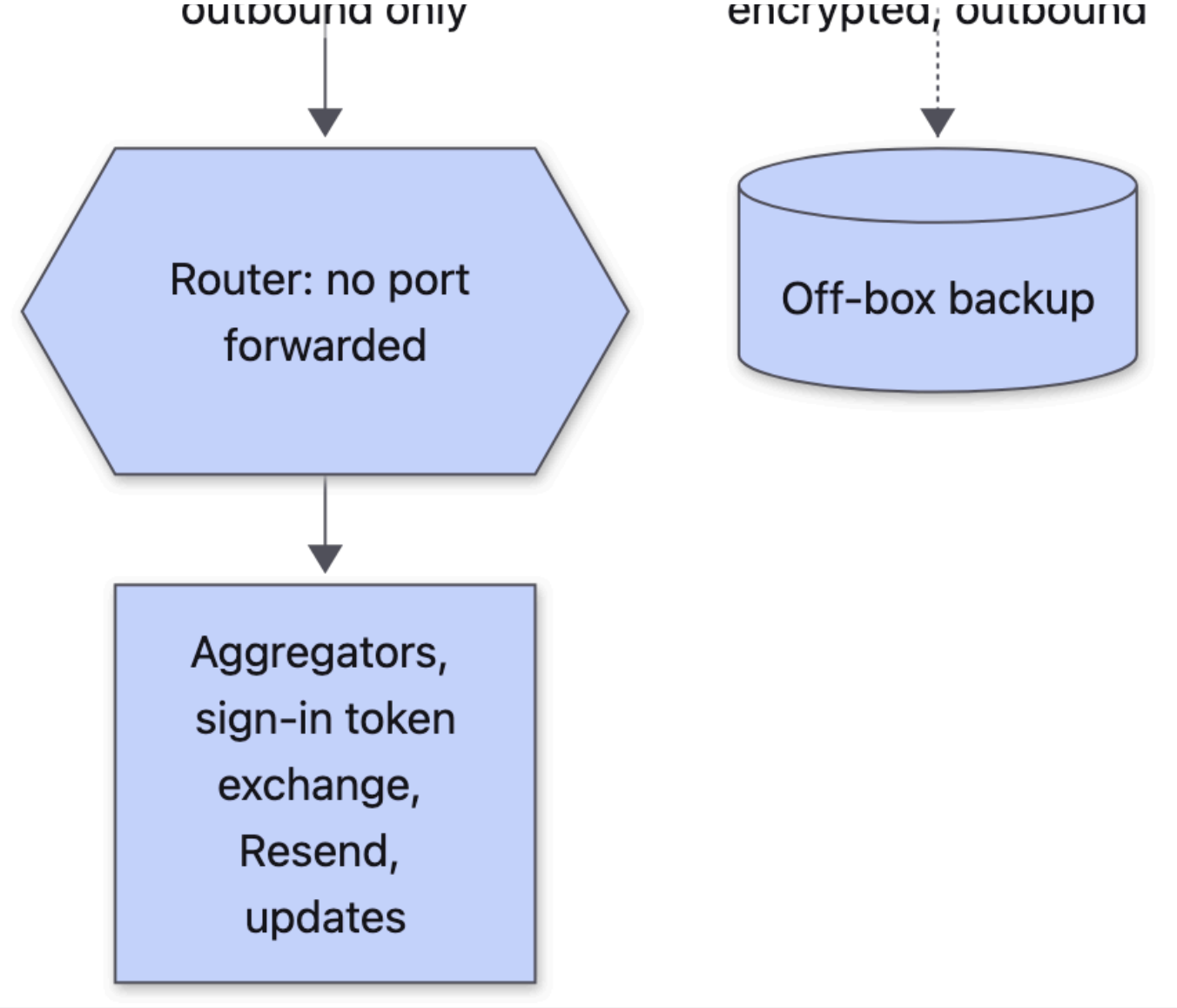
checking account's pending transactions reward polling. Floor is set looser than the aggregator floor (1 hour) because the provider headroom is real, not because the data changes faster; default matches the existing EOD price-provider cadence (§4.4) at once daily. Reopen when POLY-67/69 registers a real developer app and its actual configured limits are known — "provisional" here means the number, not the reasoning, may move.

- **Enforcement:** both floors are a single config value each (`SYNC_FLOOR_AGGREGATOR_SECONDS`, `SYNC_FLOOR_BROKER_SECONDS`), read by the same validation SECURITY §4.2.1 already specifies for "Sync now" and the per-entity schedule UPDATE policy — one seam, not a per-provider branch in application code. A provider added later slots into whichever floor its own rate limit and pricing most resembles; a provider that doesn't fit either (a hard per-call cost, for instance) gets its own floor then, not before.

7.8 Local-Network Secondary Target

Principal, 2026-11-06. The same Compose project on a user-owned Linux box, outbound-only. SECURITY specifies the posture (OQ-24); this section states what changes against §7.


Browsers on the LAN
HTTPS
Local Linux box
Caddy, LAN TLS
web, worker, migrate
Postgres



Concern	On the local box
Ingress	LAN only. The router forwards no port, and the Cloud Firewall (C-71) has no equivalent, so the host firewall does that job. Docker can bypass `ufw`; SecEng specifies the control.
TLS (settled, Principal, 2026-11-07, P7;	HTTP-01 cannot work, since nothing reaches in, so Caddy runs in one of two modes, set by explicit configuration (`TLS_MODE`). **Default, real domain:** a subdomain of a domain the operator owns, resolved only on the LAN (split horizon, no public A or AAAA record), with a DNS-01 wildcard certificate so the box's own name stays out of CT logs; the registrar token is scoped to that one zone and mounted into Caddy only (R-18). **Supported fallback, `.local` with a private CA:** an operator-run CA whose root, name-constrained to the box's name, is generated off the box and kept offline; the box holds only an intermediate issuing short-lived leaves; the root is installed on each

Concern	On the local box
SECURITY C-76)	device (R-20, browser enforcement of the constraint measured at the first home-box deploy). Plain-http sign-in is refused in both modes.
OAuth sign-in	The callback is a redirect in the user's own browser and the token exchange is outbound, so neither needs an inbound path; the hostname is the constraint (Google requires a public-suffix TLD, measured 2026-11-06; Apple a real domain, unmeasured). **Real-domain mode:** Google and Apple work. **Private-CA mode:** the OAuth providers are disabled server-side: the buttons are not rendered and every OAuth route returns 404, so a stale link or a crafted request finds nothing. Sign-in is email, password and the second step (P1); TOTP is always offered, since passkeys on a `.local` RP ID are unmeasured.
Mode cross-check at startup	The app derives the mode it needs from its public origin and refuses to start on a mismatch: a name ending in `.local`, or whose registrable domain is not under a public suffix with a publicly resolvable zone, requires `TLS_MODE=private-ca`; any other name requires the real-domain mode. The check is on the zone, not the host: in real-domain mode the host deliberately has no public record, so "does the name resolve publicly" would misfire (OQ-32). Startup needs outbound DNS, which C-76 already allows.
Moving a `.local` box to a real domain	The origin changes, so every session ends and passkeys, bound to the old RP ID, do not carry over. Before the move every owner and manager must hold an authenticator app or backup codes, and the runbook's move step checks that first. After it, Google sign-in is linked from Settings while signed in; never auto-linked by matching email. The INFRA runbook covers both modes and the move.
Email links	Resend sends outbound as usual, but a verification or magic link only opens from the LAN.
Aggregators	Unchanged. Polling is outbound and the design uses no provider webhooks, so nothing needs to reach in.
Deploy	Unchanged. Pull-based deploy is already outbound-only.
Backups (Princi	Same `age` encryption, shipped nightly outbound to an append-only internet storage box (Hetzner Storage Box or equivalent) on a credential that can add but never delete (C-73). A rotated drive is optional and never kept beside the box. They must leave the

Concern	On the local box
pal, 2026-11-07, P9)	box, because a fire or theft takes the box and anything beside it. Disk unlock is TPM-sealed (P8, C-72; INFRA).
Monitoring	An outside monitor cannot reach in, so the box sends a heartbeat outbound and an absent heartbeat raises the alert.
Access away from home	None. Access from outside the LAN, and from later native clients off Wi-Fi, needs a VPN into the LAN, which is outside v1.
Staging	Not required; the box runs tagged releases that staging already proved.

8. CI/CD Pipeline

Owned by `devops-engineer`; this is the shape, INFRA holds the detail.

Stage	Tool	Gate
Change detection	Turborepo affected graph; docs-only changes skip test jobs	—
Lint + types	ESLint (incl. layer-boundary rules), `svelte-check`, `tsc`	Must pass
Unit / package tests	Vitest, affected packages only	Must pass
DB battery	Apply all migrations to a fresh Postgres as `poly_migrator`; pgTAP RLS + trigger battery (incl. inversion self-test, pool-reuse test)	Must pass when `packages/db` or any schema changes
OpenAPI freshness	Regenerate and diff against the committed spec	No diff
Build	`adapter-node` image; `adapter-vercel` build (both Vercel projects, §7.6)	Both must build
Vercel topology check	Route manifest vs. allowlist per project; no keyring reference in the `web` build; node build 404s job routes; env-name check via Vercel API (§7.6)	Must pass when `apps/web` routes or Vercel project config change
Smoke (integration milestones)	Playwright: login, import a batch, confirm a draft, view a statement	Must pass
Security scan	Dependency audit, secret scan	No high-severity findings
Deploy → staging	Pull-based: the box polls and verifies attestation (§8.1)	Auto on merge to `main`
Deploy → prod	Same mechanism, triggered by a release tag cut by `/merge-pr`	Tag only

8.1 Deploy mechanism

- **Decision, revised per SECURITY C-83: pull-based, not push-over-SSH.** An earlier draft of this section chose GitHub Actions → SSH push; SecEng's explicit preference is pull-based specifically because it means **no GitHub-held credential can ever reach prod** — the box, not a GitHub Actions runner, decides when and what to deploy. A `systemd` timer on the box (every 5 minutes) runs a small poll script: query the GitHub Releases API for the newest `v*` tag (prod) or the registry for the newest `main`-branch image digest (staging); if it differs from what's running, pull the image **by digest** (never a mutable tag), then verify. A failed check at either verification step aborts the poll cycle and alerts (§7.4) — the box never runs an unverified image.
- **Attestation verification, pinned per SecEng's delta (SECURITY C-73/C-83 @ 61b4d2a):** `gh attestation verify <digest> --repo richmosko/polycarpic --signer-workflow <path-to-the-release-workflow>` — pinning the repo and the exact signer workflow, not just "an attestation exists" (an unpinned check would accept a provenance attestation from any workflow in any repo the same GitHub identity can sign for). **Prod adds a second, independent check:** the `v*` tag itself must carry an SSH signature verifiable against a checked-in `allowed_signers` file (the same signing `/merge-pr` already does before cutting a release tag, SECURITY §8 "Signing") — so a compromised registry or attestation service alone still can't move prod without also forging a tag signature staging never checks.
- **The box's GitHub credential:** a fine-grained PAT, scoped to this one repo only, `contents:read` + `packages:read`, with an expiry (rotated per the runbook, same cadence discipline as §6.4's other operator secrets) — never a classic PAT, never org-wide. If the GHCR package and the repo are ever made public, the PAT is dropped entirely: an unauthenticated `gh api` / registry pull needs no credential on the box at all, which is strictly less attack surface than a scoped-but-present one.
- **Build once, promote:** CI builds a single `adapter-node` image per commit, pushes it to GHCR tagged with the git sha, with an SBOM (CycloneDX) and a provenance attestation attached (SECURITY §8); that same image digest is promoted from staging to prod on tag, never rebuilt. This rules out a staging/prod code-drift class of bug. `adapter-vercel` builds separately in CI as the tertiary-target check (§7) but is not deployed by this pipeline. The one image serves three compose services through different entrypoints: `web`, `worker` and `migrate`. Each gets only its own secrets; the credential keyring is mounted into `worker` alone (TB-4), and lint keeps `open()` out of the web bundle.
- **Deploy script** (idempotent, same for staging/prod, invoked by the poll timer once it has a verified digest to move to): `docker compose pull` (by digest) → one-off `migrate` → `docker compose up -d` → poll `/health` with retry → non-zero exit on failure without

swapping traffic (Caddy keeps routing to the last healthy containers until the new ones pass health).

- **Rollback (one button):** a runbook command points the poll script's "desired digest" file at the previous release's digest and re-runs the deploy script directly (no need to wait for the next timer tick, and no dependency on GitHub's Releases API being reachable at rollback time — the operator already knows the prior digest from the deploy log). Migrations are additive / backward-compatible by convention (expand-contract — no destructive migration ships in the same release as the code that depends on the drop), so a code rollback never needs a schema rollback. A schema-level rollback is rare and falls back to the DR restore path (§7.2).
- **Operator secrets** on the box live as files under `/run/secrets/`, one per container that needs it (§7.5) — placed once during VPS bootstrap and never touched by CI or the poll script, which only ever reads a public digest and a public attestation. Neither transmits a secret value onto, or reads one off, the box.

9. Integration Points

Limits marked (unmeasured) are to be verified against provider docs before each adapter's milestone.

Service	Purpose	Auth	Limits	Failure mode
Fake source	Deterministic transactions for three personas; fake EOD walk	none	none	n/a — used in dev, CI and staging
CSV / OFX file import	Credential-free import path (SecEng's preferred default for self-hosters)	none (user upload)	upload size cap	Parse errors surface per row; nothing is posted without confirmation
SimpleFIN Bridge	Fallback aggregator, for institutions Plaid does not cover (e.g. Synchrony Bank). Not used or paid for unless a custodial needs it. A demo setup token gives sample data without an account (measured, developer guide, 2026-11-06).	Setup token → access URL (bearer-equivalent; sealed)	24 requests/day/connection, measured (§7.7) — binding constraint on the shared aggregator sync floor	Job retries with backoff; the dashboard shows "last synced"; no data loss because de-dup is by hash
Plaid	Aggregator for custodials with no file or free direct API. Plaid Sandbox is the pre-GA test source (§7.1).	client id + secret (operator secret) + per-item access token (sealed)	Per-item monthly pricing; 50 req/min/Item on `/transactions/sync`, measured (§7.7) — looser than SimpleFIN's, not the binding constraint	As above; item-login-required surfaces a relink prompt; `/item/remove` on unlink
Schwab / IBKR APIs	Positions, trades, lots. Preferred over Plaid where free, so a trial-tier Item is not spent	OAuth 2.0 / broker-specific	Provisional floor set (§7.7); exact per-app limits unmeasured until POLY-67/69	As above

Service	Purpose	Auth	Limits	Failure mode
			registers a real developer app	
EOD price provider	Daily closes for held securities	API key (operator secret)	Provider-dependent (OQ-5)	Fall back to the last observation, flagged stale; source priority lets a better source replace it later
BLS API v2	CPI-U (CUUR0000SA0), monthly	Free per-operator key (low-tier secret); v1 without a key	500 queries/day with a key (PRD §8)	Use the latest stored month; manual CSV import fallback
Resend	Transactional email (verification, magic link, MFA recovery)	API key (operator secret)	Free-tier volume cap (unmeasured)	Auth email fails loudly; OAuth login still works
Google / Apple / GitHub	OAuth identity providers	OAuth 2.0 client credentials per operator	n/a at this scale	That login method is unavailable; others still work
Off-box backup store	Encrypted backups (Hetzner Storage Box or S3-compatible)	Store credential (operator secret)	Storage quota	Backup job alerts; the RPO clock starts

Data-source priority per custodial (Principal, 2026-11-06): 1 files, 2 a free direct API (e.g. Schwab), 3 Plaid, 4 SimpleFIN. Each custodial account uses the highest-priority source available for it, through the same adapter interface. **Implementation order follows:** fake → files → direct API (Schwab) → Plaid → SimpleFIN. GA needs the files path working; direct and Plaid follow for the custodials the Principal actually holds.

Operator secrets (provider client secrets, API keys, the credential keyring, the DB passwords) live as files on the VPS, each mounted only into the container that needs it (the keyring into `worker` only; SECURITY §6.2). The local box handles them the same way; on the Vercel target they are env vars. They never live in the repo or the database. **User credentials** (access URLs, access tokens) live sealed in the database (§3.1, SECURITY).

10. Trade-offs & Alternatives

- **Home-grown GL over pgledger.** We own correctness, and in exchange we get a model that fits (multi-leg, tenancy, multi-currency-ready lines). §6.1.
- **Drizzle over dbmate.** One TS schema for queries and migrations, plus BetterAuth in the same chain, at the cost of hand-written files for triggers and functions and an unproven policy differ. The spike gates the choice. §6.2.
- **Transaction-local RLS context over per-user DB roles or session `SET`.** Per-user roles don't scale with memberships and complicate pooling; session `SET` leaks across pooled connections. `set_config(…, true)` is pool-safe on both targets.
- **Import split into raw (L2) and posting (L4).** This honours "import is the lowest layer" for raw data while keeping dependencies strictly downward. It costs one extra package.
- **One entry per entity for cross-entity transfers.** Each entity's books balance on their own; there are no cross-entity entries. The cost is that matching must pair across entities and post twice (§4.3).
- **Signed single amount per line over debit/credit columns.** The balance check becomes a single `SUM = 0`; presentation derives Dr/Cr from sign and the account's normal side.
- **Balances computed, not stored.** No running-balance column until a measured need. That removes a whole class of lock contention and drift bugs, at the cost of a query that grows with history (indexed).
- **pg-boss (Postgres) over Redis for jobs.** No extra service. The cost is job throughput we will never need.
- **Web pages call packages directly, not through `/api/v1`.** This skips a serialization hop and keeps SSR fast. The two paths share DTO types, so they can't drift; the cost is that the web UI doesn't dogfood the HTTP API (the Playwright smoke and the native client will).
- **Staging on the prod VPS.** Saves a second box. Staging holds no real books, so a staging compromise exposes none, but the two share a kernel. INFRA may split them later.
- **Pull-based deploy over push-over-SSH (SECURITY C-83).** No credential GitHub holds can ever reach prod directly, at the cost of a systemd timer on the box and up to a five-minute delay between a tag landing and the box noticing it (acceptable for a single-operator deployment with no SLA).
- **Age-recipient backup encryption over restic's symmetric password (SECURITY §6.6).** A compromised box can encrypt new backups but never decrypt any backup, past or future — the cost is giving up restic's deduplication, negligible at this data volume.
- **Period closes over per-entry or year-end accrual.** Liabilities stay current without correcting earlier entries, at the cost of a user step each period and one computed line between closes.

§4.5.

- **Chart by nature with custodian sub-accounts (Option C).** Allocation's top level and the ledger agree by construction, and a custodial keeps its own view. The cost is more accounts, plus an invariant tying lots to the ledger that every securities write must keep. §3.6.
- **An effective-dated, append-only cap table over a mutable share table.** Any past allocation or economic figure is reproducible, and a mid-year change splits the year correctly, at the cost of a "latest row per holder on a date" query on every allocation read. Equity sits apart from access, so a co-owner needs no purge or export rights to hold units. §3.1a.
- **Refusing entity chains deeper than one link over composing them or treating them as outside.** Tax never silently drops income and the composition stays one join, at the cost of not modelling a holding company inside a trust in v1. §3.1a.
- **A sibling economic function over a weighted mode inside `app.rollup_totals`.** The gross function's shape and tests stay untouched and the economic one is reviewed on its own, at the cost of two definers with one guard to keep in step. §3.3c.
- **Recursive CTE over a closure table for nesting.** One copy of the structure and no path counting on delete, at the cost of walking the graph on each roll-up read. §3.3b.
- **Locked counter row over a sequence for the Journal ID.** Gapless numbers, as accountants expect, at the cost of serializing postings per entity. §3.2.7.
- **Collect deletion choices, then apply.** Abort stays a true abort, at the cost of one more screen and a run that can stop midway on a failed revocation. §3.4a.
- **Reports as data (declarative) over code.** Users can clone and edit, and upgrades don't clobber clones. The cost is building a small formula grammar. Scripting stays deferred (PRD §3).

11. Milestone Map

Development milestones under `POLY-V1`, ordered by the layer model. The files in `process/cairn/milestones/` are authoritative; this is the rationale.

Milestone	Layers	Stories	Integration check at close
POLY-M1 — Foundations	L0, L1, L6 shell	POLY-76 (spike), POLY-62	A user signs in, creates an entity, grants a viewer; the RLS battery is green; the multi-owner gate and the founder trigger refuse as §3.1b says; migrate runs in CI and on staging
POLY-M2 — Ledger, Import & Dashboard	L2, L3, L4, first L6 screens	POLY-64, POLY-65, POLY-63; the ledger hash chain and its verifier (C-24, §3.2.9) at decomposition	The fake persona imports, drafts are confirmed into balanced, hash-chained immutable entries, and the verifier passes, categories / tax tags apply, and the per-entity dashboard shows them in under 2 s
POLY-M3 — Transfers & Reconciliation	L4, L6	POLY-71, POLY-72	The couple persona's transfers net CIT to zero; the custodial reconciliation matches
POLY-M4 — Securities & Market Data	L2 refdata, L3 securities, L5 portfolio	POLY-67, POLY-69, POLY-75	Persona 3's lots, EOD valuation and allocation are correct; the real / nominal toggle works

Milestone	Layers	Stories	Integration check at close
POLY-M5 — Statements	L5 reports	POLY-73, POLY-68	Seeded templates render; clones survive a template upgrade
POLY-1.0 — Tax, Sharing & Real Data (GA)	L5 tax, L6, L2 real adapter	POLY-66, POLY-70 (with co-owner approval and the founding owner, §3.1b), POLY-77 (file import as the real-data path; direct and Plaid follow per §9); purge of a cap-table holder (§3.4b)	Tags `v1.0.0`: a backup restore drill passes (GA gate, in place of early real data), then the Principal runs one entity end to end on real data through the files path (PRD goal 1)
POLY-1.1 — Consolidated Roll-up (planned, file not yet created)	L5 reports, L6	To be filed: `app.rollup_totals`, the consolidated view, the parent holdings popover, `app.rollup_economic` and the gross / economic toggle	Tags `v1.1.0`: a parent viewer sees totals equal to the parent's own accounts plus the entities beneath it, each counted once at any depth, and a drill stops at the boundary. In the economic view, every holder's figures plus the outside owners' share sum to the gross figure, one spouse's separate trust adds nothing to the other's economic net worth, and the economic net worth sums the parent's roll-up set only

First slice (Principal, 2026-11-06): story 1 (invite sign-in), story 2 (per-entity dashboard) and transaction import through the fake source. It runs from POLY-M1 through POLY-M2, which is why the dashboard moves up from POLY-M3: it needs posted entries to show, so it follows the ledger and import rather than preceding them. The order is POLY-62, the ledger and POLY-64, then POLY-63. The rows above state the intended shape; the milestone files carry it once the tracker is updated.

Roll-up follow-up: the parent link and membership ship in POLY-M1. POLY-1.1 comes immediately after GA and adds the view.

12. Open Questions

Each has an owner and a target resolution point.

#	Question	Owner	Resolve by
OQ-1	*Resolved, revised by SecEng review 70c9d05 (SECURITY §6.3–§6.5):* HPKE (X25519 / HKDF-SHA256 / AES-256-GCM). `web` seals; only `worker` holds the keyring and opens. Stored as `enc`, `ciphertext`, `key_id`, `alg_version`. pgcrypto and KMS are rejected. Rotation is a re-seal job.	seceng	Closed 2026-09-28
OQ-2	*Resolved (SECURITY §12, §4.2.1, §4.3):* the §3.3 shape is confirmed, with additions (a)–(l) folded in. Roles are uniform across entity types.	seceng + qa-engineer	Closed 2026-09-28
OQ-3	*Resolved (SECURITY §4.4):* jobs run under the linking user's membership (`linked_by_user_id NOT NULL`), with a `principal_invalid` state that fails loudly. The system policy clause was rejected.	seceng	Closed 2026-09-28
OQ-4	*Resolved, revised by 70c9d05 (SECURITY §4.5):* the purge function is owned by the NOLOGIN `poly_purge` role, with role-targeted DELETE policies. Eight guards, including export freshness and a zero-rows-remain assert. See §3.4.	seceng	Closed 2026-09-28
OQ-5	Which EOD price provider (free tier, coverage of mutual funds / ETFs)? mosko-fintech used provider-implied prices plus FMP.	architect	Start of POLY-M4
OQ-6	*Resolved (Principal, 2026-11-06):* the user picks per transfer, contribution / distribution preselected, the other choice being due-to / due-from, explained in plain language at the point of choice. See §4.3.	Principal (via team-lead)	Closed 2026-11-06

#	Question	Owner	Resolve by
OQ-7	*Resolved (Principal, 2026-11-06):* data-source priority per custodial is files, free direct API, Plaid, SimpleFIN, and implementation follows fake → files → direct → Plaid → SimpleFIN. GA needs the files path. See §9. POLY-77 is scoped to files.	Principal (via team-lead)	Closed 2026-11-06
OQ-14	*Resolved (Principal, 2026-11-06):* daily only. Groups span one bank date (gate G3).	Principal (via team-lead)	Closed 2026-11-06
OQ-15	Operator-only Admin view (POLY-81, Principal direction 2026-10-02). It shows basic box and stack status (container health, last migration, backup age, disk, link-health counts) plus admin links still to be defined, and is reachable only through an SSH tunnel to a loopback-bound listener, never through Caddy. Topology options: (a) a route group inside the web process on a second listener bound to `127.0.0.1`, which is cheap but shares the web process and its attack surface; (b) a separate `admin` compose service bound to loopback with its own minimal DB role (read-only status views, no tenant data), which costs one service but isolates it. What it reads, and whether it may act (trigger a backup, rotate keys), is open. Leaning (b), pending SecEng.	architect + secceng + devops-engineer	Under POLY-81
OQ-8	*Resolved (Principal, 2026-11-06):* no real books before v1.0. Everything before GA runs on staging, with Plaid Sandbox and SimpleFIN's demo token through the real adapters and the fake source as the fixture. A backup restore drill is a GA gate. Any `0.x` tag before GA is a staging release.	Principal (via team-lead)	Closed 2026-11-06
OQ-9	**Resolved** — see §7.2: staging co-located on the prod VPS; backup store is Hetzner Storage Box (S3-compatible as a config-only alternative); RPO ≤ 15 min (WAL) / 24h (nightly dump fallback), RTO ≤ 4h (full loss) / ≤ 30 min (bad-migration rollback).	devops-engineer	INFRA v1 — done 2026-09-28

#	Question	Owner	Resolve by
OQ-10	Drizzle spike outcome (§6.2), which confirms or flips the migration-tool decision.	architect + backend-lead	POLY-76, first issue of POLY-M1
OQ-11	*Resolved in outline (§2.4):* the six DESIGN screens are mapped to package DTOs. Field-level DTO shapes get pinned in each package's design file at the start of its milestone.	ux-designer + architect	Per milestone
OQ-12	*Resolved (Principal, 2026-11-06):* typed per entity, per jurisdiction, per year, with the ledger's settled-filing figure beside it as a check once it exists. See §4.5.	architect + Principal	Closed 2026-11-06
OQ-13	**Resolved** — see §7.6: the Vercel target is two projects from one repo (SECURITY C-61), `web` holding public keys + `poly_app` only, `jobs` holding the keyring + `poly_worker`. CI enforces the split (route manifest, bundle scan, node-build 404, env-name check). Hobby-plan cron is capped at once daily — measured, accepted as this fallback target's data-freshness trade-off.	seceng + devops-engineer	Before POLY-77 — done 2026-09-28
OQ-16	Does BetterAuth's `user.create.after` hook run inside the user insert's transaction? If yes, the Person-at-sign-up provisioning (§3.1) is atomic and the session-loader re-run becomes a pure backstop; if no, the re-run is the guarantee. Add as an acceptance item to the M1 Drizzle/BetterAuth spike.	architect + backend-lead	POLY-76
OQ	*Resolved (Principal, 2026-11-06):* either owner suffices, with visibility: the child's owner sees the names of parents that include their entity and	Principal	Closed

#	Question	Owner	Resolve by
-17	nothing else about them. See §3.1. SecEng confirms under OQ-25.	(via team-lead) + secеng	2026-11-06
OQ-18	*Resolved (Principal, 2026-11-06):* class totals plus income and expense by top-level category (§3.3a). SecEng reviews the exposure given nesting depth under OQ-25.	Principal + secеng	Closed 2026-11-06
OQ-19	*Resolved (SECURITY §12.1, C-19 amended, C-25):* the owner-on-both insert rule stays; only `poly_rollup`'s role-targeted policies read `entity_rollup`, behind fixed aggregate shapes; the one-bit refusal is accepted (R-15). Finding folded: the graph triggers move to `poly_graph` (§3.3b).	secеng	Closed 2026-11-07
OQ-20	*Resolved (Principal, 2026-11-06):* yes, a parent may hold accounts, and the refusal rule is dropped. The roll-up adds a parent's own accounts to its sum.	Principal (via team-lead)	Closed 2026-11-06
OQ-21	*Resolved (Principal, 2026-11-06):* notify and allow pull-out. Agreeing to a parent accepts its later placement; every owner beneath is notified and can remove their entity. See §3.3b. SecEng confirms under OQ-25.	Principal (via team-lead) + secеng	Closed 2026-11-06
OQ	*Resolved (Principal, 2026-11-06):* Option C: the ledger is organised by nature with a custodian sub-account under each nature account,	Principal	Closed

#	Question	Owner	Resolve by
-22	`custodial_account` is its own table, and lots reference it. See §3.6.	(via team-lead)	2026-11-06
OQ-23	*Resolved (Principal, 2026-11-06):* no delay. Existing guards stand, and every member is emailed immediately. SecEng's rationale was presented alongside. See §3.4.	Principal (via team-lead) + seceng	Closed 2026-11-06
OQ-24	*Resolved (SECURITY C-76; Principal, 2026-11-07, P7–P9):* host nftables default-deny with a LAN-only allow, Caddy bound to the LAN address, outbound DNS / NTP / HTTPS only, an outbound heartbeat; TLS in two modes, a real domain by default and a `.local` private CA as fallback with OAuth disabled; TPM-sealed disk unlock; append-only off-box backups. See §7.8.	seceng + devops-engineer	Closed 2026-11-07
OQ-25	*Resolved (SECURITY v1.1 §12, §3.8, §4.6; C-25, C-27, C-52, C-77):* every item confirmed, with the shapes folded here: junction read and `app.parents_of` (§3.1); upward consent and the ancestor-invite notice (§3.3b); the grain unchanged at depth; the popover on ordinary RLS (§3.3a); cap-table reads through `poly_stake` and `app.my_stakes` (§3.1a); the economic view's disclosure (§3.3c); allocated income (§4.5); the deletion purge invoker (§3.4a). SecEng's new asks are A-16 to A-22, folded in v1.4.	seceng	Closed 2026-11-07
OQ-27	*Resolved (Principal, 2026-11-06):* realized-gain income accounts split by holding period only. Jurisdiction lives on the tax side: the close posts one expense and payable pair per jurisdiction, and prepaid accounts settle the payable at filing. See §3.6 and §4.5.	Principal (via team-lead)	Closed 2026-11-06

#	Question	Owner	Resolve by
OQ-28	*Resolved (Principal, 2026-11-06):* parents carry the same settable classification, with `joint filers` (accrues its own payable) or `pass-through to members` (default; income allocates to holders by the cap table, which replaced the share table on 2026-11-07). See §4.5. A combined filing group across a Household and its Persons is not in v1; it is a follow-up on the CPA item (PRD §10).	Principal (via team-lead)	Closed 2026-11-06; filing group with the CPA item
OQ-29	*Resolved (Principal, 2026-11-07):* in POLY-1.1 the economic view and economic net worth weight only the selected parent's roll-up set (§3.3c). A cross-parent "my net worth" is a later option that needs SecEng.	Principal (via team-lead) + secеng	Closed 2026-11-07
OQ-30	The numbering trigger (§3.2.7) needs a definer identity, since `poly_app` has no write grant on the counter; v1.3 left it unnamed. The chain trigger needs the same. Proposed: one NOLOGIN, NOBYPASSRLS role `poly_journal` owning both, with UPDATE on `entity_journal_counter`, INSERT on `journal_chain`, and SELECT on `journal_entry` / `journal_line`, justified as for `poly_graph`. SecEng confirms or reshapes.	secеng	Before POLY-64 (the ledger)
OQ-31	Direct-path invitations outstanding when a sole owner's entity gains its second owner: §3.1b revokes them at the 1 → 2 transition, so an owner invite pre-issued alone cannot land after the approval rule applies. SECURITY §4.2.2's bootstrap edge does not cover it. SecEng confirms.	secеng	Before POLY-70
OQ-	SECURITY C-76's startup cross-check says a name "that does not resolve publicly" must be in private-CA mode, but in the default real-domain mode the host deliberately has no public record. §7.8 checks the zone instead: a	secеng	Before the first

#	Question	Owner	Resolve by
32	public suffix with a publicly resolvable zone means real-domain mode. SecEng confirms the predicate.		local-box deploy
OQ-33	*Resolved (Principal, 2026-11-07), option (c):* a purged holder shows as "Former holder (purged <date>)", plus an optional label the other entity's owners type themselves. The system copies nothing of the purged entity. The label is that entity's own data, purged only with it, and counts as a cap-table edit, so the approval gate applies on a multi-owner entity. See §3.4b.	Principal (via team-lead)	Closed 2026-11-07
OQ-26	Backups on the Vercel target: §7.2 describes the Compose path only, since a cron function cannot run `pg_dump` and WAL shipping. Neon's own restore window, or a scheduled export from outside? No RPO is stated for this target.	devops-engineer	Before the Vercel target carries real data

Source hand-offs folded in: PRD-import spillover (kickoff §2.4–§2.11 architecture detail) and SecEng's Research-phase notes. Their [Plan] items surface here as OQ-1 through OQ-4, and in §3.3 / §3.4.